

从0到1

OceanBase 原生分布式数据库 内核实战基础版



OceanBase 研发团队

联系我们

欢迎 OceanBase 爱好者、用户和客户联系我们反馈问题：

- 社区版官网论坛：<https://ask.oceanbase.com/>
- 社区版项目网站提 Issue
 - MiniOB GitHub: <https://github.com/oceanbase/miniob/issues>
 - OceanBase GitHub: <https://github.com/oceanbase/oceanbase/issues>
 - OceanBase Gitee: <https://gitee.com/oceanbase/oceanbase/issues>
- 技术交流及关注动态：群号 33254054



钉钉技术交流群
(群号：33254054)



微信 OB 小助手



基础视频教程

目 录

第 1 章：数据库系统概述	5
1.1 数据库系统的发展历史	6
1.2 数据库系统架构	10
1.3 MiniOB 概述	13
1.4 MiniOB Gitee 使用说明	15
1.5 MiniOB 开发调试环境搭建	19
1.6 MiniOB 线程模型	27
1.7 课后实践	33
第 2 章：数据库的存储结构	34
2.1 存储器的层次结构	35
2.2 磁盘存储器	38
2.3 块与记录组织	41
2.4 变长数据和记录	44
2.5 记录的修改	48
2.6 MiniOB 存储实现原理	50
2.7 课后实践	55
第 3 章：索引结构	56
3.1 B+Tree	57
3.2 散列表	64
3.3 LSM-Tree	68
3.4 MiniOB B+Tree 实现	71
3.5 课后实践	76
第 4 章：SQL 引擎	77
4.1 SQL 层整体架构	78
4.2 Parser 和 Resolver 模块	80
4.3 Transformer 和 optimizer 模块	83

4.4 Executor 模块	86
4.5 Fast-parser 和 Plan cache 模块	89
4.6 基础代数符号与操作介绍	90
4.7 关系表达式的等价规则转换	95
4.8 执行计划的选择	97
4.9 代价	99
4.10 课后实践	101
第 5 章：事务引擎	102
5.1 事务简介	103
5.2 故障类型和事务模型	104
5.3 undo 日志	107
5.4 redo 日志	114
5.5 undo/redo 日志	118
5.6 隔离级别	122
5.7 锁	124
5.8 时间戳	127
5.9 多版本并发控制	129
5.10 课后实践	132

第 1 章：数据库系统概述

本章主要介绍数据库系统概述、MiniOB 概述及 MiniOB 研发环境搭建。

本章目录

- 1.1 数据库系统的发展历史
- 1.2 数据库系统架构
- 1.3 MiniOB 概述
- 1.4 MiniOB Gitee 使用说明
- 1.5 MiniOB 开发调试环境搭建
- 1.6 MiniOB 线程模型
- 1.7 课后实践

1.1 数据库系统的发展历史

背景

在 20 世纪 60 年代之前，面对科学计算必须要解决一个问题：怎么来解决问题，解决应用的需求，具体体现在以下几个方面：

1. 怎么实现一个持久化，当出现故障的时候，从故障中恢复？
2. 如何实现并发，当有很多并发的时候，怎么去解决？
3. 如何定义数据的格式？
4. 如何定义数据的访问接口？

在数据库还没有出现前，基本上就是应用程序直接来操作底层的文件。利用文件系统将数据持久化，自己设计并发控制、故障恢复，自己定义文件格式和访问方式来设计数据的逻辑结构、数据物理结构、存储结构、存取方法和输入方法等。

但是这种情况下，很容易出现问题，并且在做迁移或者改动时都比较麻烦。因此，20 世纪 60 年代，数据库应运而生。

相关概念

- 数据库系统：带有数据库的计算机系统，一般由数据库、数据库管理系统、相关的硬件、软件和各类人员组成。
- 数据库：按照数据结构来组织存储管理数据的仓库。
- 数据库管理系统：数据库系统的核心组成部分，它是介于用户与操作系统之间的一层数据管理软件，是用户和数据库的接口。
- 数据模型：描述数据、数据联系、数据语义以及一致性约束的概念工具的集合。

数据库模型

在数据库技术领域，数据库所使用的典型数据模型主要有层次数据模型（Hierarchical Data Model）、网状数据模型（Network Data Model）和关系数据模型（Relational Data Model）。这三种模型是按照它们的数据结构来命名的，它们之间的根本区别就在于数据之间联系的表达方式不同。

层次模型

层次模型是最早用于商用数据库管理系统的数据模型。在数据库中定义满足以下两个条件的记录以及它们之间联系

的集合为层次模型:

1. 有且只有一个结点没有双亲结点, 这个结点称为根结点;
2. 根结点以外的其他结点有且只有一个双亲结点。

层次数据模型中最基本的数据关系是基本层次关系, 它代表两条记录之间一对多 (包括一对一) 的联系。层次模型类似于今天的树状模型, 非常适合于关联的, 就是类似于树状的这种查询。但是在离开树状查询之外, 其实它的应用场景会非常的受局限。

网状模型

网状模型是一种可以灵活地描述事物及其之间关系的数据库模型。最早由美国的查尔斯·巴赫曼发明。

网状模型的数据结构主要有以下两个特征:

1. 允许有一个以上的节点无双亲。
2. 至少有一个节点可以有多个的双亲。

网状模型中每个结点表示一个记录型 (实体), 每个记录型可包含若干个字段 (实体的属性), 结点间的连线表示记录类型 (实体) 间的父子关系。

从上述特征中可以看出, 层次模型中子结点与双亲结点的联系是唯一的, 而在网状模型中这种联系可以不唯一。因此, 在网状模型中要为每个联系命名, 并指出与该联系有关的双亲记录和子记录。

关系模型

关系模式实际就是记录类型, 包括: 模式名、属性名、值域名及模式的主键。关系模型利用表的集合来表示数据和数据间的联系, 在逻辑层和视图层描述数据, 使用户不必关注数据存储的底层细节。

关系模型有很多的优点:

- 建立在严格的数学概念的基础上, 有理论基础
- 数据模型清晰, 所有模型都使用关系来表示
- 接口非常清晰, 逻辑层和物理层非常清晰的剥离

所以关系模型最后成为真正的数据库的主流模型一直延续到今天。

数据库的发展

接下来将以数据模型和时间为依据, 介绍数据库的发展过程。

网状数据库和层次数据库

网状数据库是数据库历史上的第一代产品, 它成功地将数据从应用程序中独立出来并进行集中管理, 网状数据库是

基于网状数据模型建立数据之间的联系，能反映现实世界中信息的关联，是许多空间对象的自然表达形式。

1964 年，通用电气的工程师 Charles W. Bachman 领导开发了全球第一个数据库管理系统——IDS（Integrated Data Storage，集成数据存储），并正式推出。IDS 是网状数据库，奠定了数据库发展的基础，在当时得到了广泛的应用。IDS 具有数据模式和日志的特征，但它只能运行于通用电气的主机上，且数据库只有一个文件，所有的表必须通过手工编码生成，有着极大的局限性。5 年后，美国数据库系统语言协会（Conference on Data Systems Languages, CODASYL）下属的数据库任务组（Database Task Group, DBTG）发布了一份报告，阐述了网状数据库系统的许多概念、方法和技术，成了网状数据库的代表。

1968 年，IBM 公司 McGee 等人开发的层次式数据库系统的 IMS（Information Management System，信息管理系统）发表，它可以让多个程序共享数据库。IMS 是世界上第一个层次数据库系统，也是世界上第一个大型商用的数据库系统。

1973 年，Charles W. Bachman 获得图灵奖，以表彰他在数据库领域，尤其是在网状数据库管理系统方面的杰出贡献。

但网状数据库也存在一些问题。首先，网状模型和层次模型描述世界的方式非常受限，比如层次模型就仅仅适合于树状模型。其次，这两种模型的接口层实际上是非常局限的，没有高级查询语言，用户在复杂的网状结构中进行查询和定位操作比较困难。所以关系数据库应运而生。

关系数据库

1970 年，IBM 的研究员 Edgar F. Codd 发表了《A Relational Model of Data for Large Shared Data Banks》论文，提出了关系数据模型的概念，奠定了关系数据模型的理论基础，这是数据库发展史上具有划时代意义的里程碑。随后，Edgar F. Codd 又陆续发表了多篇文章，论述了范式理论，用数学理论奠定了关系数据库的基础，为关系数据库建立了一个数据模型——关系数据模型。Edgar F. Codd 也于 1981 年获得图灵奖。

1973 年，加州大学伯克利分校的 Michael Stonebraker 和 Eugene Wong 利用 IBM 公司已发布的信息，以及关系模型的理论，开始开发自己的关系数据库系统 Ingres。

1976 年，霍尼韦尔公司（Honeywell）开发了世界上第一个商用关系数据库系统——Multics Relational Data Store。

1974 年 IBM 的 Ray Boyce 和 Don Chamberlin 将 Edgar F. Codd 论述的关系数据库的 12 条准则的数学定义以简单的关键字语法表现出来，里程碑式地提出了 SQL（Structured Query Language，结构化查询语言）。

1978 年，Larry Ellison 在为美国中央情报局做一个数据项目的时候，敏锐地发现关系数据库的商机。几个月后，Oracle 1.0 诞生了，当时的 Oracle 除了完成简单关系查询之外，不能做任何事情。但是经过短短十几年，Oracle 公司的数据库产品不断发展成熟，成为了数据库行业的巨头。至此，关系数据模型的理论才通过 SQL 在商业数据库 Oracle 中使用。

NoSQL 数据库

20 世纪以来，非关系型数据库（NoSQL）开始盛行，NoSQL 数据库主要包括 4 种类型：文档数据库、列簇式数

据库、键值数据库和图数据库。

Google 于 2003 年至 2006 年发布的三篇论文，分别是《Google File System》、《Google Big table》和《Google MapReduce》，奠定了分布式数据系统基础，其中《Google BigTable》一篇中介绍了 NoSQL 数据库。

NoSQL 数据库支持分布式存储，数据可以无限的线性扩张，解决了过去传统数据库存储容量不能够线性扩张的问题。并且 NoSQL 底层的结构要求不一定是关系模型。NoSQL 没有预定义的模式。同时 NoSQL 还支持对象存储，同时为了更高效的存储，NoSQL 还提供 API 的一些访问机制。

NewSQL 数据库

NewSQL 是对各种新的可扩展/高性能数据库的简称，这类数据库不仅具有 NoSQL 对海量数据的存储管理能力，还保持了传统数据库支持 ACID 和 SQL 等特性。

NewSQL 系统虽然在的内部结构变化很大，但是它们有两个显著的共同特点：都支持关系数据模型，都使用 SQL 作为其主要的接口。

目前已知的第一个 NewSQL 系统叫做 H-Store，是由美国布朗大学，麻省理工学院，和耶鲁大学联合开发的，为在线事务处理应用而设计的数据库管理系统。它是一个分布式并行内存数据库系统。

1.2 数据库系统架构

整个关系数据库大概分为三大块：存储、事务和 SQL。

- 存储：相当于关系数据库的数据结构。
- 事务：相当于关系数据库的算法。
- SQL：相当于关系数据库操作的描述语言，其实就是相当于接口。SQL 在数据库实现中相当于接口的实现。

存储

存储在工业界基本上最常见的两种基础的数据结构：哈希表、B+ 树。

哈希表可以实现 $O(1)$ 的这种计算，支持 put/get，哈希表 put/get 的性能非常快。唯一的缺陷是不支持范围 scan。

另外一种最常见的是 B+ 树，也就是今天关系数据库里面用的最多的一种方式。B+ 树最大的优势它可以做范围 scan，并且它是一个平衡的二叉树，它的读、写、删除都非常的均衡，不会出现读性能特别好，写性能特别差或者读性能特别差，写性能特别好的情况，是一个非常均衡的算法。

最近一二十年出现了一个叫做 LSM-Tree 的基础数据结构。LSM-Tree 最大的特点就是它会更适合于写多读少的场景，其核心思想就是将离散的随机写请求都转换成批量的顺序写请求，大幅度减少随机写的这种随机操作，提高写的性能。

数据库存储就是面向磁盘设计的一颗 B+ 树。

事务

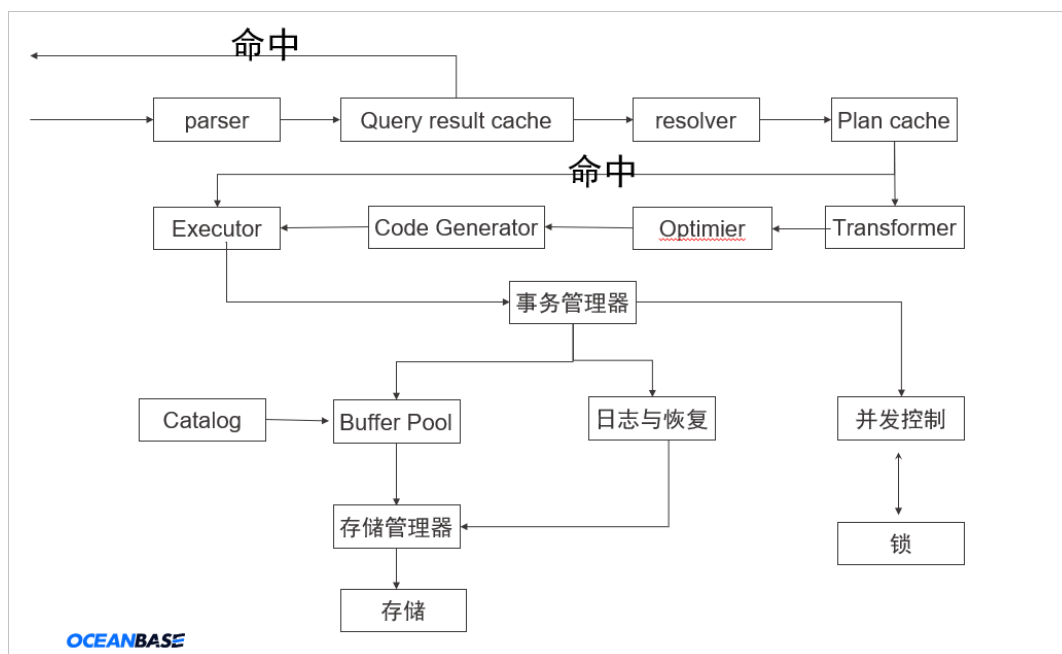
事务就相当于关系数据库的算法。是整个数据库中最难的部分。事务里面最核心的有四个概念。

- 原子性 (Atomicity)：事务操作要么全部成功，要么全部失败。即一旦事务出错，就回滚事务。
- 一致性 (Consistency)：一个事务只能使数据库从一个一致的状态跳到另一个一致的状态，不能存在一个中间的状态。
- 隔离性 (Isolation)：多个并发事务互相不影响，就如同多个事务串行执行一般。
- 持久性 (Duration)：事务一旦提交成功对数据库的影响就是永久的。

SQL

SQL 就相当于关系数据库的操作语言，它其实就是一层基于关系代数的接口，并且非常有理论基础。在数据库内部他们有专门一个模块（SQL 引擎）来实现这一套接口。

执行流程



SQL 层

下表为 SQL 请求执行流程的步骤说明。

步骤	说明
Parser（词法/语法解析模块）	在收到用户发送的 SQL 请求串后，Parser 会做词法/语法分析，将字符串分成一个个的“单词”，并根据预先设定好的语法规则解析整个请求，将 SQL 请求字符串转换成带有语法结构信息的内存数据结构，称为语法树（Syntax Tree）。
Query result cache	直接对 SQL 进行硬解析，若发现命中，将直接绕过下面所有过程，返回结果给应用方。
Resolver（语义解析模块）	Resolver 将生成的语法树转换为带有数据库语义信息的内部数据结构。在这一过程中，Resolver 将根据数据库元信息将 SQL 请求中的 Token 翻译成对应的对象（例如库、表、列、索引等），生成的数据结构叫做 Statement Tree。
Plan Cache（执行计划缓存模块）	执行计划缓存模块会将该 SQL 第一次生成的执行计划缓存在内存中，后续的执行可以反复执行这个计划，避免了重复查询优化的过程。Resolver 中生成的 Statement Tree 将会与该模块中的执行计划做匹配，若匹配命中，Plan Cache 直接将其物理执行计划发送到 Executor 中进行执行。
Transformer（逻辑改写模块）	分析用户 SQL 的语义，并根据内部的规则或代价模型，将用户 SQL 改写为与之等价的其它形式，并将其提供给后续的优化器做进一步的优化。Transformer 的工作方式是在原 S

块)	atement Tree 上做等价变换, 变换的结果仍然是一棵 Statement Tree。
Optimizer (优化器)	优化器是整个 SQL 请求优化的核心, 其作用是为 SQL 请求生成最佳的执行计划。在优化过程中, 优化器需要综合考虑 SQL 请求的语义、对象数据特征、对象物理分布等多方面因素, 解决访问路径选择、联接顺序选择、联接算法选择、分布式计划生成等多个核心问题, 最终选择一个对应该 SQL 的最佳执行计划。
Code Generator (代码生成器)	将多个算子合并到一起, 生成一个更高效的算子。
Executor (执行器)	启动 SQL 的执行过程。 - 对于本地执行计划, Executor 会简单的从执行计划的顶端的算子开始调用, 根据算子自身的逻辑完成整个执行的过程, 并返回执行结果。 - 对于远程或分布式计划, 将执行树分成多个可以调度的子计划, 并通过 RPC 将其发送给相关的节点去执行。

事务层和存储层

事务层和存储层在真实的工业界并没有上图中切分的那么干净, 很多时候其实是相互掺杂, 相互交错在一起。

事务管理器里面可以分为两个部分。

1. 日志与恢复, 所有的 SQL 在对磁盘进行操作的时候都会写一些日志, 比如物理日志, 逻辑日志, 并且会提供一定的恢复功能, 该部分负责事务的持久性。
2. 并发控制, 即怎么来控制对数据的事务, 并发控制包括锁和 MVCC。并发控制负责保证事务的原子性和孤立性。

在存储层存在 Buffer Pool (缓冲池), 所以从不会直接从底层的磁盘中将数据提取出来。很多时候是通过 Buffer Pool 查询 Catalog, 然后从 Catalog 中拿到所有的元数据信息, 之后再发请求给存储管理器, 由存储管理器最后发送到存储, 然后从存储中将数据拿取出来。

1.3 MiniOB 概述

MiniOB 背景

本节主要介绍 MiniOB，希望能通过 MiniOB 帮助大家从底层来深入了解数据库的实现原理。

首先介绍为什么制造 MiniOB。我们软件行业有一句话，软件行业有三驾马车，操作系统、中间件，还有数据库系统。可见，数据库系统是非常基础的一个组件。

数据库系统学习难度非常大，有工作经验的同学会经常使用和学习 MySQL 数据库，但依然不能精通数据库系统，如果要再实现一个数据库系统，那是更是难上加难。

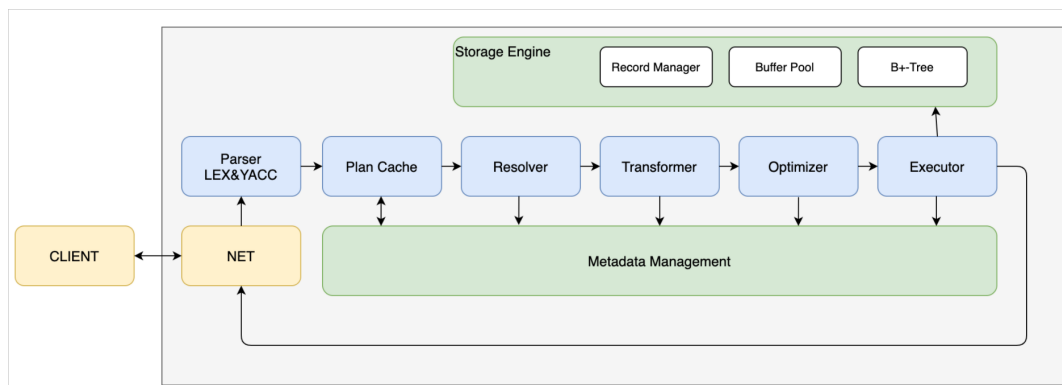
一方面是因为数据库系统涉及的知识面非常广。想要实现一个数据库系统，必须对操作系统、内存、IO 以及网络等知识都非常熟悉，甚至要达到精通的程度，才能构建一个可用性较高、性能较好的数据库系统。而现在数据库除了单机版的内容，又涉及到了很多分布式方向的知识，如多节点主备复制，多机房等，这些分布式容灾的相关知识，涉及知识面会更广。通过这些信息可知，数据库上手难度非常大，也很难入门去做一个数据库。

另一方面是因为国内数据库系统发展非常晚，国外数据库大概在六十年代就开始起步。国内却是近几年才开始高速发展，拥有完整自主产权的国产数据库更是屈指可数。其次国内培养数据库人才的高校也很少，只有部分高校有专门针对数据库实现方向的课程，而且一般在研究生阶段才会开设，本科阶段只是学习一些基本的原理和使用。近几年，国内的数据库厂商都在大力投入数据库领域，人才却非常匮乏。

基于上述背景，我们制造出了 MiniOB，希望通过 MiniOB 能够吸引更多的同学去从事数据库系统的研发，更希望通过 MiniOB 的系列课程，帮助有兴趣、有能力的同学学习到更多数据库的底层原理知识。

MiniOB 框架

MiniOB 是一个有基础功能的小型数据库，功能模块的设计比较简单，旨在为了帮助大家快速上手学习。另外它的起步时间比较晚，有些模块还不是很完善，欢迎有兴趣的同学一起参与共建。



如图所示左边是客户端，右边是服务端。客户端发起命令，通过网络通讯和服务端的网络模块去通讯收发命令。服务端的网络模块收到 SQL 请求后，交给词法/语法解析模块（Parser 模块），经过词法解析和语法解析模块将

SQL 请求字符串转换成带有语法结构信息的内存数据结构（语法树），再转发给下一个 Plan Cache，Plan Cache 目前不做特殊的处理，直接再转发给 Resolver 模块。

Resolver 模块会将判断模块解析出来的语法树，进一步细化后转换成真实的对象。比如查询一张表，会把表名转换成具体的表对象；如果是查询某个字段，会转换成对应的字段名；如果是 `select *`，会把`*`转换成对应的表的各个字段。除此之外还会做一些预检，比如查询一张不存在的表，就会提前返回错误。

经过 Resolver 阶段，会把处理的结果给到 Transformer 和下一个优化阶段。这两个阶段在部分数据库中，就是优化模块，如 MySQL。虽然图中是两个模块，但不一定按照 Transformer 和 Optimizer 的流程顺序来执行的，可能会多次的循环，在保证低成本下选择较优的查询方案。

Transformer 可以理解为根据一些规则去做转换，让后面的优化器能更好的优化。比如在查询表时，查询条件为 `where 1=1`，永远是真，那就会直接把这个条件删除。

经过这个规则转换后，交给优化模块。优化模块会根据一些条件找到更好的查询路径。比如查询数据，需要先判断直接使用索引查询，速度更快，还是通过全表遍历查询速度更快。因为有时部分因素或条件会导致索引查询更慢，所以优化模块会做一些判断，选择较好的查询计划，再转发给下一个执行模块。

执行模块（Executor）会按照查询计划去执行，访问索引、Buffer Pool、记录管理、以及底层的模块。然后把查询的结果返回给网络模块。网络模块再通过 Socket 返回给客户端。

以上是 MiniOB 简单的框架介绍。可以看出 MiniOB 结构清晰、简单很适合用来入门数据库。

MiniOB 比赛

2021 年，OceanBase 开源团队发起了首届 OceanBase 数据库大赛。初赛使用了 MiniOB 系统，赛题是联合华中科技大学的老师一起设计了一套从浅入深的题目。从简单到复杂，层层递进学习 MiniOB，边做边学习数据库理论知识，非常提升编程能力。我们也收到很多同学的反馈说代码能力、数据库知识都得到非常充分的提升。

建议同学们在做 MiniOB 题目的时候，最开始往深处做的时候，不要去一股脑的往上堆代码，要仔细的去思考，应该怎么做，及时去重构。在实际工作中也会经常遇到这样的问题，在原有的功能基础上，再加新功能时，可能之前设计的模式已经不适合新功能了，如果还一直用堆代码的方式，会导致代码越堆越乱。因此重构是非常常态的，也是个人编程实践能力的体现。

2022 年 OceanBase 数据库大赛初赛还会继续使用 MiniOB。我们的目标是没有变的，想要吸引更多的同学去上手学习数据库开发，帮助大家从 0 到 1 打造自己的数据库。基于上一届的经验，今年会提供更多的入门教程，帮助有编程基础能力的同学去更容易的上手这个数据库，同时也会提供更多的内容，包括 MiniOB 理论知识及 OceanBase 相关的内容。

不管是否参加比赛，只要能够坚持跟着课程学习，都会有不少的收获。2022 年大赛还会增加一些题目，提高 2021 年大赛进阶题目的难度，设计题目之间交叉，加大测试难度，尽可能覆盖更多的数据库理论知识。

1.4 MiniOB Gitee 使用说明

实战 MiniOB 编程需要在 Gitee 上创建自己的 private 仓库，在开发完成后，将代码提交到自己的仓库中，然后在训练营中进行测试。

MiniOB 仓库地址: <https://github.com/oceanbase/minioob>

训练营地址: <https://open.oceanbase.com/train>

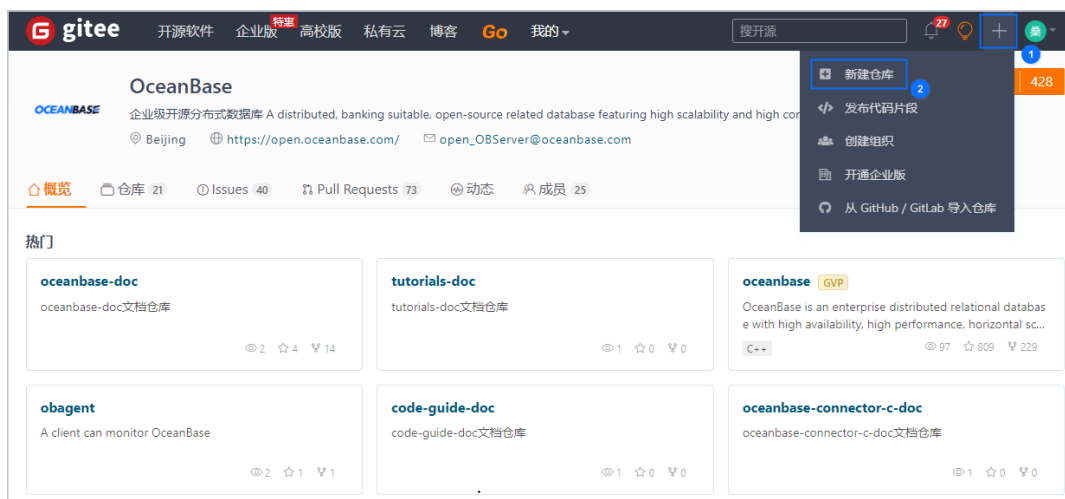
本文将以 Gitee 为例介绍如何在训练营中进行提测以及常用的 Git 操作命令。

Gitee 提测流程

前提条件: 已注册 Gitee 账号, Gitee 官网地址: <https://gitee.com>。

- 创建私有仓库

1. 登录 Gitee 平台, 选择 **新建仓库**。



2. 输入仓库信息, 单击 **创建**。设置为私有仓库后其他人无法查看到你的代码。



- 下载代码

```
# 将代码拉到本地
git clone https://github.com/oceanbase/miniob -b miniob_test
```

说明

若网络状态不好，也可以直接在 GitHub 上下载代码压缩包，下载时需要先选择 miniob_test 分支。

- 将 MiniOB 代码 push 到自己的仓库

```
# 进入到 miniob 目录，删除 .git 目录，清除已有的 git 信息
cd miniob
rm -rf .git

# 重新初始化 git 信息，并将代码提交到自己的仓库
git init
git add .
git commit -m 'init' # 提交所有代码到本地仓库

# 将代码推送到远程仓库
git remote add origin https://gitee.com/xxx/miniob.git # 注意替换命令中的 息为自己的库信息
git branch -M main
git push -u origin main
```

- 赋权官方测试账号

对于私有仓库，默认情况下其他人看不到，同样 OceanBase 测试后台也无法拉取到代码，这时想要提交测试，需要先给 OceanBase 的官方测试账号增加一个权限。

官方测试账号为：`oceanbase-ce-game-test`

首先在网页上打开自己的仓库，然后按照如下顺序操作即可。如果有疑问，也可以在 [OceanBase 社区论坛](#)或钉钉群（33254054）提问。

1. 选择 **管理 > 仓库成员管理 > 观察者**。



2. 选择 **直接添加**，搜索官方测试账号。

邀请用户

仓库成员配额说明
个人私有仓库最多支持 5 人协作 (如个人拥有多个私有仓库, 所有协作人数总计不得超过 5 人)

链接邀请 **直接添加** 通过仓库邀请成员

直接添加 Gitee 用户

权限
观察者

Gitee 用户
oceanbase-ce-game-test

添加

3. 添加完成后, 单击 **提交**。

添加成员

序号	成员	仓库角色权限	操作
1	oceanbase-ce-game-test (oceanbase-ce-game-test)	观察者	移除

取消 **提交**

日常 Git 开发命令

- 查看当前分支

```
git branch # 查看本地分支  
git branch -a # 查看所有分支, 包括远程分支
```

- 创建分支

```
git checkout -b 'your branch name'  
git branch -d 'your branch name' # 删除一个分支
```

- 切换分支

```
git checkout 'branch name'
```

- 提交代码

```
# 添加想要提交的文件或文件夹
git add 'the files or directories you want to commit'
# 这一步也可以用 git add . 添加当前目录

# 提交到本地仓库
# -m 中是提交代码的消息, 建议写有意义的信息, 方便后面查找
git commit -m 'commit message'
```

- 推送代码到远程仓库

```
git push
# 可以将多次提交, 一次性 push 到远程仓库
```

- 合并代码

```
# 假设当前处于分支 develop 下
git merge feature/update
# 会将 feature/update 分支的修改, merge 到 develop 分支
```

- 临时修改另一个分支的代码

```
# 有时候, 正在开发一个新功能时, 突然来了一个紧急 BUG, 这时候需要切换到另一个分支去开发
# 这时可以先把当前的代码提交上去, 然后切换分支。
# 或者也可以这样:
git stash # 将当前的修改保存起来

git checkout main # 切换到主分支, 或者修复 BUG 的分支

git checkout -b fix/xxx # 创建一个新分支, 用于修复问题

# 修改完成后, merge 到 main 分支
# 然后, 继续我们的功能开发

git checkout feature/update # 假设我们最开始就是在这个分支上
git stash pop

# stash 还有很多好玩的功能, 大家可以探索一下
```

1.5 MiniOB 开发调试环境搭建

本文将介绍两种方式的构建方法，可根据实际情况选择手动构建或直接使用 Dockerfile 构建。

手动打造开发环境

MiniOB 编译

前提条件：系统上已经安装了 Make 等编译工具。

MiniOB 需要使用：

- CMake：3.10 版本以上
- GCC/Clang：GCC 建议 8.3 以上，编译器需要支持 C++14 等新标准

安装 CMake

需要安装 3.10 或以上版本的 CMake，在 [CMake官网](#) 下载对应系统的 CMake。

```
wget https://github.com/Kitware/CMake/releases/download/v3.24.0/cmake-3.24.0-linux-x86_64.sh
bash cmake-3.24.0-linux-x86_64.sh
```

如果是 MAC 系统，执行以下命令安装。

```
brew install cmake
```

安装 GCC

如果是个人学习使用，Clang 通常没有问题，如果想参加比赛或使用 OceanBase 官网训练营，建议安装 GCC。因为 Clang 某些情况下的表现会与 GCC 不一致。

另外，如果 GCC 的版本比较旧，需要安装较新版本的 GCC。有些比较旧的操作系统，如 CentOS 7，自带的编译器版本是 4.8.5，对 C++14 等新标准，支持不友好，建议升级 GCC。

在 Linux 系统上，通常使用 `yum install gcc gcc-g++` 就可以安装 GCC，但是有些时候安装的编译器版本比较旧，需要手动安装。

- 如何查看 GCC 版本

```
gcc --version
```

- 下载 GCC 源码

可以在 [GCC 官网](#) 浏览，找到镜像入口，选择速度比较快的镜像，如 [Russia, Novosibirsk GCC 镜像链接](#)。

建议选择 8.3 或以上版本。MiniOB 是为了学习使用，可以选择较新版本的 GCC，不要求稳定。

- 编译 GCC 源码

注意

编译高版本的 GCC 时，需要本地也有一个稍高一点版本的 GCC。比如编译 GCC 11，本地的 GCC 需要能够支持 `stdc++11` 才能编译，可以先编译 GCC 5.4，然后再编译高版本。

```
# 解压
tar xzvf gcc-11.3.0.tar.gz
cd gcc-11.3.0

# 下载依赖
./contrib/download_prerequisites

# 配置。可以通过 prefix 参数设置编译完成的 GCC 的安装目录，如果不指定，会安装在 /usr/local 下
# 可以配置为当前用户的某个目录
./configure --prefix=/your/new/gcc/path --enable-threads=posix --disable-checking \
    --enable-long-long --with-system-zlib --enable-languages=c,c++

# 开始编译
make -j

# 安装
# 编译产物会安装到 configure --prefix 指定的目录中，或系统默认目录下
make install

# 修改环境变量
# 可以将下面的配置写到 .bashrc 或 .bash_profile 中，这样每次登录都会自动生效
export PATH=/your/new/gcc/path/bin:$PATH
export LD_LIBRARY_PATH=/your/new/gcc/path/lib64:$LD_LIBRARY_PATH
export CC=/your/new/gcc/path/bin/gcc
export CXX=/your/new/gcc/path/bin/g++

# NOTE: 上面的环境变量 CC 和 CXX 是告诉 CMake 能够找到我们的编译器。CMake 优先查找的
#       编译器是 CC 而不是 GCC，而一般系统中会默认带 CC，所以使用环境变量告诉 CMake。
#       也可以使用 CMake 参数
#       -DCMAKE_C_COMPILER=/your/new/gcc/path/bin/gcc -DCMAKE_CXX_COMPILER=/your/new/gcc/path/b
in/g++`
#       来指定编译器
```

如果 `./contrib/download_prerequisites` 下载时特别慢或下载失败，可以手动从官网上下载依赖，解压相应的包到 GCC 的目录下。

```
ftp://ftp.gnu.org/gnu/gmp
https://mpfr.loria.fr/mpfr-current/#download
https://www.multiprecision.org/mpc/download.html
```

构建 Libevent

```
git submodule add https://github.com/libevent/libevent deps/libevent
cd deps
cd libevent
git checkout release-2.1.12-stable
mkdir build
cd build
cmake .. -DEVENT__DISABLE_OPENSSL=ON
make
sudo make install
```

构建 Googletest

```
git submodule add https://github.com/google/googletest deps/googletest
cd deps
cd googletest
mkdir build
cd build
cmake ..
make
sudo make install
```

构建 JsonCpp

```
git submodule add https://github.com/open-source-parsers/jsoncpp.git deps/jsoncpp
cd deps
cd jsoncpp
mkdir build
cd build
cmake -DJSONCPP_WITH_TESTS=OFF -DJSONCPP_WITH_POST_BUILD_UNITTEST=OFF ..
make
sudo make install
```

构建 MiniOB

```
cd `project home`
mkdir build
cd build

# 建议开启DEBUG模式编译, 更方便调试
cmake .. -DDEBUG=ON
make
```

MiniOB 调试

调试 C/C++ 程序, 常用的有两种方式, 一是通过打印日志进行调试, 二是借助 GDB 调试器进行调试。通过调试程序, 可以熟悉程序执行的每一步细节, 不仅可以定位问题, 也可以用来熟悉代码, 增加编程能力。

MiniOB 的关键数据结构

```
parse_def.h:
    struct Selects; // 查询相关
    struct CreateTable; // 建表相关
    struct DropTable; // 删表相关
    enum SqlCommandFlag; // sql 语句对应的 command 枚举
    union Queries; // 各类 dml 和 ddl 操作的联合
table.h
    class Table;
db.h
    class Db;
```

MiniOB 的关键接口

```
RC parse(const char *st, Query *sqln); // sql parse 入口
ExecuteStage::handle_request
ExecuteStage::do_select
DefaultStorageStage::handle_event
DefaultHandler::create_index
DefaultHandler::insert_record
DefaultHandler::delete_record
DefaultHandler::update_record
Db::create_table
Db::find_table
Table::create
Table::scan_record
Table::insert_record
Table::update_record
Table::delete_record
Table::scan_record
Table::create_index
```

打印日志调试

MiniOB 提供的日志接口

```
deps/common/log/log.h:
#define LOG_PANIC(fmt, ...)
#define LOG_ERROR(fmt, ...)
#define LOG_WARN(fmt, ...)
#define LOG_INFO(fmt, ...)
#define LOG_DEBUG(fmt, ...)
#define LOG_TRACE(fmt, ...)
```

日志相关配置项 observer.ini

```
LOG_FILE_NAME = observer.log
# LOG_LEVEL_PANIC = 0,
# LOG_LEVEL_ERR = 1,
```

```
# LOG_LEVEL_WARN = 2,
# LOG_LEVEL_INFO = 3,
# LOG_LEVEL_DEBUG = 4,
# LOG_LEVEL_TRACE = 5,
# LOG_LEVEL_LAST
LOG_FILE_LEVEL=5
LOG_CONSOLE_LEVEL=1
```

GDB 调试

1. Attach 进程

```
[admin@localhost run]$ gdb -p `pidof observer`
```

```
GNU gdb (GDB) Red Hat Enterprise Linux 8.2-15.el8 Copyright (C) 2018 Free Software Foundation, Inc.
```

```
(gdb)
```

2. 设置断点

```
(gdb) break do_select
Breakpoint 1 at 0x44b636: file /home/admin/source/stunning-engine/src/observer/sql/executor/execute_stage.cpp, line 526.
```

```
(gdb) info b
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x000000000044b636	in ExecuteStage::do_select(char const*, Query*, SessionEvent*) at /home/admin/source/stunning-engine/src/observer/sql/executor/execute_stage.cpp:526

```
(gdb) break Table::scan_record
```

```
Breakpoint 2 at 0x50b82b: Table::scan_record. (2 locations)
```

```
(gdb) inf b
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x000000000044b636	in ExecuteStage::do_select(char const*, Query*, SessionEvent*) at /home/admin/source/stunning-engine/src/observer/sql/executor/execute_stage.cpp:526
2	breakpoint	keep	y	<MULTIPLE>	
2.1			y	0x000000000050b82b	in Table::scan_record(Trx*, ConditionFilter*, int, void*, void (*)(char const*, void*)) at /home/admin/source/stunning-engine/src/observer/storage/common/table.cpp:421
2.2			y	0x000000000050ba00	in Table::scan_record(Trx*, ConditionFilter*, int, void*, RC (*)(Record*, void*)) at /home/admin/source/stunning-engine/src/observer/storage/common/table.cpp:426

```
(gdb)
```

3. 继续执行

```
(gdb) c
Continuing.
```

4. 触发断点

执行: `miniob > select * from t1;`

```
[Switching to Thread 0x7f51345f9700 (LWP 54706)]

Thread 8 "observer" hit Breakpoint 1, ExecuteStage::do_select (this=0x611000000540,
    db=0x60400000005e0 "sys", sql=0x6200000023080, session_event=0x6080000003d20)
    at /home/admin/source/stunning-engine/src/observer/sql/executor/execute_stage.cpp:526
526      RC rc = RC::SUCCESS;
(gdb)
```

5. 单步调试 (跳过函数调用)

```
575      std::vector<TupleSet> tuple_sets;
(gdb) next
576      for (SelectExeNode *&node: select_nodes) {
(gdb) n
577          TupleSet tuple_set;
(gdb)
578          rc = node->execute(tuple_set);
(gdb)
```

6. 单步调试 (进入函数调用的内部)

```
(gdb) s
SelectExeNode::execute (this=0x607000002ce80, tuple_set=...)
    at /home/admin/source/stunning-engine/src/observer/sql/executor/execution_node.cpp:43
43      CompositeConditionFilter condition_filter;
(gdb)
```

7. 打印变量

```
(gdb) p tuple_set
$3 = (TupleSet &) @0x7f51345f1760: {tuples_ = std::vector of length 0, capacity 0, schema_ = {
    fields_ = std::vector of length 0, capacity 0}}
(gdb)
```

8. watch 变量

```
(gdb) n
443      RC rc = RC::SUCCESS;
(gdb) n
444      RecordFileScanner scanner;
(gdb) n
445      rc = scanner.open_scan(*data_buffer_pool_, file_id_, filter);
(gdb) watch -l rc
Hardware watchpoint 3: -location rc
```

```
(gdb) c
Continuing.

Thread 8 "observer" hit Hardware watchpoint 3: -location rc

Old value = SUCCESS
New value = RECORD_EOF
0x000000000050c2de in Table::scan_record (this=0x60f000007840, trx=0x606000009920,
    filter=0x7f51345f12a0, limit=2147483647, context=0x7f51345f11c0,
    record_reader=0x50b74a <scan_record_reader_adapter(Record*, void*)>)
    at /home/admin/source/stunning-engine/src/observer/storage/common/table.cpp:454
454     for ( ; RC::SUCCESS == rc && record_count < limit; rc = scanner.get_next_record(&recor
d)) {
(gdb)
```

9. 结束函数调用

```
(gdb) finish
Run till exit from #0  0x000000000050c2de in Table::scan_record (this=0x60f000007840,
    trx=0x606000009920, filter=0x7f51345f12a0, limit=2147483647, context=0x7f51345f11c0,
    record_reader=0x50b74a <scan_record_reader_adapter(Record*, void*)>)
    at /home/admin/source/stunning-engine/src/observer/storage/common/table.cpp:454
```

10. 结束调试

```
(gdb) quit
A debugging session is active.

    Inferior 1 [process 54699] will be detached.

Quit anyway? (y or n) y
Detaching from program: /home/admin/local/bin/observer, process 54699
[Inferior 1 (process 54699) detached]
```

使用 Dockerfile 构建

Docker 环境说明

MiniOB 镜像文件基于 CentOS:7 制作。

镜像包含:

- jsoncpp
- google test
- libevent
- flex

- bison(3.7)
- gcc/g++ (version=11)
- MiniOB 源码(/root/source/miniob)

在 Docker 中 `/root/source/miniob` 目录下载了 GitHub 的源码, 可以根据个人需要, 下载自己仓库的源代码, 也可以使用 `git pull` 拉取最新代码。 `/root/source/miniob/build.sh` 提供了一个编译脚本, 以 DEBUG 模式编译 MiniOB。

操作步骤

首先要确保本地已经安装了 Docker。

- 使用 Dockerfile 构建

Dockerfile: <https://github.com/oceanbase/miniob/blob/main/Dockerfile>

```
# build
docker build -t miniob .
docker run -d --name='miniob' miniob:latest

# 进入容器, 开发调试
docker exec -it miniob bash
```

- 使用 docker hub 镜像运行

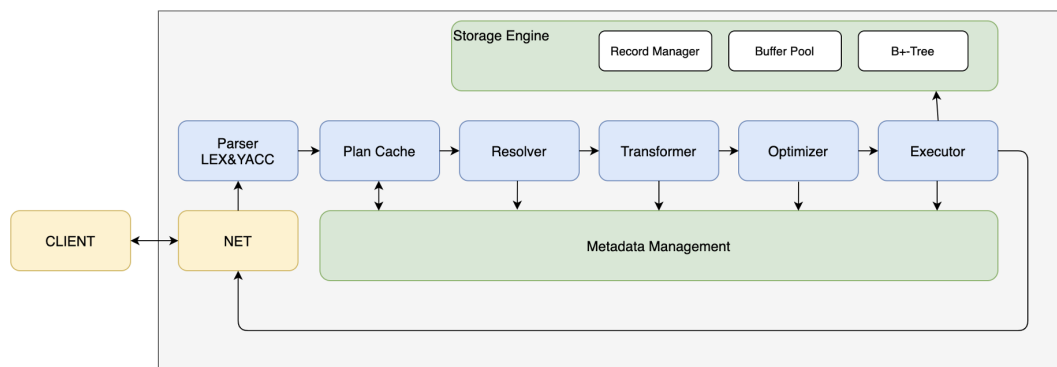
```
docker run oceanbase/miniob
```

1.6 MiniOB 线程模型

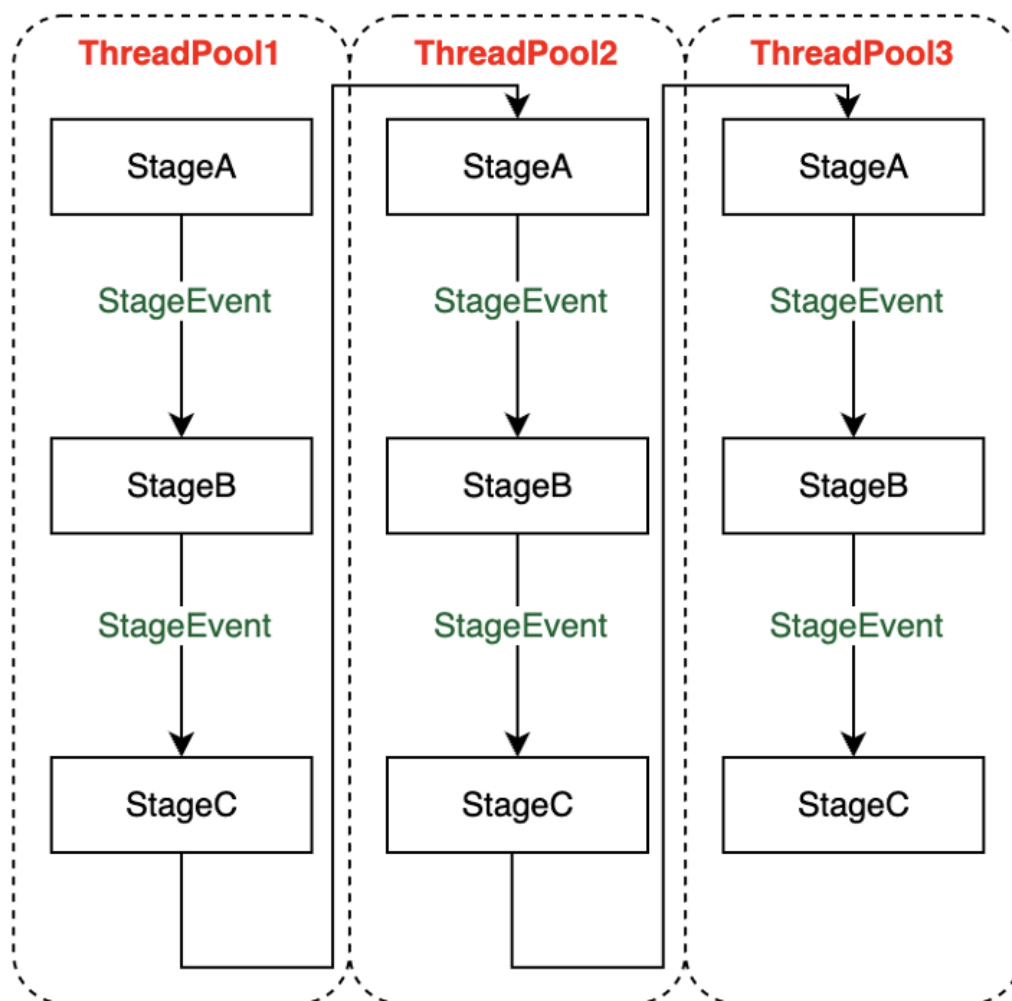
本节主要介绍 MiniOB 的底层数据结构线程池，线程池是做并发处理时非常常用的手段。希望通过本节内容的学习，大家在做 MiniOB 训练需要阅读代码时，不用花费特别多的精力去挖掘相关技术内容。

SEDA 框架

首先回顾一下 MiniOB 框架，MiniOB 有很多处理阶段，如词法解析、语法解析、Resolver、优化器等。MiniOB 使用了 SEDA 的框架，像 Resolver、优化器等在 SEDA 中都是一个个的 Stage，各个 Stage 之间通过事件来传递数据。MiniOB 的多线程能力是基于 SEDA 来做的，SEDA 本身就是支持线程池的。

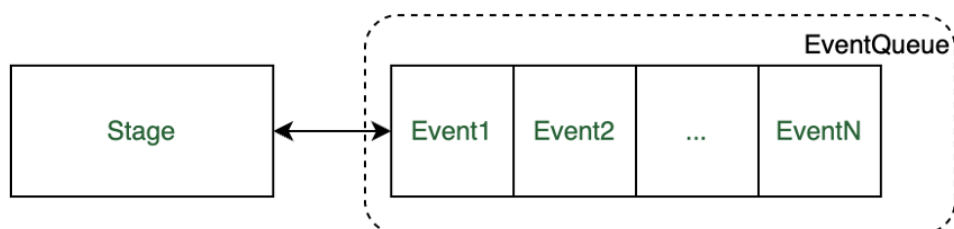


下图所示是一个简单的 SEDA 框架图，能够很好的帮助理解 MiniOB 中的每个 Stage 和 event，以及线程池之间的关系。线程池可以有多个，每个线程池可以包含多个 Stage，一个 Stage 会绑定到某一个线程池里面。每个线程池可以设置自己的线程的个数、事件队列的大小，线程池中的多个 Stage 之间是通过事件通讯的。虽然示例图中写的都是 StageEvent，但也可以是不同的名字。另外，不同的线程池之间的 Stage 也可以通过事件通讯，并且每一个线程池里面的 Stage 可以是不同的。



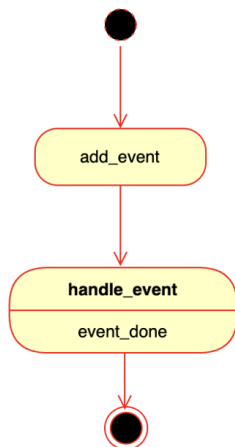
SEDA 框架有一个好处是可以将 Stage 进行分类，比如把处理 SQL 的分作一类，把 IO 处理的分作为一类，并把不同的分类分配到不同的线程池。这样在处理任务时不容易受到其他干扰，并且也能够根据真实的业务场景去调整每个线程池的大小。

接下来看一个简单的 Stage 模型。一个 Stage 对应一个类型的事件，事件是放在线程池事件队列中的。线程池事件指当事件队列有事件后，就将其交给对应的 Stage 来处理。



事件的生命周期

创建 event 的时候，它就会被加到一个线程事件队列里面（参考 `net/server.cpp` 中 `session_stage->add_event(sev);`）。添加到事件队列里面之后，就会由 Stage 调用 `handle_event` 去处理（参考 `session/session_stage.cpp` 中 `SessionStage::handle_event`）。处理完成之后，event 就结束了。

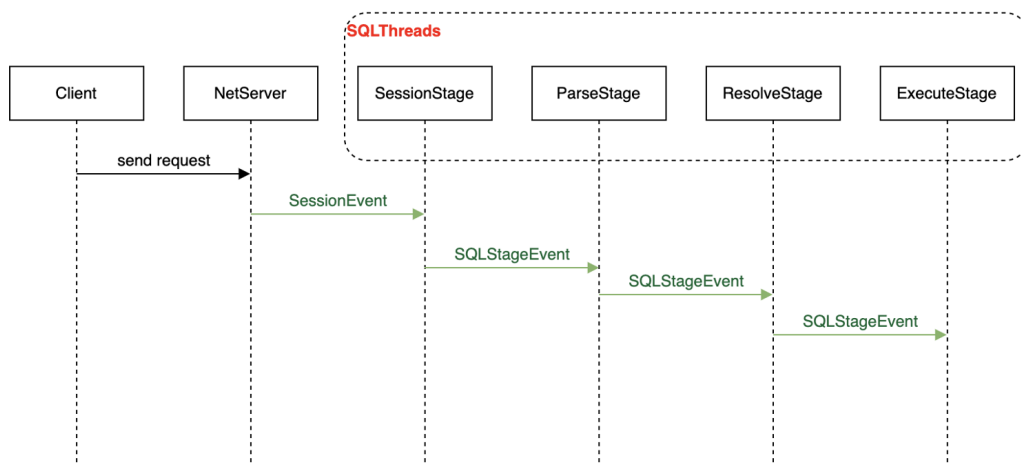


事件在处理的时候，还可以增加一些回调函数（比如 `session/session_stage.cpp` 中的 `sev->push_callback(cb);`），后续将事件交给其它的 Stage 做处理，处理完成后就会调用回调函数。也可以在回调函数中做一些任务处理，在代码里可以看到很多，比如 `done_immediate` 就是立即处理完成（比如 `parser/parse_stage.cpp` 中的 `event->done_immediate()`）。

MiniOB 事件执行过程

客户端通过网络发送一个请求到服务端，服务端在接收到消息后会创建一个消息包，并创建对应的 `SessionEvent`。然后调用 `add_event` 把 `SessionEvent` 添加到 `SessionStage` 里（参考 `net/server.cpp` 中 `session_stage->add_event(sev);`），再放到对应的事件队列中。`SessionStage` 以及后续处理事件的几个 Stage，都在 `SQLThreads` 线程池中。

`SessionStage` 处理 `SessionEvent` 后，会创建一个 `SQLStageEvent` 传递给后面的 Stage。后续的 Stage 都是处理这个 `SQLStageEvent` 的事件。如下图所示，事件经过一个流转，处理完成后返回结果给客户端。



MiniOB 代码解读

接下来介绍一下 MiniOB 代码中的 SEDA 框架、线程池以及各个 Stage。如下所示，在配置文件 `etc/observer.ini` 中可以很清晰的看到 MiniOB 有哪些线程池（SQL、IO 和 Default）以及 Stage。每个 Stage 可以配置自己位于哪个线程池、关联的下一个 Stage 以及所属线程池，如果不配置，则使用默认的线程池。

```
[SQLThreads]
# 线程的个数, 0 就会用 CPU 的个数
count=3

[IOThreads]
count=3

[DefaultThreads]
# 如果 Stage 没有配置线程池, 就会用这个默认的线程池
count=3

[SessionStage]
ThreadId=SQLThreads
NextStages=PlanCacheStage

[PlanCacheStage]
ThreadId=SQLThreads
#NextStages=OptimizeStage
NextStages=ParseStage

[ParseStage]
ThreadId=SQLThreads
NextStages=ResolveStage

[ResolveStage]
ThreadId=SQLThreads
NextStages=QueryCacheStage

[QueryCacheStage]
ThreadId=SQLThreads
NextStages=OptimizeStage

[OptimizeStage]
ThreadId=SQLThreads
NextStages=ExecuteStage

[ExecuteStage]
ThreadId=SQLThreads
NextStages=DefaultStorageStage,MemStorageStage

[DefaultStorageStage]
ThreadId=IOThreads
BaseDir=./miniob
SystemDb=sys

[MemStorageStage]
ThreadId=IOThreads

[MetricsStage]
```

```
NextStages=TimerStage
```

初始化阶段时先读取配置文件 `etc/observer.ini`，然后创建各个 Stage。`prepare_init_seda` 函数中创建了很多静态的 StageFactory。这里根据名字可以关联到配置文件中的相关配置，并且提供了一个创建 Stage 对象的函数，这些 Stage 会有一些共同的接口。

SEDA 框架本身的初始化函数是 `seda_config.cpp` 中的 `SedaConfig::init` 函数。这里会创建默认线程池和一些 Stage，比如 SEDA 框架自带的 Metric Stage。接着把各个线程池、各个 Stage 做关联和启动，参考 `SedaConfig::init_stages@seda_config.cpp`，然后做 Stage 对象的实例化。最后按照配置文件，通过刚才创建的静态 StageFactory 对象注册的一些 Stage 对象，执行初始化、启动。

```
// MiniOB 的初始化函数，在 init.cpp 中
int init(ProcessParam *process_param)
{
    ...
    // seda is used for backend async event handler
    // the latency of seda is slow, it isn't used for critical latency
    // environment.
    prepare_init_seda();
    rc = init_seda(process_param);
    if (rc) {
        LOG_ERROR("Failed to init seda configuration!");
        return rc;
    }
    ....
}

// MiniOB 初始化钱，执行的 prepare 动作，在 init.cpp 文件中
int prepare_init_seda()
{
    static StageFactory session_stage_factory("SessionStage", &SessionStage::make_stage);
    static StageFactory resolve_stage_factory("ResolveStage", &ResolveStage::make_stage);
    ....
    return 0;
}
```

接下来以 SessionStage 为例介绍一下 Stage。

- `make_stage` 就是创建一个 Stage 对象，参考 `SessionStage::make_stage@session_stage.cpp`，这里就对应了 StageFactory 的创建对应的接口。
- `set_properties` 接口可以从配置文件中读取一些配置数据，参考 `SessionStage::set_properties@session_stage.cpp`，不过 SessionStage 没有任何配置。
- `SessionStage::initialize` 是 Stage 的初始化接口，这里可以做一些初始化动作。
- `SessionStage::handle_event` 是 Stage 处理事件的关键接口，当有对应事件到达时，SEDA 就会调用此接口。

对事件回调函数的使用可以参考 `SessionStage::handle_request`，在调用下一个 Stage 之前，先创建一个 callback，然后 push_back 到 event 回调队列中，代码参考：

```
CompletionCallback *cb = new (std::nothrow) CompletionCallback(this, nullptr);
if (cb == nullptr) {
    sev->done_immediate();
    return;
}

sev->push_callback(cb);
```

SEDA 框架是一个非常优秀的线程池事件处理框架，它的扩展性、灵活度都非常高。感兴趣的同学可以去 `deps/common/seda` 目录下查看源码。

1.7 课后实践

MiniOB 题目：在 <https://github.com/oceanbase/miniob> 提交一个PR

选择适合自己的 Issue，MiniOB Issue 较少，建议从优化 MiniOB 入手。

推荐以下方式：

- 为 MiniOB 增加注释，模块注释为主，不限中英文；
- 优化某个模块的设计。

第 2 章：数据库的存储结构

本章主要介绍数据库存储结构基础及 MiniOB 存储实现原理。

本章目录

2.1 存储器的层次结构

2.2 磁盘存储器

2.3 块与记录组织

2.4 变长数据和记录

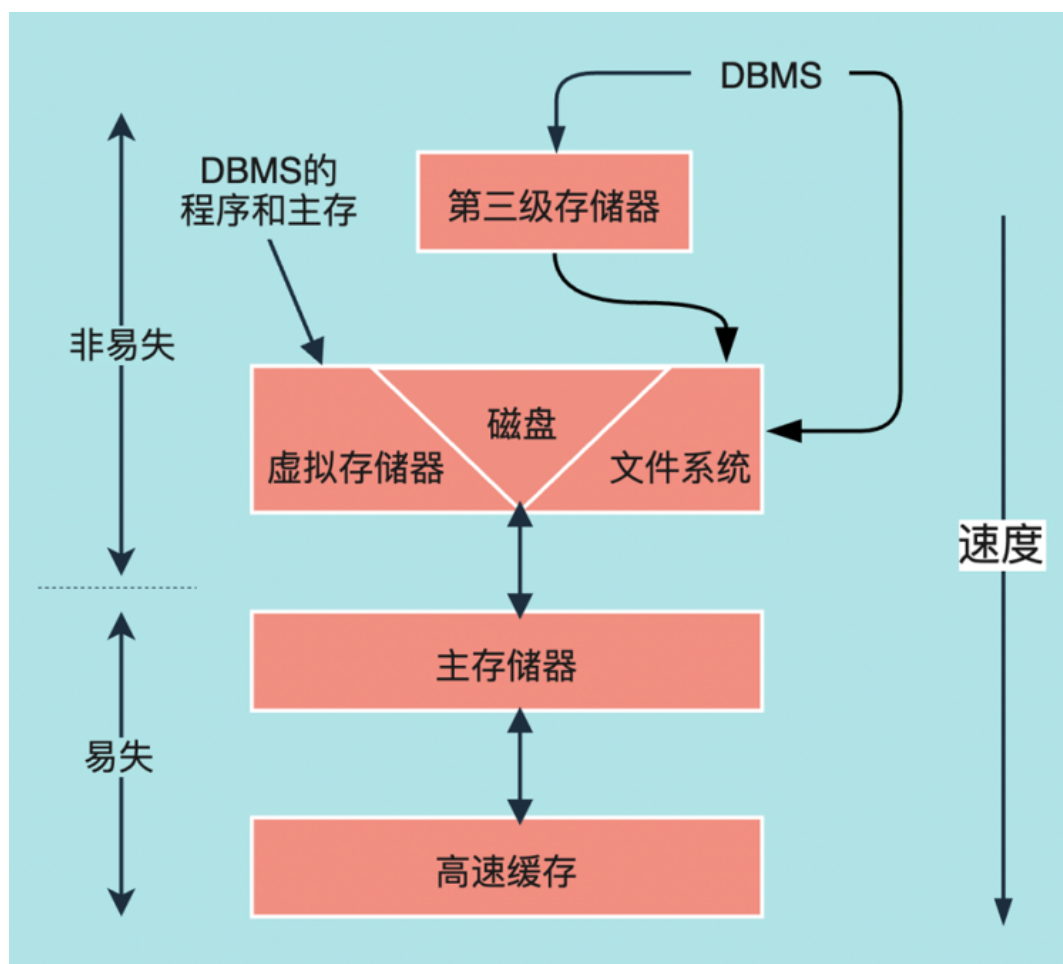
2.5 记录的修改

2.6 MiniOB 存储实现原理

2.7 课后实践

2.1 存储器的层次结构

存储器层次



如上图所示是一个比较经典的存储层次的组织图，该图是按照存储的容量或速度的大小进行规划。图的最下面是高速缓存，高速缓存大家应该都不陌生，例如我们在买电脑时，经常听到的 CPU 多级缓存技术，指的就是高速缓存，高速缓存的特点是速度快，可以达到 ns 级别，但容量小，一般只有几 MB 的大小。

高速缓存往上是主存储器，一般称为主存或内存，大家可能都碰到过 Linux 系统的 OOM 问题，其实就是内存不足所引发的一个异常。普通家用电脑的内存一般是 4G 或 8G，生产环境的机器一般能达到几百 G 的内存大小。

高速缓存和主存有个共同的特点——易失性，简单来说就是掉电会丢失数据，掉电不丢失的内存有 ROM、EEPROM、闪存等，相比来说，RAM 的速度更快。

再往上，这一层也叫辅助存储层，主要是指磁盘，虚拟存储器和文件系统都是建立在磁盘之上的组织结构。

再接着往上是第三级存储器，例如 DVD 光盘，磁带机等，其目的都是为了扩展整体的存储空间，并在不同的场景下使用。

存储器特点

这里介绍几种主要存储方式的一些特点，例如高速缓存的容量小，但速度快；主存的访问性能和高速缓存相比虽然差点，但是容量较高，能达到 G 级别；磁盘的存储虽然很大，但是访问磁盘的 IO 耗时严重，一般和主存的访问速度差几个数量级。目前 SSD 磁盘的性能相比于传统的机械盘要快的多，有兴趣可以去了解 SSD 和机械盘的差异。

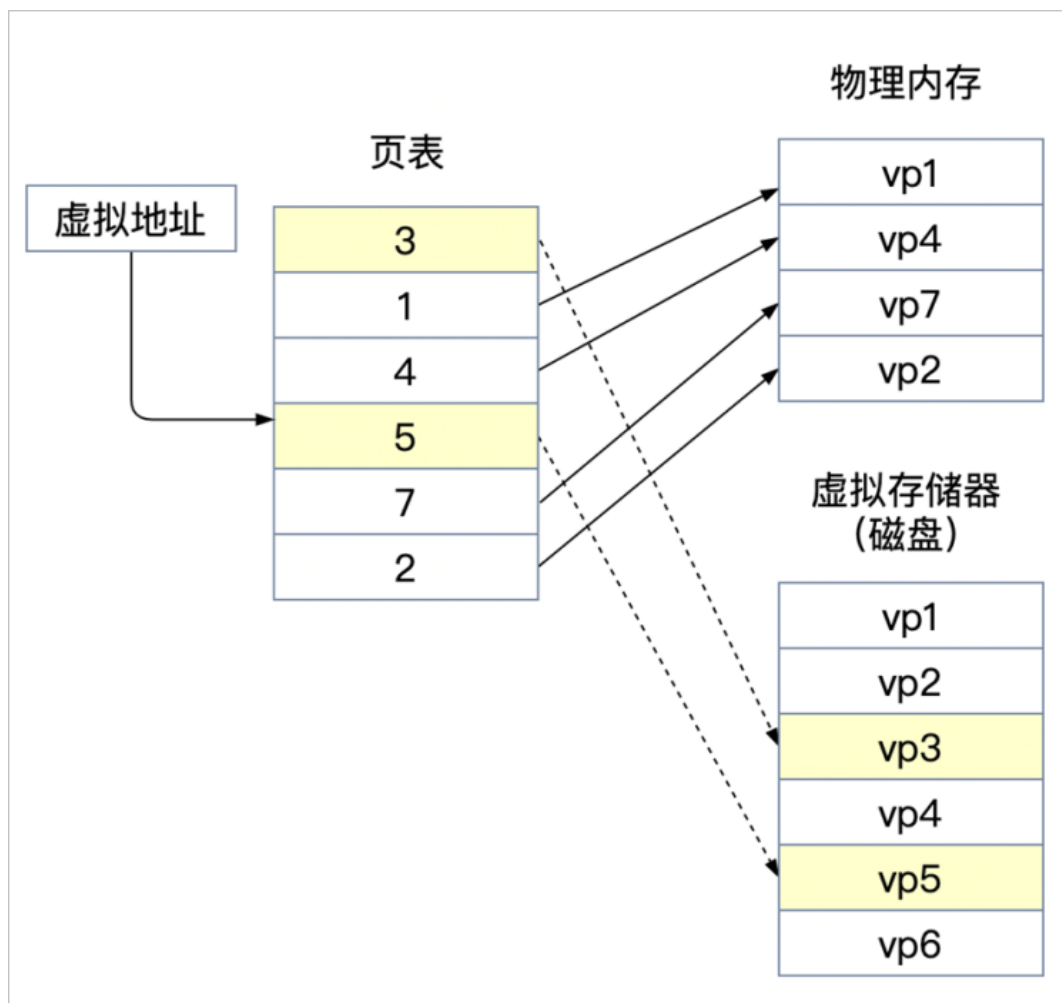
存储方式的具体差异请参考下表：

存储器	容量	特点
高速缓存存储器	1-6 MB	- 高速缓存和处理器间速度为 < 10 纳秒 - 高速缓存和主存储器间速度为 10-100 纳秒
主存储器（内存）	10GB 以上	随机访问，访问时间 10-100 纳秒
辅助存储器（磁盘/固态硬盘）	1TB-10TB	数据传输：一次 < 10ms
第三级存储（磁带/DVD 光盘）	PT 级别	成本低、速度慢（s/min）

虚拟存储器

介绍虚拟存储器之前，我们先简单介绍一下虚拟地址空间，可能大家对于进程空间、虚拟地址空间、物理地址空间、地址映射等概念都比较熟悉。进程在启动时，操作系统会为进程分配虚拟地址空间，例如 32 位系统的虚拟地址空间最大为 4G，假设进程不止一个，如 2 个进程，那操作系统不可能为每个进程都分配 4G 实际的物理内存空间，毕竟内存总大小就 4G。

这种情况下，操作系统使用了虚拟地址空间来解决内存分配问题，并通过虚拟地址空间和物理地址建立映射关系来操作实际的物理内存。虚拟地址和物理地址之间是通过 MMU 的结构来管理复杂的映射关系。通俗来说，两个进程通过 MMU 按需共享一个物理内存，MMU 的功能其实是把 CPU 发出的虚拟地址转换为物理地址。

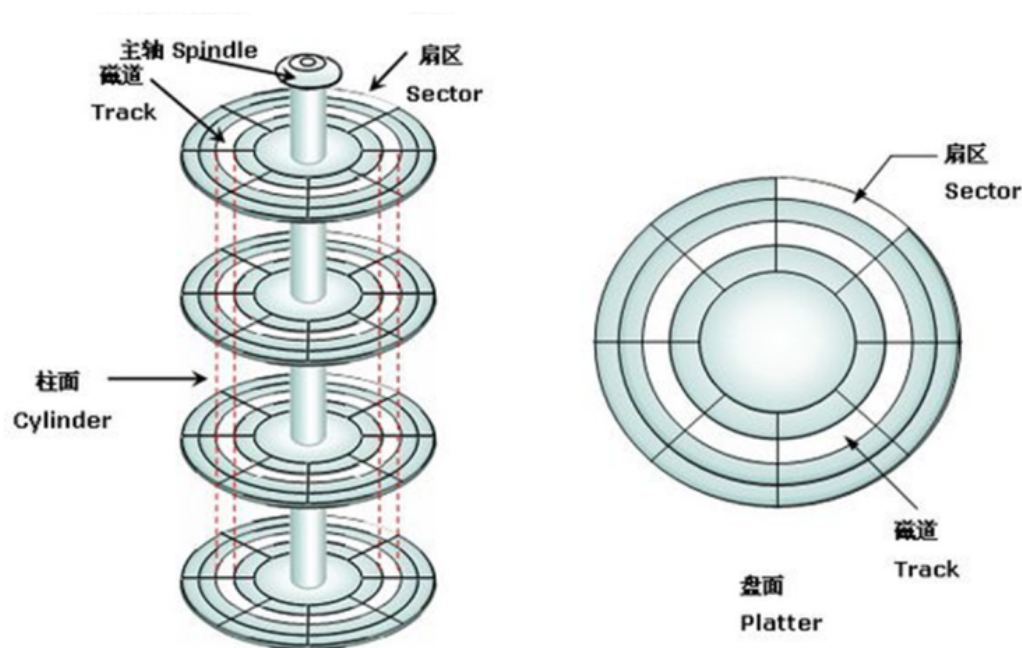


如上图所示，假设 CPU 发起对虚拟地址的访问，页表中所有需要访问的页号都能从物理内存中找到自己的映射，这是个比较理想的过程。但现实中，可能会遇到缺页错误，例如图中标黄的 page3 和 page5 就没有缓存到物理内存中，这时操作系统会从磁盘缓存一块空间到物理内存，来填补缺页的错误，维护映射关系。

因此这种使用虚拟地址来分配和管理物理内存，并且通过交换内存和磁盘的数据，来实现空间扩容的方式，叫做虚拟存储器。可以把虚拟存储器的实现理解成一种解决问题的思路，或一种按需分配的办法，例如数据库中的各种缓存技术，就是最大化的结合内存和磁盘能力的体现。

2.2 磁盘存储器

磁盘基础概念



上图描述了磁盘的整体结构。磁盘由多个盘片组成，其中一个盘片有正反两个盘面，每个盘面上有一个磁头，盘片的每个盘面被划分成多个同心圆，也就是磁道，每个磁道又被划分成若干弧段，每段称为一个扇区。柱面是指不同盘面上相同磁道编号的组合，簇是指物理相邻的若干个扇区。

磁盘空间计算

磁盘空间主要通过以下 4 个属性计算得出：

- 盘面数量
- 磁道数量
- 扇区数量
- 扇区大小

例如，某磁盘具有以下属性：

- 8 个盘片，16 个盘面
- 每个盘面有 $2 \times 16 = 65536$ 个磁道
- 每个磁道 256 个扇区

- 每个扇区 4096 个字节

那么这块磁盘总空间为:

$$16 \times 65536 \times 256 \times 4096 \text{Bytes} = 1\text{TB}$$

磁盘存取性能

日常使用磁盘中，特别是在读写磁盘操作过程中，偶尔会听到磁盘里面会发出一些噪音，这是磁盘的磁头在不停寻址产生的高速旋转。

磁盘的性能主要受以下因素的影响：

- 寻道时延

磁臂移动到指定磁道所需要的时间

- 旋转时延

把扇区移动到磁头下面的时间

- 传输时延

从磁盘读出或将数据写入磁盘的时间

所以总时延 = 寻道时间 + 旋转延迟 + 传输时间。

磁盘性能优化

虽然磁盘的性能由自身决定，但可以通过一些外部的方法对磁盘的使用性能进行优化，我们知道磁盘在使用上主要的性能问题是磁盘访问 IO，因此，解决思路就是在相同访问数据量的情况下，降低磁盘 IO 的开销。

- 可以将磁盘的数据缓存到主存中，通过操作主存中的数据，定期批量的刷新到磁盘保存，了解数据库系统的同学应该知道，这种方式跟数据库的 checkpoint 机制很相似。
- 通过一些算法优化磁盘的访问，例如使用多盘来让更多的磁头同时参与访问磁盘块，或者将需要访问的块放到同一个柱面来降低寻道和旋转的时间等。

总而言之，所有方法的原则就是降低磁盘 IO 或者提高磁盘数据的访问。

磁盘故障

磁盘的故障一般分为以下几个方面：

- 间断性故障

反复读取间断性成功

- 介质损坏

存储数据的扇区中一个或多个二进制位永久损坏，这种故障无法通过重复读取来解决

- 写故障
不能写入扇区，也不能检索写过的扇区
- 磁盘崩溃
整个磁盘不能读写

对于上述磁盘读写错误的故障，实际中也是有一些检测机制来发现故障问题。

- 校验和技术（checksum），原理是在数据的最后加上根据数值内容进行的某种数学的叠加计算。
这里介绍一种简单的数字校验和技术：奇偶校验和。

01101000

11101110

011010001

111011100

奇偶位为1

奇偶位为0

两个数位发生变化

01101000

01111010

011110101

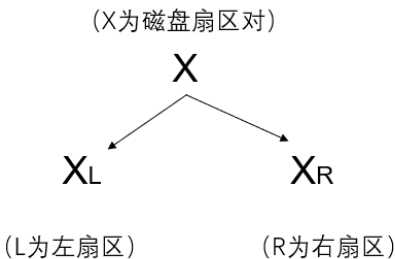
奇偶位为1

如上图所示，假设有二进制内容 01101000，使用 1 位的奇偶位，如果二进制内容中有奇数个 1，那么认为数据具有奇数奇偶性，奇偶位为 1，也就是 011010001；如果数值中有偶数个 1，那么认为数据具有偶数奇偶性，奇偶位为 0，也就是 111011100。

那磁盘在读取数据时，因为已知二进制内容奇偶性，假设二进制内容 01101000 发生了变化，那么通过计算奇偶位后，发现与原来的奇偶位不一致，就可以认为磁盘发生了读故障。

当然，一个奇偶位只有 50% 的正确率，比如 01101000 发生变化的位数有 2 位，变成了 01111010，计算后的奇偶位还是 1，整体的奇偶性并没有变化，但实际上二进制内容已经发生变换，因此故障也就无法检测，这种场景可以通过增加奇偶位的数量来解决。例如增加了 8 位的奇偶位，则检测出错的概率是 $1/2^8$ ，以此类推。

- 稳定存储，可以简单理解为扇区数据的冗余，例如下图中的 X 扇区，使用两个扇区来冗余避错。这跟数据库中多副本的原理是一样的。



2.3 块与记录组织

概念介绍

- 记录

按照若干个字段元素组成的数据集合。

- 数据块

由多个数据记录元素组成的一个数据集合，在物理上对应的是一块连续的存储空间。

总结而言，块中有若干行记录，每行记录中有若干个字段。

比如如下所示的 SQL 建表语句，按照 name, salary, address, gender, birthdate 的字段建立一个关系表。如果使用行存储的数据库，那么记录就是指数据库中的一行内容。

```
CREATE TABLE MovieStar(  
    name CHAR(30) PRIMARY KEY,  
    salary INTEGER,  
    address VARCHAR(255),  
    gender CHAR(1),  
    birthdate DATE  
);
```

记录的类型

- 定长类型

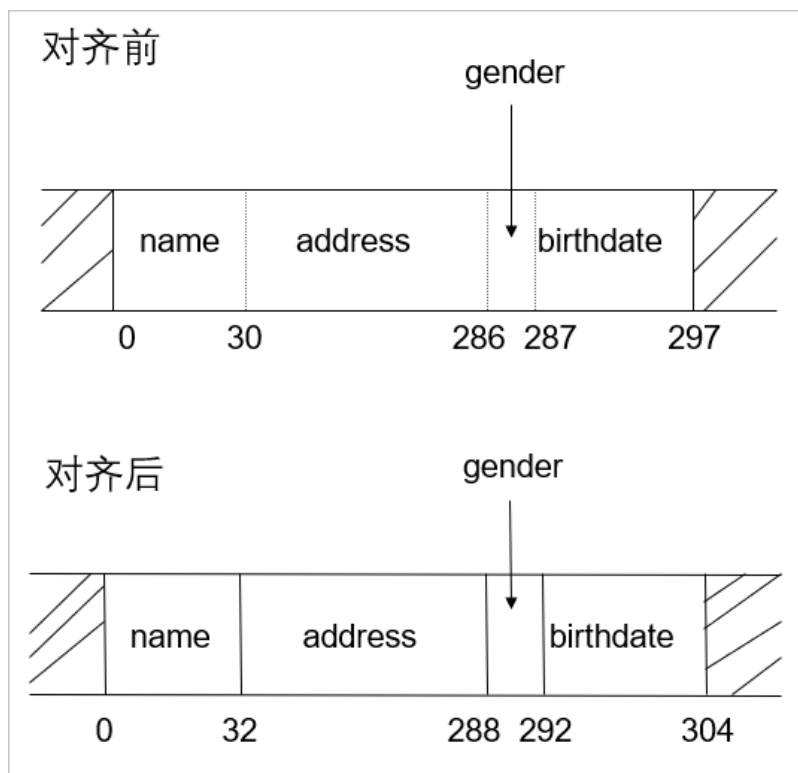
数据字段的长度固定，例如 int、short、date 等。

- 变长类型

数据字段的长度非固定，例如 varchar、blob、text 等。

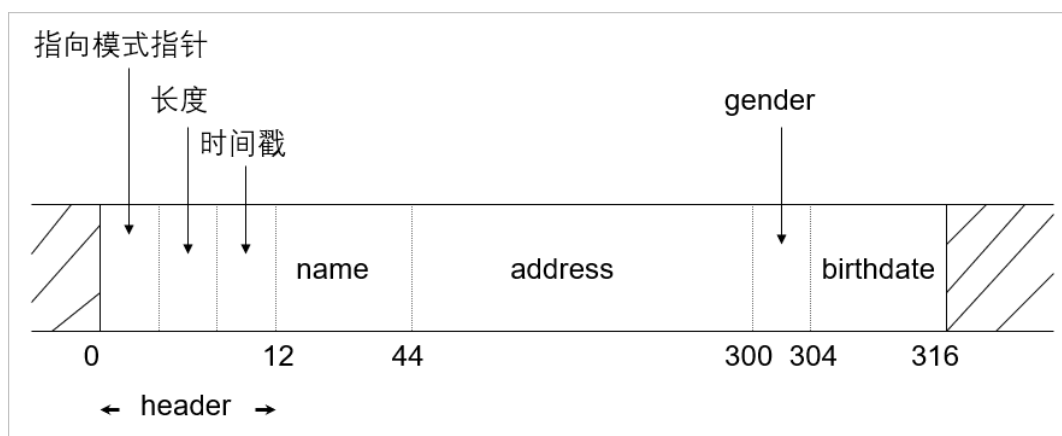
记录的存储结构

了解了字段类型、记录以及块的定义之后，我们来看看一条记录是如何保存的，假设我们运行在 32 位的操作系统上，数据是按照 4 字节对齐存储的，因此数据库在存储定长数据时，也是按照对齐之后的数据进行存储的。



如上图所示，可以看到对齐之后的存储实际占用长度要比对齐之前的多一点，这个是可以接受的，字节对齐对 CPU 进行内存访问时是友好的，感兴趣的同学可以自行了解字节对齐的介绍。

下面我们针对概念介绍中提到的 SQL 结构的数据进行存储示例：



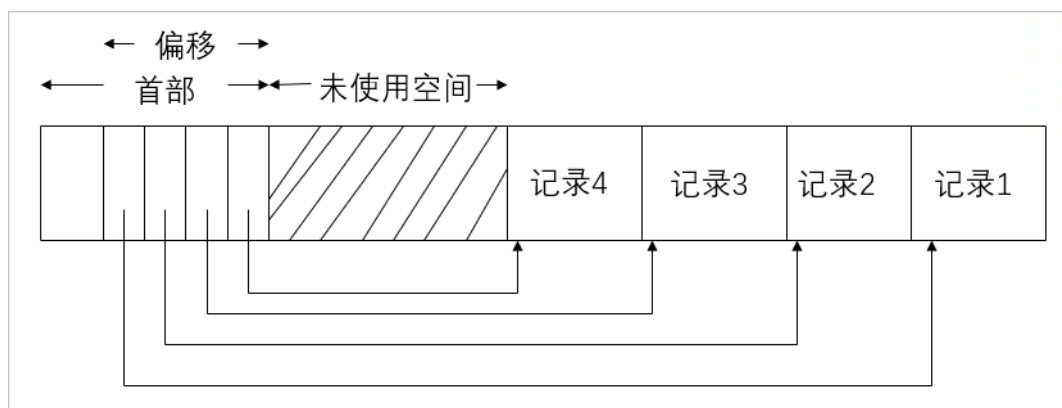
例如一条完整的记录里，通常除了存储具体的记录字段之外，还会额外存储这条记录的元数据信息，并且会把这些元数据信息放到记录最开始的位置，这部分内容称为记录的 header。

header 中存放的元数据信息主要有变长字段的偏移、字段的长度、记录最后修改的时间戳等（不同场景的记录需要存储的元数据可能是不一样的）。header 之后是具体每个字段的内容，这些内容一般叫做 data，data 中的内容会按照表结构的定义进行编排，因此一条完整的记录是按照 header + data 来存储的。

块的存储结构

块在存储中的组织与记录类似，可以简单地看成是由 header + data 数据组成，例如一个块要管理内部所有的记录，需要在块的 header 中保存每个记录的偏移、块最后修改的时间戳、块在存储结构中的位置信息等，因此 header 中的元数据主要是为了方便自身的管理和维护。

下面我们来看一个块里面是如何存放数据的，这里需要强调的是，块里面存储的地址偏移是相对于虚拟地址的偏移，并非实际物理磁盘的偏移。还需要注意，新插入的记录，是从块的末端往中间增长的，而未使用的空间是从 header 之后往块的末端增长的。



从图中可以看出记录 1 在最后，记录 4 在最前面，因为记录有可能不等长，无法事先根据记录的长度和块的总长来计算好能插入多少数据。

示例：假如一个块的大小是 100

- 如果记录都等长，都为 10，那么我们可以算出一个块最多能插入 10 条记录，并且可以从块的起始到末端的方向插入，最后刚好到 100 不会报错。
- 如果记录不等长，事先也不知道这个块最多能容纳多少条记录的，如一条长度 10，一条长度 50 的记录，从起始往末端方向编排，如果下一条记录的长度为 50，因为不知道块中还有多少空间，也不会提前报错，那必然会插入失败。

但是如果记录是从块的末端往前增长，那在插入 10 和 50 这两条记录后，由于长度为 50 的那条记录的偏移是 40 ($100 - 10 - 50 = 40$)，那就能知道当前的空间只剩 40 了，没办法容纳新的记录，然后提前报错然后重新分配块。

2.4 变长数据和记录

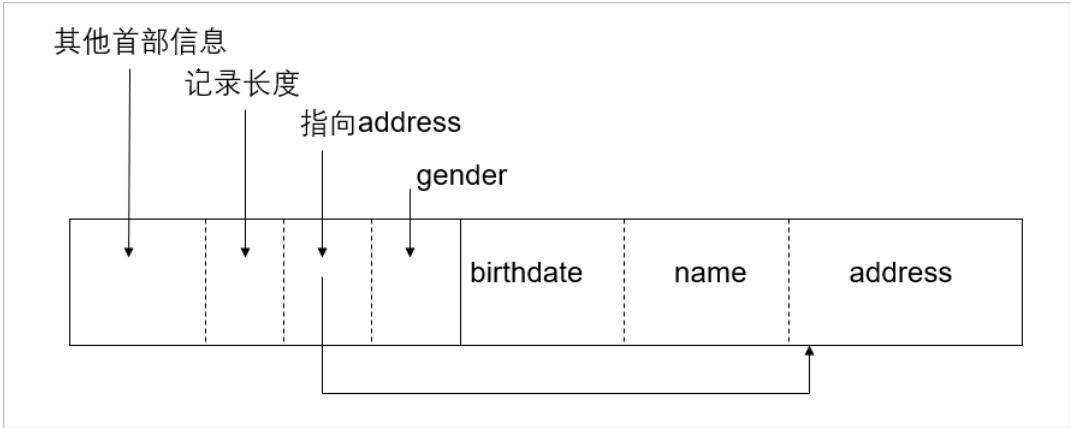
变长数据定义和类型

变长记录主要分为以下几种类型：

- 大小变化的数据项
字段中含有 varchar 等不定长字段。
- 重复字段
一行记录中一个字段重复出现多次，但重复的次数没有规律。
- 可变格式记录
不知道记录中有哪些字段、字段是否有重复、字段的类型、字段的长度等，例如 XML 类型的数据。
- 大字段
通常说的 blob 字段，可能需要几个块链接在一起存储。

具有变长字段的记录

例如上节示例的 SQL 语句中的 address 字段，是一个 varchar 类型，在存储中一般的存储原则是先存定长，再存变长。例如在 header 之后，使用一些存储空间来保存变长字段的地址长度和具体地址，最后通过访问该地址的若干长度，即可访问变长字段的完整内容。

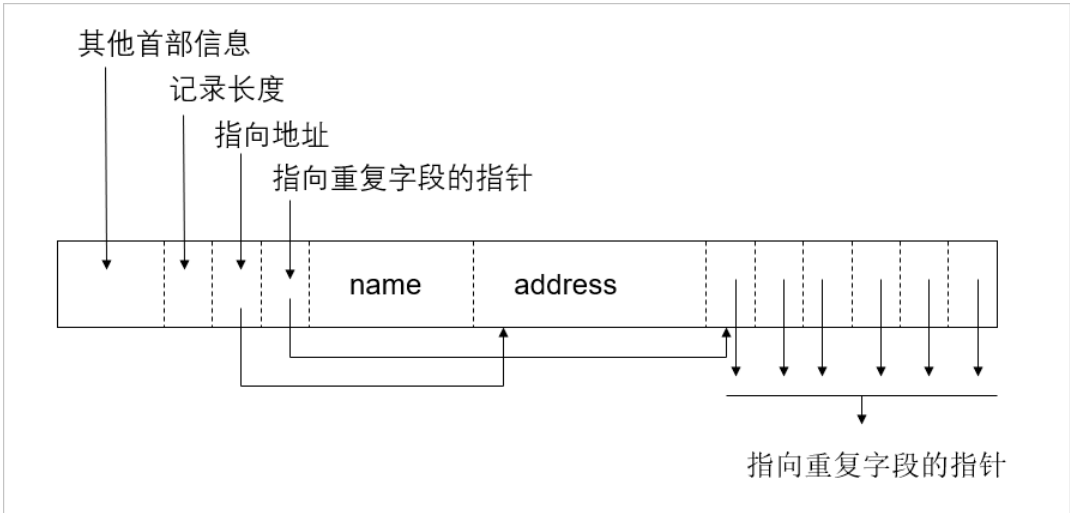


具有重复字段的记录

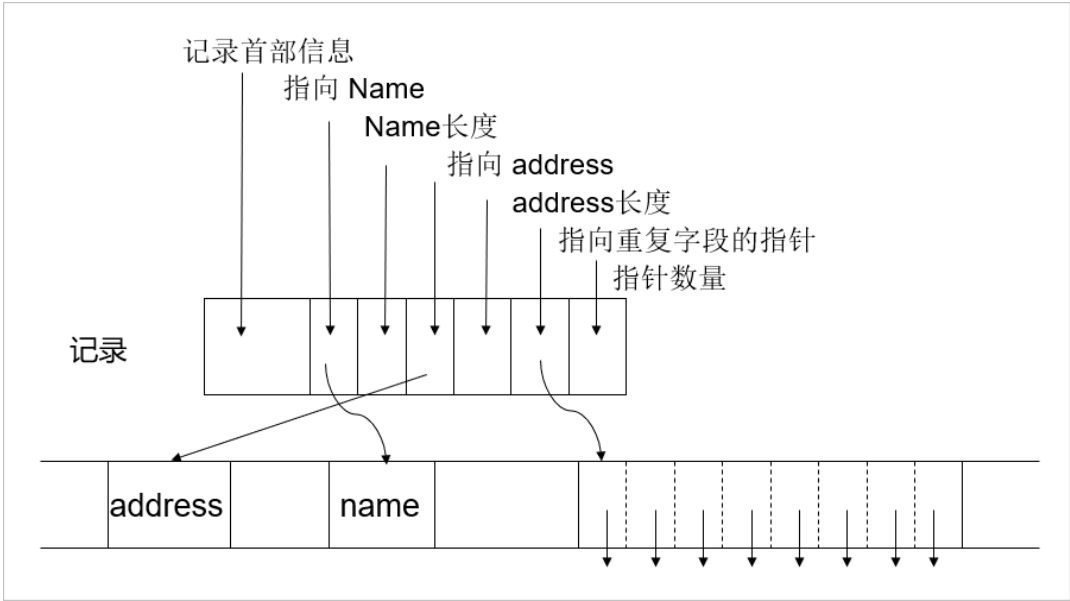
重复字段有两种存储方式：

第一种与变长字段的存储类似，其思想是把重复字段的地址使用一段空间进行保存，并把重复的字段紧密的放到行

的最后，再根据这个地址访问到第一个重复字段，因为字段长度已知，因此从第一个地址往后遍历直到获取所有重复字段，如下图所示。



另外一种存储方式是通过类似索引的方式分块存储。如下图所示，在第一个块上，除了记录定长字段外，还需要把非定长字段的指针以及非定长字段的长度保存起来，而指针指向的是另外一个块具体的数据，这种方式有好也有坏，好处是保持了第一个块上所有字段的定长属性，方便查询，坏处是访问数据时增加了额外的磁盘 IO。

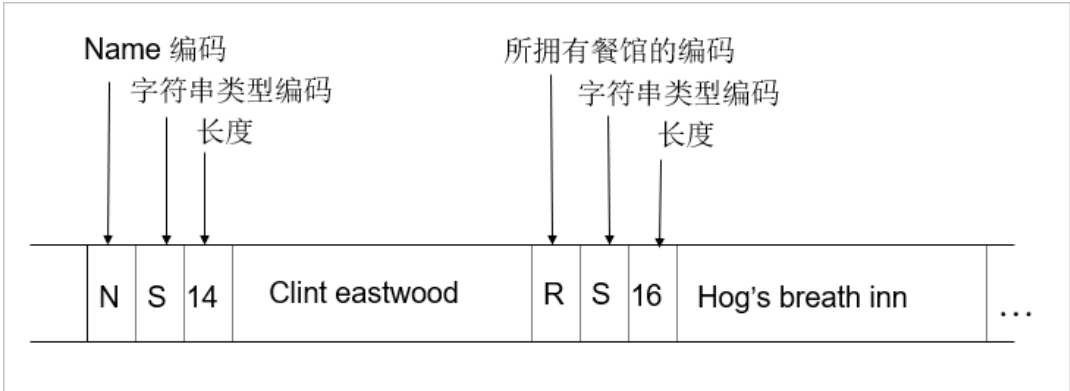


具有可变格式的记录

可变格式的记录，通俗来讲就是字段没有固定格式的记录，在数据库的维度来说是指没有具体表结构的数据，是非结构化的数据，例如 XML 文件。

示例：假设要记录一个人的信息，包括姓名、投资的餐馆等，可能字段固定但是内容不固定。这个时候可以使用标记（如姓名）以及类型（如 string）加上长度来记录，例如下图中的 N 表示 name，S 表示 string 类型，14 表示 name 的长度。这样做的好处是即便不知道存储的格式，但是可以把一些特征标识保存起来，最后根据标识解析出

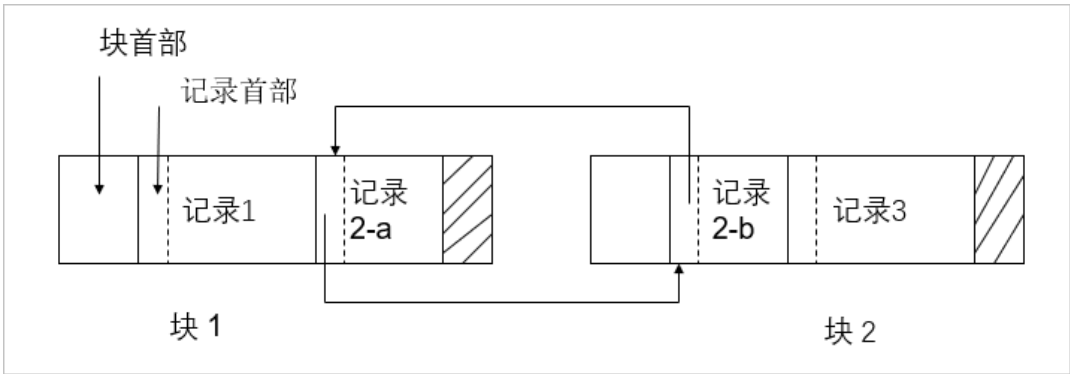
数据。



具有大值类型的记录

所谓大值，是指在一个块中无法存储的记录，大值涉及多个块之间的交互。

示例：如下图记录 2 存储时，分成了两部分进行存储，分别是记录 2-a 和记录 2-b，其中在存储记录 2-a 的时候，需要额外的存储空间来保存一些数据：一是标识，表明这是一个记录的片段；二是片段的序号，如这是第 1 个片段，或者第 N 个片段；最后还要保存指向下一个片段的指针，用于跨块之间的访问。



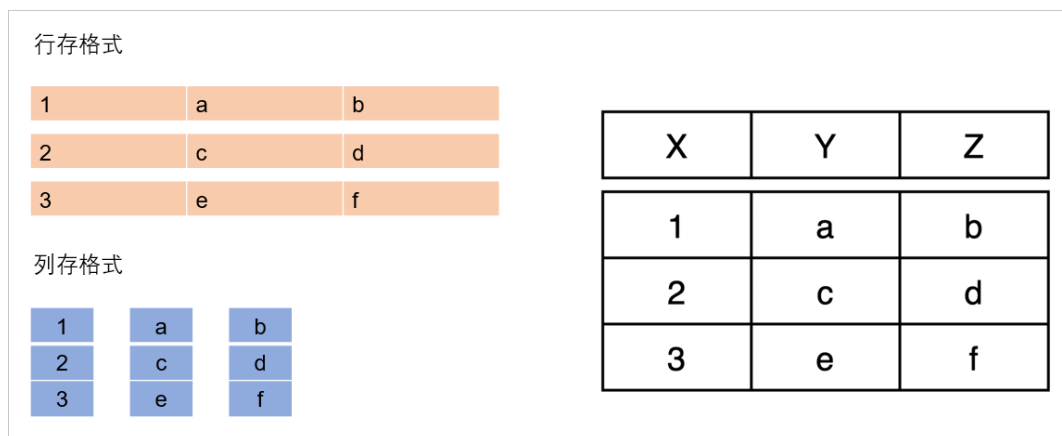
BLOB 二进制大对象

blob 是大值字段类型，常见的 blob 是音视频文件。通常 blob 需要多个连续块，或者块的链表进行保存，并通过分散保存在多盘中间，提高单位时间的访问效率。

例如我们在观看电影时，电影数据并不会全部加载到内存中，而是在播放到某个片段之前，把对应的数据从磁盘中加载到内存，或者我们快进到某个片段时，能够快速的将该片段对应的内容加载进来。这里其实就是建立 blob 的索引来快速定位到具体的 blob 块。

列存储

本节只简单介绍列存的概念，暂不对列存做深入讨论。



- 行存

简单来说就是按行存储，例如上图有一张表，表结构中有 X Y Z 三个字段，表中插入了三条记录。如图中黄色是行存的形式，可以看到数据是按照表结构的行来存储的，大部分传统的关系型数据库都是以行存为基础。

- 列存

列存顾名思义是按照表结构的列来存储，如上图中蓝色是列存的形式。

可能有人会有疑问，如果插入了很多行记录，按照列存会不会导致一列的数据非常大？当然是会的，但是数据库会把这一列使用多个存储单元进行存储，甚至可能跨越多个磁盘。

即便这样，列存也有天然的优势，例如因为列值的类型都一样，列存天然适合做数据压缩，这对于控制存储成本是非常有利的。再者，我们可能只针对某几列的数据进行数据分析，因此列存也非常合适 AP 系统。列存的具体优缺点如下：

优点：

- 按需读取列，减少非必要数据的访问
- 列字段类型统一，非常利于做压缩
- 利于做列的聚合操作
- 适合做大数据分析，AP 性能好

缺点：

- 写入/更新耗时相对行存较长，TP 性能差
- 不适合做频繁的删除操作

2.5 记录的修改

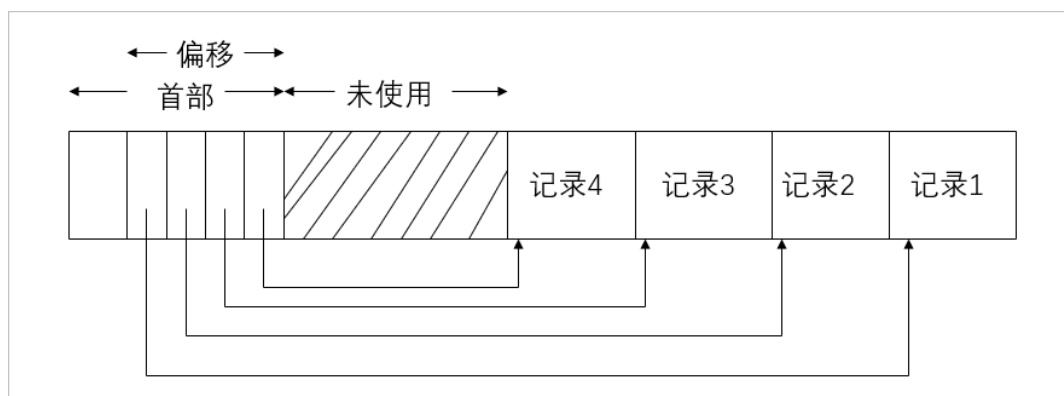
记录的修改主要分为插入、更新和删除三种场景。

数据插入

数据插入在存储中的流程可以简单概括为：

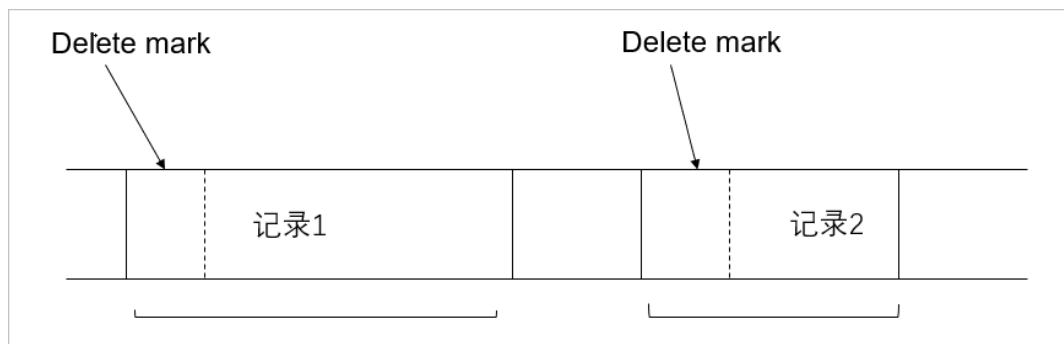
1. 找到空闲块
2. 把记录插入块的空闲位置
3. 维护新插入记录的偏移

如下图所示，先从未使用的空间中找到可以插入的位置，写入记录后，再把记录相对于块的偏移写入到偏移表中进行维护。



数据删除

数据删除一般在数据库中的实现，并非把实际的数据内容马上从存储介质中擦除，而是在需要删除的记录中打上一个删除的标记，例如删除了记录 1，则把记录 1 前面的一个字节或一个 bit 标记为 delete，然后数据库系统再根据具体的场景（如 GC 流程），再统一回收这部分的空间。



数据修改

记录的修改主要有两种场景：

- 定长记录的修改

定长记录的修改，因为记录定长，因此可以使用覆盖写的方式进行修改，把修改后的数据在原地址中覆盖写入。

- 变长记录的修改

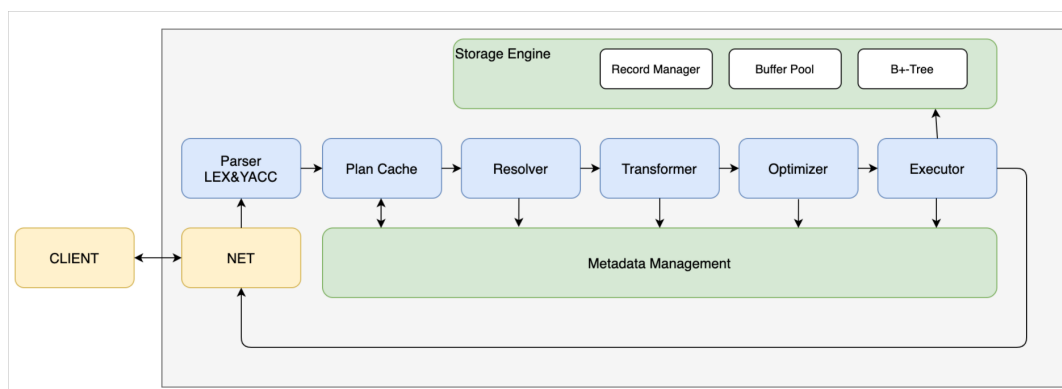
变长记录的修改，因为修改后数据的长度可能会发生变更，所以不能使用覆盖写这种方式。而是会在块中重新查找空闲的位置，把修改的记录写入到新位置中，最后对原记录打上删除的标记，避免旧数据被访问。

2.6 MiniOB 存储实现原理

本节将从存储层面介绍 MiniOB 的实现。

MiniOB 框架简介

首先回顾一下 MiniOB 的框架，在 MiniOB 概述章节已经简单的介绍过，本节重点介绍执行器（Executor）访问的存储引擎。

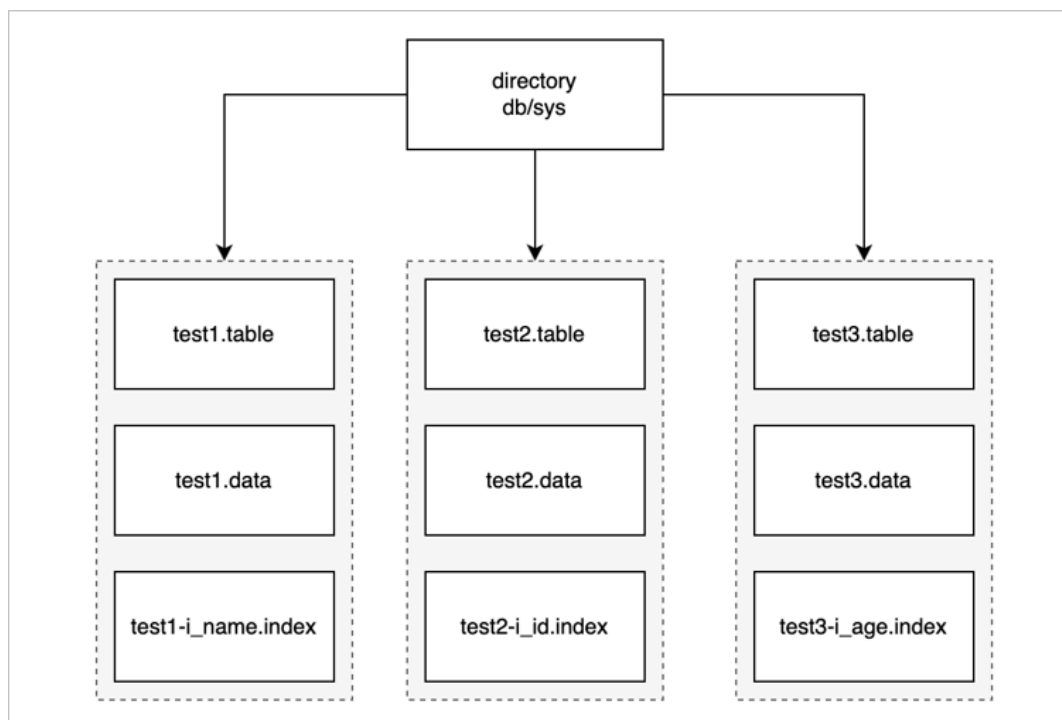


存储引擎控制整个数据、记录是如何在文件和磁盘中存储，以及如何跟内部 SQL 模块之间进行交互。存储引擎中有三个关键模块：

- Record Manager：组织记录一行数据在文件中如何存放。
- Buffer Pool：文件跟内存交互的关键组件。
- B+Tree：索引结构。

MiniOB 文件管理

首先介绍 MiniOB 中文件是怎么存放，文件需要管理一些基础对象，如数据结构、表、索引。数据库在 MiniOB 这里体现就是一个文件夹，如下图所示，最上面就是一个目录，MiniOB 启动后会默认创建一个 sys 数据库，所有的操作都默认在 sys 中。



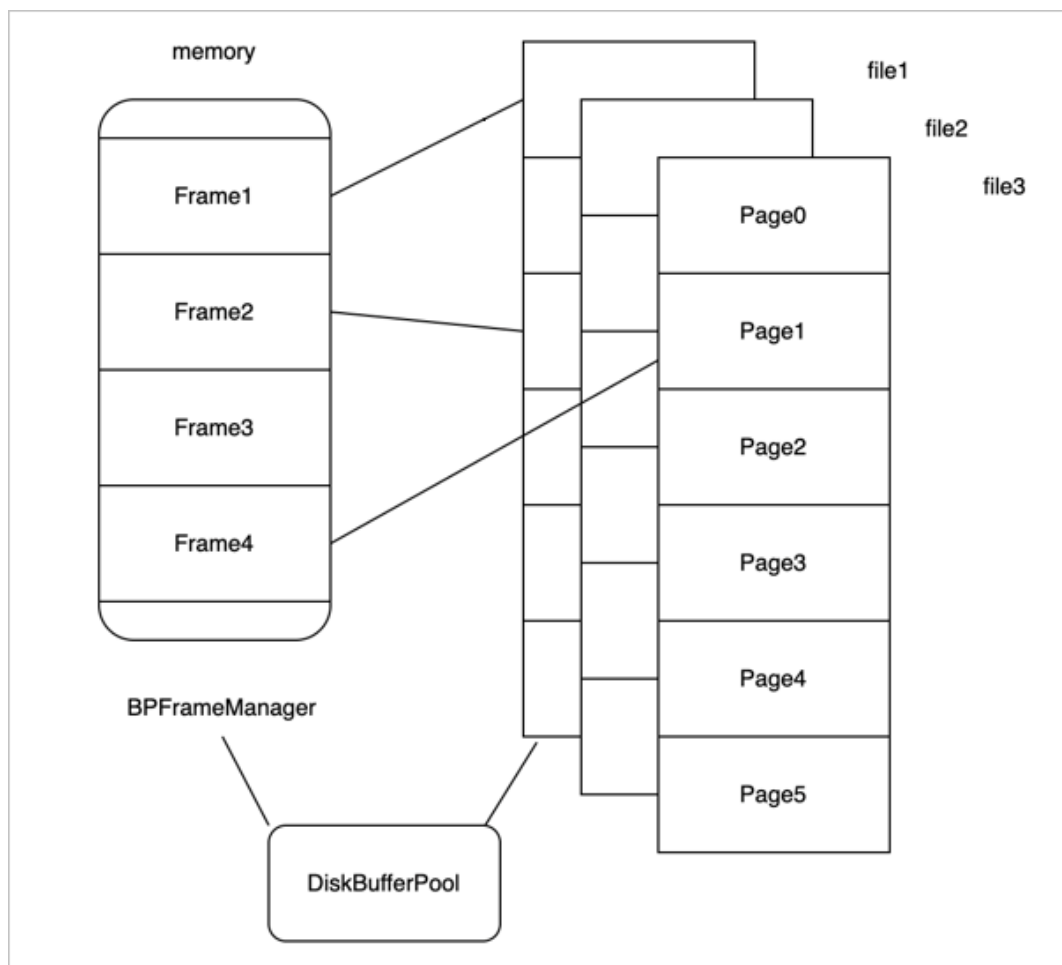
一个数据库下会有多张表。上图示例中只有三张表，接下来以 test1 表为例介绍一下表里都存放什么内容。

- test1.table: 元数据文件，这里面存放了一些元数据。如：表名、数据的索引、字段类型、类型长度等。
- test1.data: 数据文件，真正记录存放的文件。
- test1-i_name.index: 索引文件，索引文件有很多个，这里只展示一个示例。

MiniOB Buffer Pool 模块介绍

Buffer Pool 在传统数据库里是非常重要的基础组件。

首先来了解一下为什么要有一个 Buffer Pool，数据库的数据是存放在磁盘里的，但不能直接从磁盘读取数据，而是需要先把磁盘的数据读取到内存中，再在 CPU 做一些运算之后，展示给前端用户。写入也是一样的，一般都会先写入到内存，再把内存中的数据写入到磁盘。这种做法也是一个很常见的缓存机制。



接着来看 Buffer Pool 在 MiniOB 中是如何组织的。如上图所示，左边是内存，把内存拆分成不同的帧（frame）。假如内存中有四个 frame，对应了右边的多个文件，每个文件按照每页来划分，每个页的大小都是固定的，每个页读取时是以页为单位跟内存中的一个 frame 相对应。

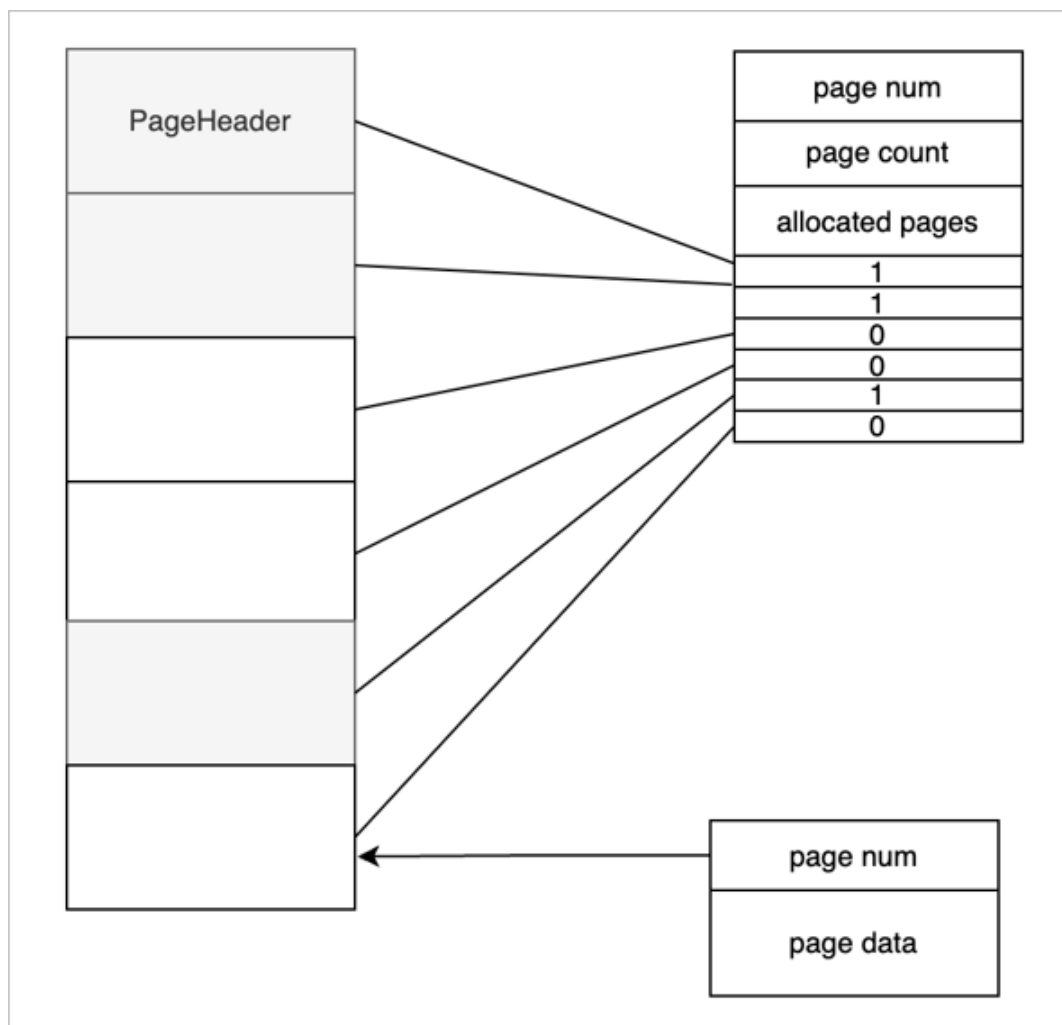
Buffer Pool 在 MiniOB 里面组织的时候，一个 DiskBufferPool 对象对应一个物理文件。所有的 DiskBufferPool 都使用一个内存页帧管理组件 BPFrameManager，他是公用的。

再来看下读取文件时，怎么跟内存去做交互的。如上图所示，frame1 关联了磁盘中一个文件的页面，frame2 关联了另一个页面，frame3 是空闲页面，没有关联任何磁盘文件，frame4 也关联了一个页面。

比如现在要去读取 file3 的 Page3 页面，首先需要从 BPFrameManager 里面去找一个空闲的 frame，很明显，就是 frame3，然后再把 frame3 跟它关联起来，把 Page3 的数据读取到 frame3 里。现在内存中的所有 frame 都对应了物理页面。

如果再去读取一个页面，如 Page5，这时候已经找不到内存了，通常有两种情况：

- 内存还有空闲空间，可以再申请一个 frame，跟 Page5 关联起来。
- 内存没有空闲空间，还要再去读 Page4，已经没有办法去申请新的内存了。此时就需要从现有的 frame 中淘汰一个页面，比如把 frame1 淘汰掉了，然后把 frame1 跟 Page4 关联起来，再把 Page4 的数据读取到 frame1 里面。淘汰机制也是有一些淘汰条件和算法的，可以先做简单的了解，暂时先不深入讨论细节。



再来看一下，一个物理的文件上面都有哪些组织结构，如上图所示。

- 文件上的第一页称为页头或文件头。文件头是一个特殊的页面，这个页面上会存放一个页号，这个页号肯定都是零号页，即 page num 是 0。
- page count 表示当前的文件一共有多少个页面。
- allocated pages 表示已经分配了多少个页面。如图所示标灰的是已经分配的三个页面。
- Bitmap 表示每一个 bit 位当前对应的页面的分配状态，1 已分配页面，0 空闲页面。

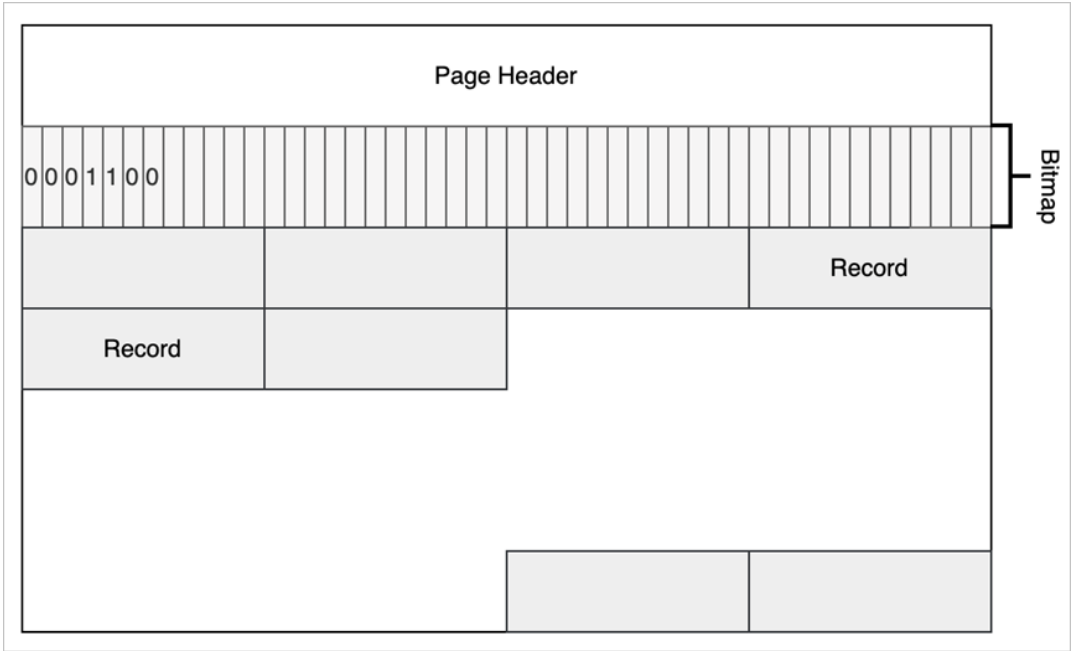
当前这一种组织结构是有一个缺陷的，整个文件能够支持的页面的个数受页面大小的限制，也就是说能够申请的页面的个数受页面大小的限制的。有兴趣的，可以思考一下怎么能实现一个无限大或支持更大页面的算法。

接下来介绍一下普通页面（除 PageHeader 外），普通页面对 Buffer Pool 来说，第一个字段是用四字节的 int 来表示，就是 page num。接下来是数据，这个数据是由使用 Buffer Pool 的一些模块去控制。比如 Record Manage 或 B+Tree，他们会定义自己的结构，但第一个字段都是 page num，业务模块使用都是 page data 去做组织。

MiniOB 记录管理

记录管理模块（Record Manager）主要负责组织记录在磁盘上的存放，以及处理记录的新增与删除。需要尽可能高效的利用磁盘空间，尽量减少空洞，支持高效的查找和新增操作。

MiniOB 的 Record Manager 做了简化，有一些假设，记录通常都是比较短的，加上页表头，不会超出一个页面的大小。另外记录都是固定长度的，这个简化让学习 MiniOB 变得更简单一点。



上面的图片展示了 MiniOB 的 Record Manager 是怎么实现的，以及 Record 在文件中是如何组织的。

Record Manage 是在 Buffer Pool 的基础上实现的，比如 page0 是 Buffer Pool 里面使用的元数据，Record Manage 利用了其他的一些页面。每个页面有一个头信息 Page Header，一个 Bitmap，Bitmap 为 0 表示最近的记录是不是已经有有效数据；1 表示有有效数据。Page Header 中记录了当前页面一共有多少记录、最多可以容纳多少记录、每个记录的实际长度与对齐后的长度等信息。

2.7 课后实践

MiniOB 题目：实现 drop table 功能

- 当前 MiniOB 已经拥有了创建表的功能，请参考建表的代码，实现 drop table 的功能，并在 [训练营](#) 上提交测试，训练营使用文档请参见 [训练营使用说明](#)。

说明

删表是建表的逆过程，是把表的元数据、关联的资源都删除。

- 训练营测试的原理：后台执行时，有一系列的 SQL 语句，发送到 MiniOB，然后将执行输出的结果，跟预期的 result 文件做对比。

说明

删表时，要删除所有与表相关的资源。

第 3 章：索引结构

本节主要介绍数据库索引结构基础，以及 MiniOB B+Tree 的实现。

本章目录

3.1 B+Tree

3.2 散列表

3.3 LSM-Tree

3.4 MiniOB B+Tree 实现

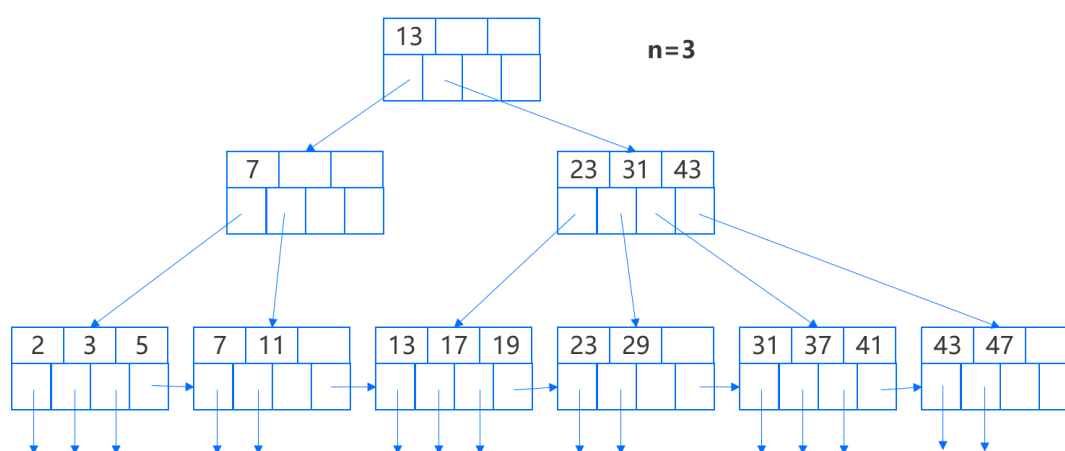
3.5 课后实践

3.1 B+Tree

概念介绍

在商用系统中经常使用一类通用的数据结构，统称为 B 树。其中最常使用的是 B+ 树（B 树的变体），接下来我们针对 B+ 树的具体细节进行介绍。

B+ 树将存储块组织成一棵树的形状，每个结点是一个存储块，这棵树是平衡的，即从树根到树叶的所有路径都一样长。通常 B+ 树有三层：根、中间层和叶，但也可以是任意多层。

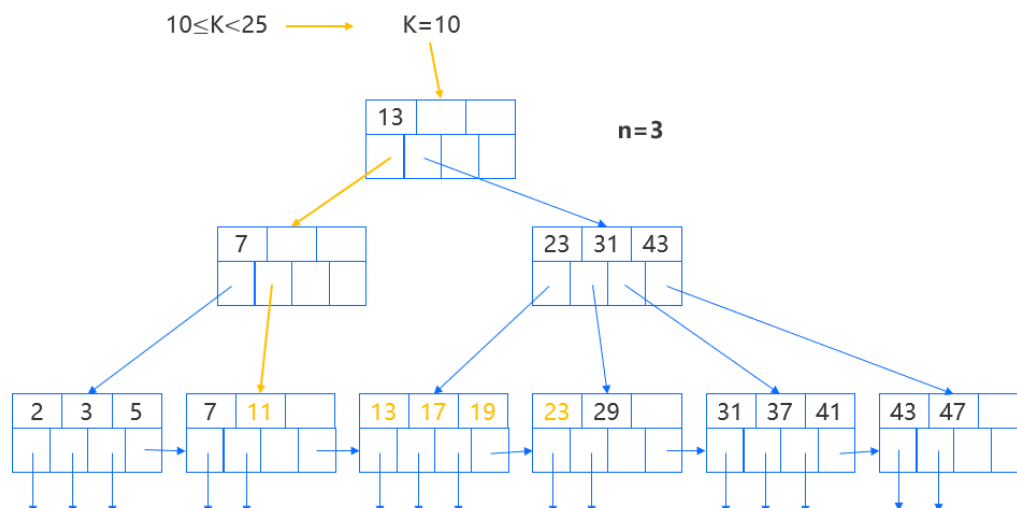


如图所示，对于每个 B+ 树都有一个参数 n 来决定 B+ 树所有存储块的布局，每个存储块存放 n 个查找键和 $n+1$ 个指针，我们希望在存储块能够容纳的前提下 n 取值尽可能大，从而使得树的层数尽可能少，来减少每次访问叶结点所需要访问的存储块。

接下来介绍几个 B+ 树的常用基础规则，这些限制并不适用于所有 B+ 树，比如叶结点通常存放键值对，而不是指向数据的指针。但无论具体实现如何，通常都需要保证树是平衡的。

- 根结点至少有两个指针被使用，所有指针指向位于下一层的存储块。
- 叶结点中，最后一个指针指向它右边的下一个叶结点存储块。其他 n 个指针中，至少有 $\lfloor (n+1)/2 \rfloor$ 个指针被使用且指向数据记录。
- 在内层结点中，所有的 $n+1$ 个指针都可以用来指向下一层的块。其中至少 $\lfloor (n+1)/2 \rfloor$ 个指针被使用，如果 j 个指针被使用，那该块中将有 $j-1$ 个键。
- 所有被使用的键和指针通常都存放在数据块开头位置，除了叶结点的第 $(n+1)$ 个指针。

在上图示例中 $n=3$ ，也就是说，块中可存放 3 个键值和 4 个指针。



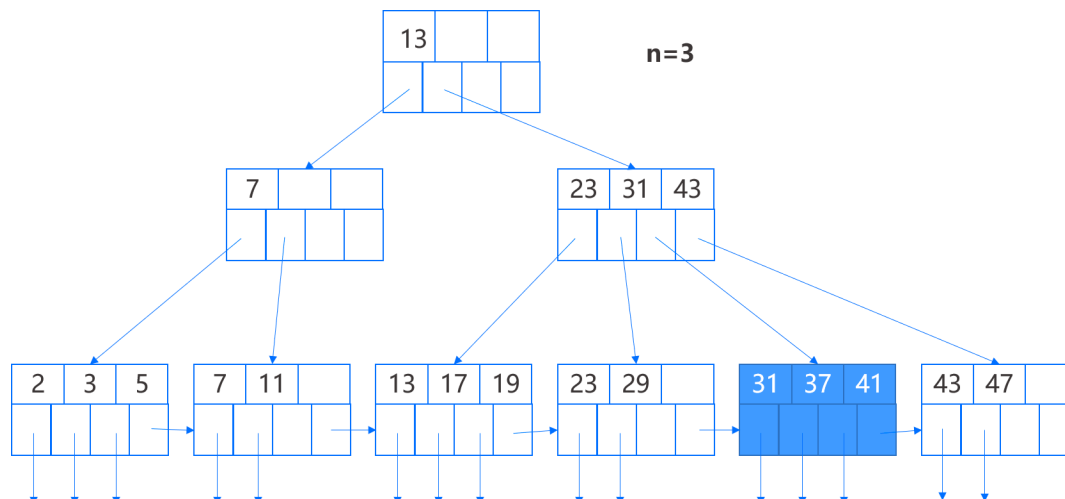
由于 B+ 树的叶结点前后存在链接，对于范围查询非常友好。以查询 10~25 范围的键为例，首先通过键等于左边界 10 进行查找，找到叶结点后按顺序向后依次判断键是否位于范围内，将位于范围内的记录取出即可。

B+ 树的插入

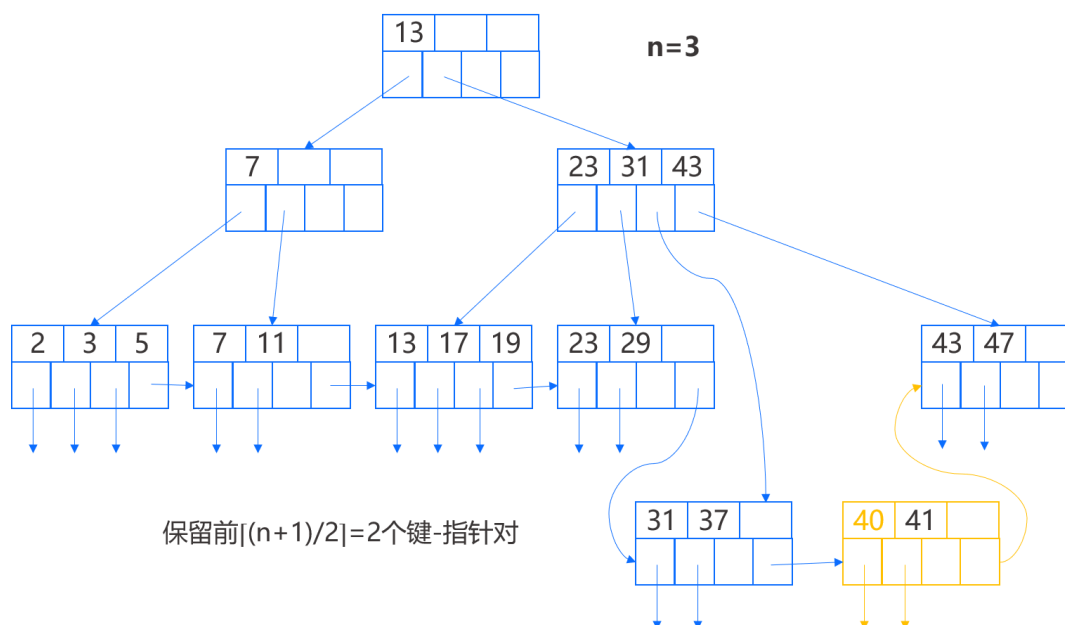
B+ 树的插入在原则上是递归的：

- 首先设法在适当的叶结点为新键找到空闲空间，如果有的，就把键放在那里。
- 如果在适当的叶结点中没有空间，就把该叶结点分裂成两个，并且把其中的键分到这两个新结点中，使每个新结点有一半或刚好一半的键。
- 某一层的结点分裂在上一层看来，相当于要在这一较高层次上插入一个新的键-指针对。因此可以在这一较高层次上递归上述插入策略。
- 例外的情况是，在试图插入键到根结点且其没有空间时，将分裂根结点成两个结点且在更上一层创建一个新的根结点，这个新的根结点有两个刚分裂成的结点作为它的子结点。

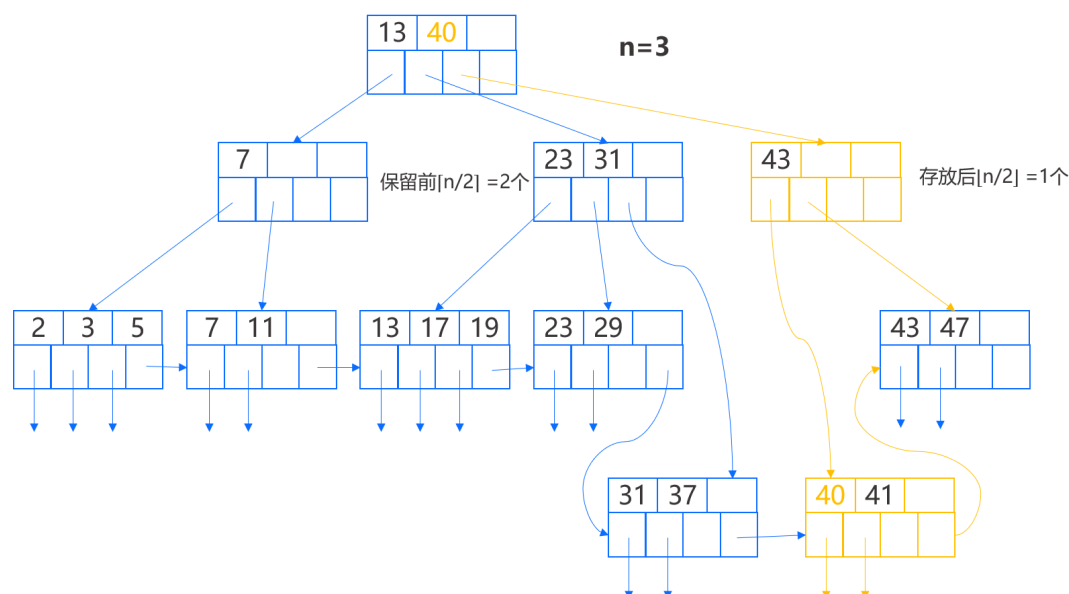
当我们分裂结点并在其父结点插入时，需要小心处理。我们结合实例来进行解读，向下图中的 B+ 树插入键 40，根据前述的查找过程我们将找到第 5 个叶结点进行插入，因为该叶结点已满，我们需要分裂这个叶结点。



首先创建一个新结点，在原结点保留 2 个键-指针对，并把其他两个键-指针对移到新结点。



现在我们必须插入一个指向新叶结点的指针到它上一层的结点，并将该指针与键 40 关联起来，因为键 40 是通过新叶结点可访问到的最小键。同样该父结点已满，需要分裂。

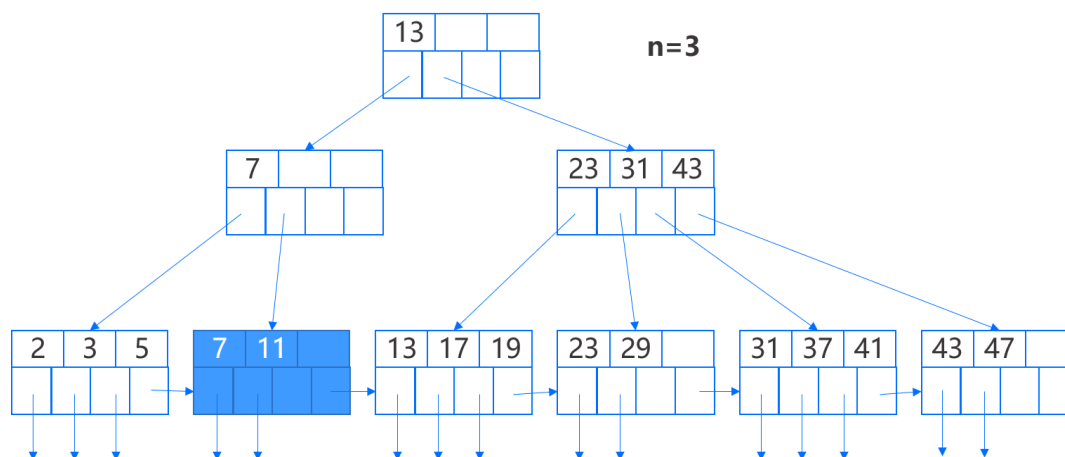


理论上我们将在父结点增加键 40，形成 23、31、40、43 的键序列，而该结点已满，因此我们将前 $\lfloor n/2 \rfloor$ 个键，即 23、31 保留在原父结点中，后 $\lfloor n/2 \rfloor$ 个键，即 43 移到新结点，而中间那个键将移动到更上一层的父结点，用来划分这两个结点之间的查找。

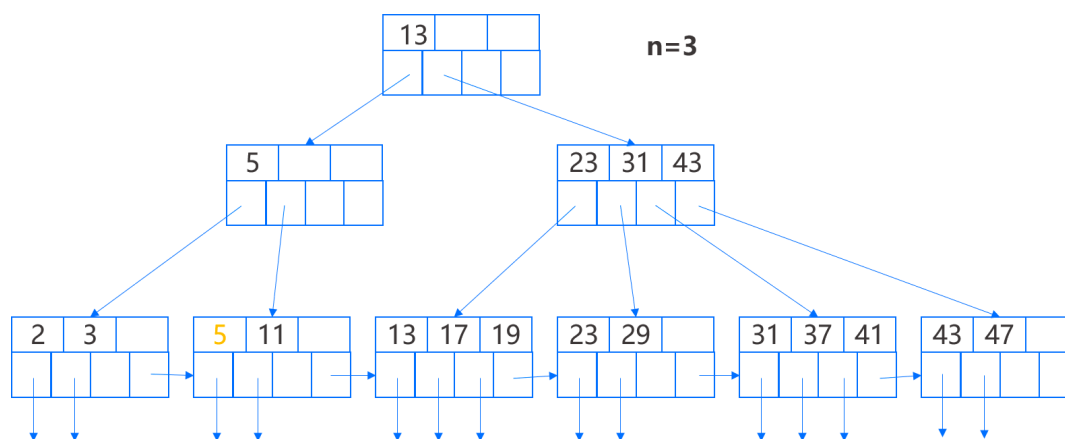
B+ 树的删除

如果我们找到叶结点 N，并将键-指针对删除后，结点仍然满足充满度的最小约束，那么我们不需要再额外处理。但反之，我们需要为这个结点做以下两件事之一：

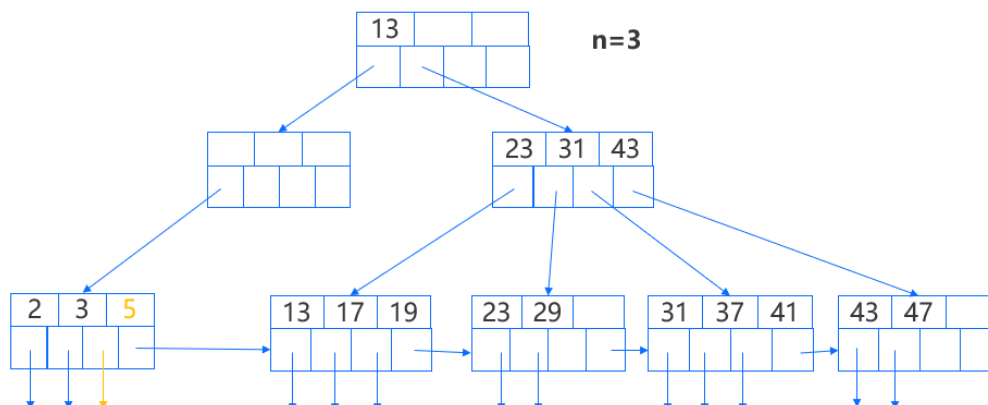
- 重构
 - 如果与结点 N 相邻的左右兄弟中有一个的键-指针对数目超过最小约束，那么将其中的一个键-指针对移到结点 N 并保持键的顺序，并且按需调整这两个结点的父结点的键。
 - 如果从右兄弟结点借键，需要保证借到最小键。
- 合并
 - 如果两兄弟没有一个能够提供键值给结点 N 时，兄弟结点的键数必然为最小数目，那么我们把结点 N 和其中一个兄弟合并成一个新结点，即删除其中一个结点，此时新结点并不会超过最大约束。并且我们需要调整父结点中的键并删除其中一个键-指针对。
 - 如果删除后父结点不再满足最小约束，那么我们需要在父结点上递归的运用这个删除算法。



如上图 B+ 树, 假设我们删除键 7, 该键位于第二个叶结点, 我们将其键-指针对以及指向的记录删除, 此时结点只剩下一个键, 而我们要求每个叶结点至少有两个键, 因此我们从左兄弟移过来一个键-指针对, 并将父结点的键从 7 改为 5。

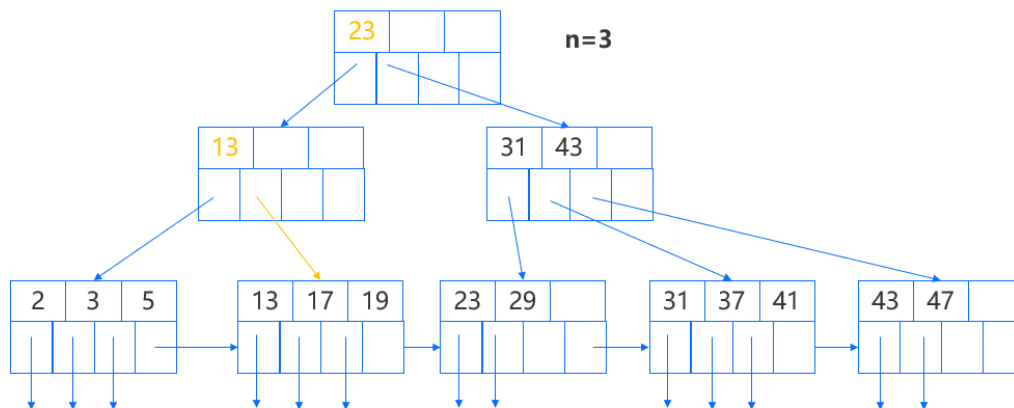


下一步, 我们再删除键 11, 这个删除对第二个叶结点产生同样的影响, 但这次我们不能从第一个结点借键, 因为后者的键数达到了最小数, 另外, 它的右边没有兄弟, 因此我们需要合并第二个叶结点和它的兄弟, 即第一个叶结点。合并后我们调整父结点, 具体来说, 它的两个指针被换成了一个指针, 并且键 5 不再有用也被删除。



此时该父结点从右兄弟结点获得了一个指针, 并更新该指针对应的键为 13, 而键 13 原来位于根结点且表示通过那

个被转移的指针可访问的最小键，因此根结点的键需要修改为 23，表示当前通过根结点第二个子结点可访问到的最小键。



在插入和删除中的特殊操作本质上都是为了保证 B+ 树始终处于平衡状态。

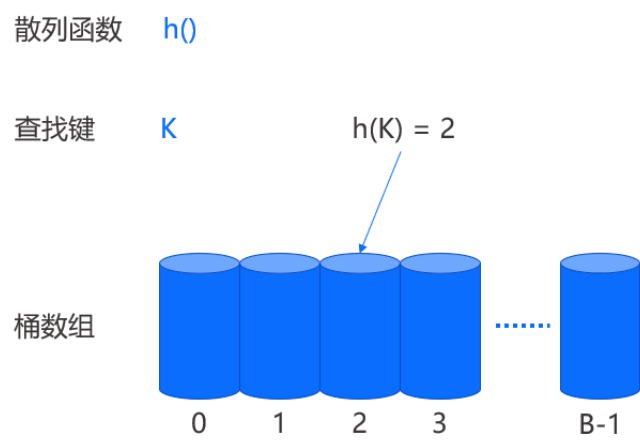
这里我们提两点 B+ 树存在的主要问题：

- 随着数据量的增长，树的层数增加，叶结点的访问路径变长，开销会逐渐增大，但这通常发生在数据量足够大的场景下；
- 高速海量的数据更新下，容易发生结点的频繁分裂与合并，这种行为在树的多层之间迭代传递从而带来大量开销。

3.2 散列表

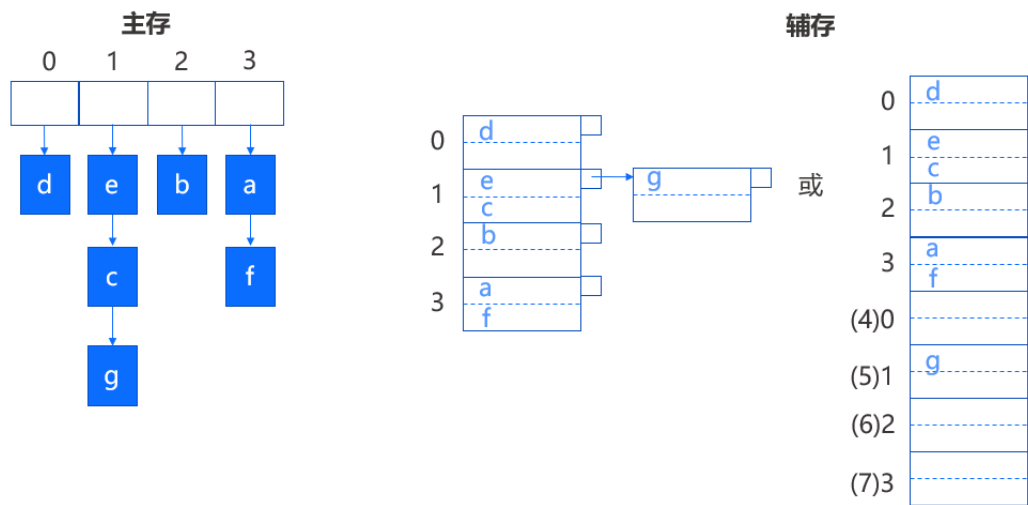
简介

散列表又称哈希表，是线性表中一种重要的存储方式和检索方法。在这种结构中有一个散列函数 h ，它以查找键（散列键）为参数并计算出一个介于 0 到 $B-1$ 的整数，其中 B 是桶的数目。桶数组表示一个序号从 0 到 $B-1$ 的数组，其中包含了 B 个数据集合，每个对应于数组中的一个桶。如果记录的查找键为 K ，那么我们通过将该记录映射到 $h(K)$ 的桶中来存储它。



在主存中，桶数组通常由指向链表头的指针组成，每个数据集合由链表构成。

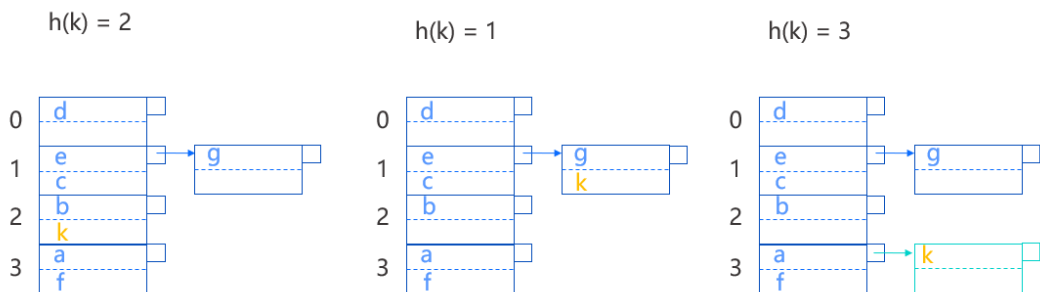
在记录更多的辅存中，桶数组通常由一个或多个存储块组成，多个存储块可以通过存储附加信息来链接，或是将存储块顺序存放，通过存储块号取余来划分同一个桶中的多个存储块。以 $B=4$ 为例，第 5 块存储块可以作为第 0 号桶的第 2 个存储块。



散列表的插入

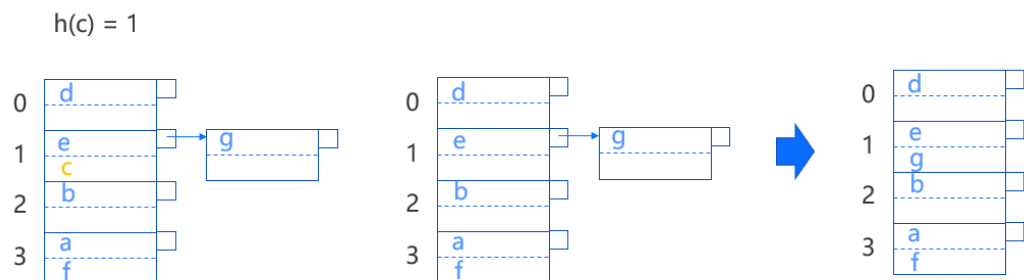
当一个查找键为 k 的新纪录需要被插入时，首先计算 $h(k)$ 。

- 如果桶号为 $h(k)$ 的桶还有空间
 - 把该记录存放在该桶的存储块中，如 $h(k)=2$ ；
 - 存储在多个存储块中尚有空间的存储块中，如 $h(k)=1$ 。
- 如果该桶没有空间，那么就增加一个新的存储块并链接到该桶当前的最后一个存储块，然后将记录存入该块，比如 $h(k)=3$ 。



散列表的删除

删除记录与插入记录的操作方式相同。当查找键为 c 的记录需要被删除时，首先找到桶号为 $h(c)$ 的桶且从中搜索查找键为 c 的记录，继而将找到的记录删除。如果可以将记录在块中移动，那么删除记录后可以选择合并前后两个存储块。



辅存散列表的问题

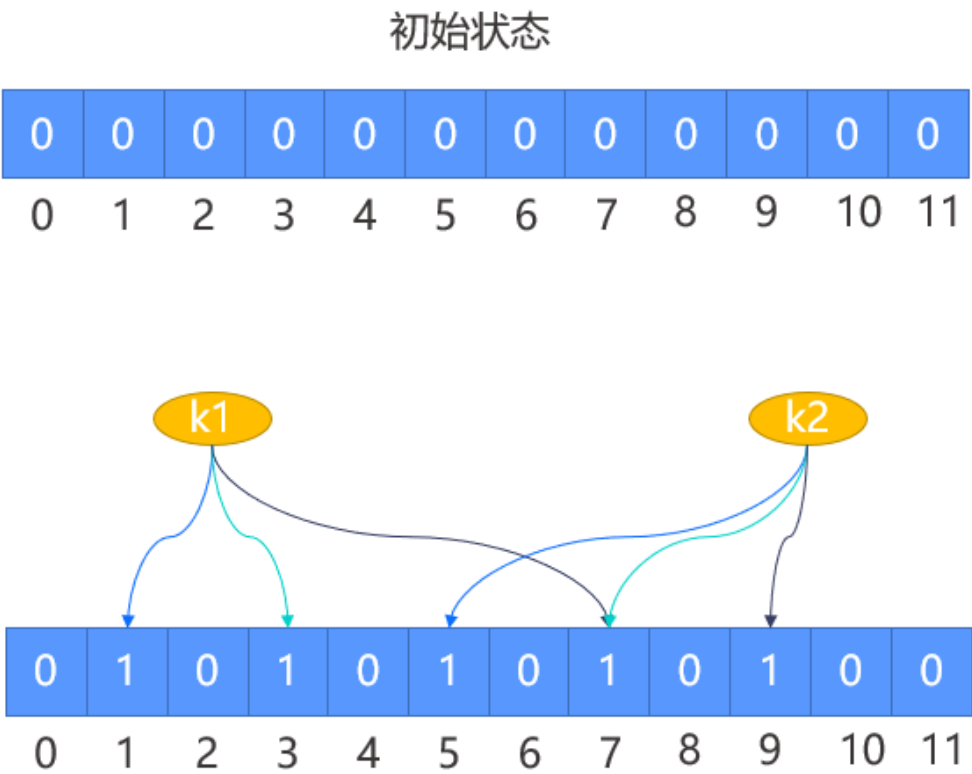
由于记录的无序性，散列表无法支持范围查询，通常的做法是使用混合索引结构或多索引结构来额外支持范围查询。

此外，散列表在理想情况下是有足够的桶，绝大多数桶都只由单个块组成，这样一般查询只需要一次磁盘 I/O，且插入删除也只需两次磁盘 I/O。但如果记录数不断增长，最终就会出现一个桶由许多存储块构成的情况，导致我们需要在多个块中进行查找，每个块至少需要一次磁盘 I/O。

因此，我们必须设法减少每个桶的块数，比如通过减小哈希冲突，使用多个哈希函数的布谷哈希算法，来减少每个存储块的记录数，从而间接减少每个桶的块数。

但更有效的方法其实是减少无效访盘。我们都知道，为了查找一个键是否在块中，通常需要从磁盘读取该块，然后对其中的记录进行比较，而如果该块中不存在查找键，那么这次访盘就相当于一次无效的访盘。如果我们能够尽可能减少这样无效的访盘，那么桶的块数就不是大问题了。

关于这一点，目前常见的解决方案是引入布隆过滤器，这里简单介绍一下原理。布隆过滤器由一个固定大小的二进制向量或位图和多个映射函数组成。初始状态下，对于长度为 m 的位数组，它的所有位都为 0，当有元素加入集合时，通过 K 个映射函数将这个元素映射到位数组中的 k 个点，并把它们置为 1，这里以 3 个映射函数为例。



那么在查询某个元素时，我们只需要查看这些映射位是否都为 1 即可，如果任意一位为 0，被查询元素一定不存在，如果都为 1，则表示可能存在。将布隆过滤器应用在哈希桶的存储块中，则可以过滤元素不存在的存储块。但布隆过滤器是存在误判的，如果映射位都为 1，我们并不能准确判断是否存在，此时必须去读取存储块，这仍可能是一次无效的访盘。

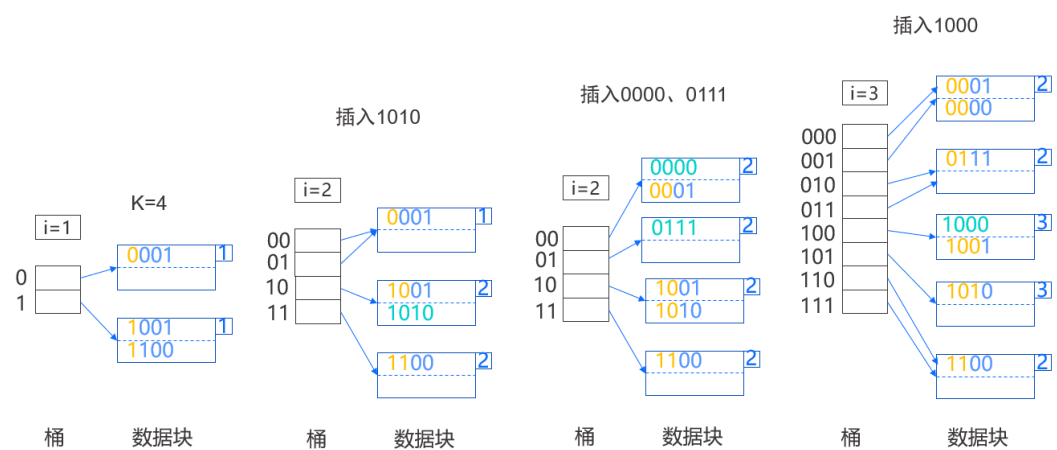
动态散列表

前面介绍的散列表都为静态散列表，因为桶的数目 B 从不改变。但实际上散列表中还有一些动态散列表，它们允

许桶的数目 B 改变，通过桶的数目增长同样能够间接减少每个桶的块数。

这里简单介绍一种动态散列表——可扩展散列表。它使用指向块的指针数组来表示桶，与主存的散列表类似。指针数组能够增长，其长度总是 2 的幂，数组每增长一次桶的数目就翻倍。在某些情况下多个桶可能共享一个存储块。散列函数 h 为每个键计算得到一个 K 位的二进制序列，桶的数目使用 2 的 i 次幂表示，其中 i 指从二进制序列第一位或最后一位算起的 i 位。

如下图所示假设 K=4，当前 i=1，因此桶数组只有两项，分别对应 0 或 1，其中第一个桶存储的是键被散列成以 0 开头的二进制序列的记录，第二个桶存储的是键被散列成以 1 开头的二进制序列的记录。可以注意到每个存储块都存储了一个数字，该数字表示由散列函数得到的二进制序列中有多少位用于确定记录在该块的成员资格。



当我们向其中插入一个散列值为 1010 序列的记录时，因为第一位是 1，所以该记录属于第二块，但该块已满，因此需要分裂，此时发现该块的数字 1 等于 i，因此我们需要将桶数组加倍，从而变成 i=2 的情况。

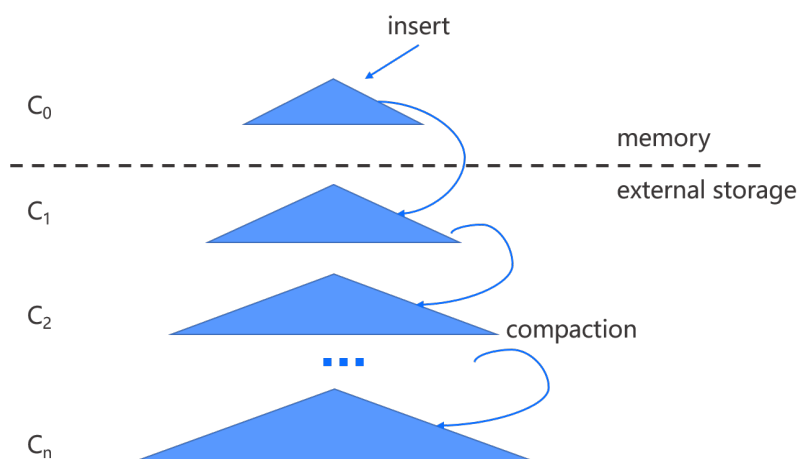
当我们再向其中分别插入散列值为 0000 和 0111 的记录，这两个记录都属于第一个存储块，于是该块会溢出，此时块内的数字 1 小于 i，因此我们不需要调整桶数组，只需要分裂该块，分别将记录调整到对应的存储块，并将桶数组中的 01 项指向新块。

最后我们再插入一个散列值为 1000 的记录，以相同的逻辑，散列表将扩展成如上图中所示的样子。

可扩展散列表有一个明显的好处是查找一个记录时总是只需要查找一个数据块，并且桶数组通常可以一直存留在主存。

3.3 LSM-Tree

LSM-Tree 的全称为日志结构合并树（Log-Structured Merge Tree），核心思想是将内存中的缓存数据以记录日志的形式追加写入到存储介质，初衷是为了将小粒度的随机写聚合成大粒度的顺序追加写，从而减少机械磁盘悬臂的频繁机械运动，提升 I/O 效率。

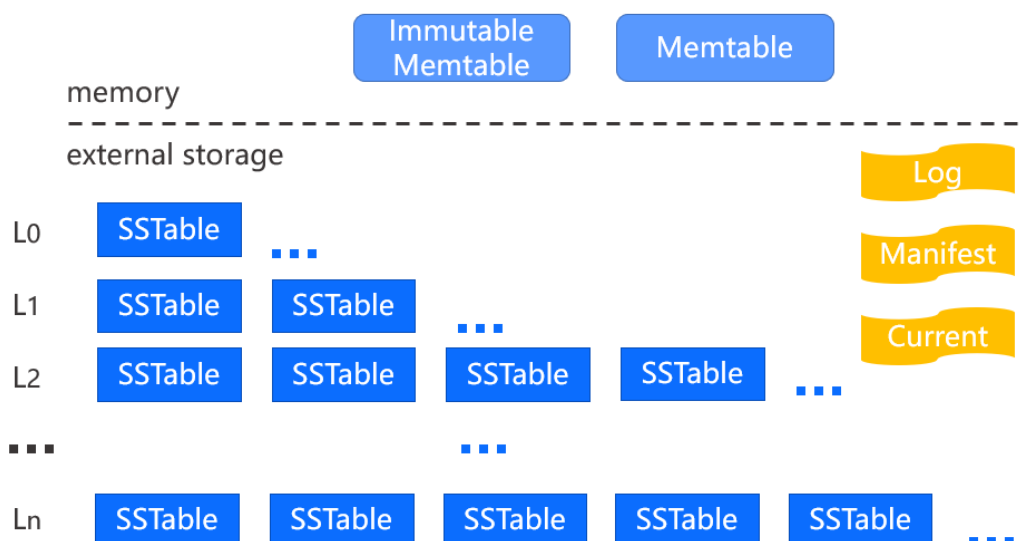


LSM-Tree 的基本结构如上图，数据被分成了多层，其中 C_0 存储在内存，其他层存储在外存，数据在每一层按照 Key 的大小依次排列，并且这些层的容量依次成倍增加。

当键值对写入 LSM-Tree 时，首先会被插入到位于内存中的 C_0 ，当 C_0 满了以后， C_0 中的数据会和 C_1 中的数据进行合并，合并后的结果会重新写入 C_1 。同理，当 C_n 满了时， C_n 也会和 C_{n+1} 合并，合并后的结果写入 C_{n+1} 。这些合并操作被称为 Compaction 操作。

而在查找指定键时我们需要从新向旧，即从 C_0 开始逐层向下进行查找，可能需要遍历多层才能找到。

这些是 LSM-Tree 的基本概念，在实际应用时需要考虑很多问题（如每一层以什么结构组织），为了让大家对 LSM-Tree 有一个更直观的理解，我们以业内常见的 LevelDB 的 LSM-Tree 实现为例进行详细介绍。



如图, LevelDB 主要由两部分组成, 一部分是驻留在内存的 Memtable (内存表) 和 Immutable Memtable (只读内存表), 对应前面讲的 C0。

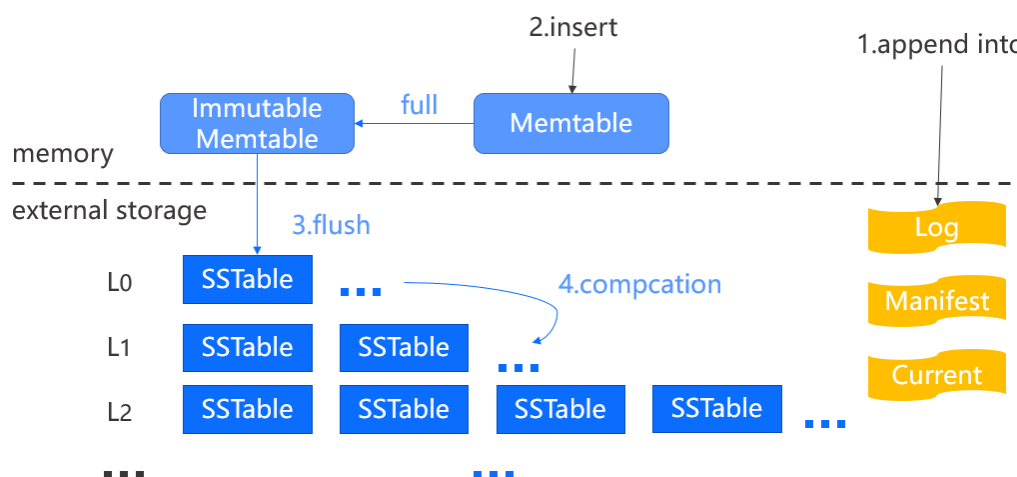
另一部分存储在外存并且分成了多层, 从 L0 开始分别对应前面讲的 C1~Cn, 其中每层含有许多的 SSTable (Sorted String Table, 顺序字符串表) 文件, 而 Key-value 数据就被封装在这些 SSTable 文件里。除了 L0 层, 其他层存放的 SSTable 都是按 Key 的顺序排列的, 也就是说这些层中 SSTable 文件的 Key 范围是没有重叠的。

除了数据外, LevelDB 还有 Log、Mainfest 以及 Current 文件:

- Log 文件存储最近的数据更新, 采用追加的方式将每次的更新数据写到日志文件的末尾, 每个日志文件对应当前的 Memtable, 更新操作先写入日志文件然后更新内存表。当内存表被写入 SSTable 文件后, 相应的日志文件会被删除。Log 文件也可做故障恢复。
- Mainfest 文件存储当前数据库元数据, 如每层包含哪些 SSTable 文件、每个文件中 key 的范围以及其他的元数据信息, 如日志文件等。
- Current 文件是一个简单的文本文件, 记录当前使用的 Mainfest 文件名, 通常通过判断当前文件是否存在来判断数据库是否已经创建。

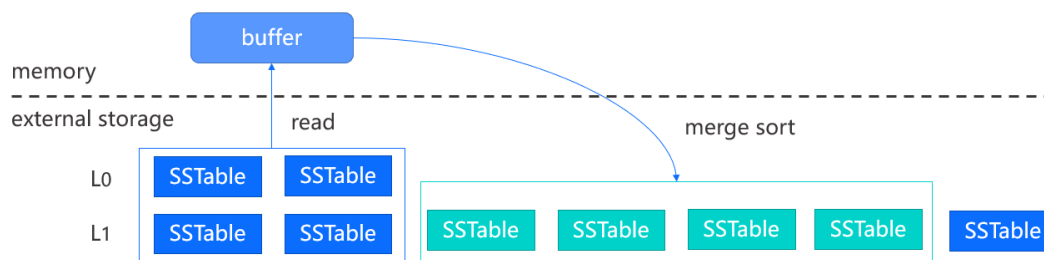
LevelDB 的数据写入流程可以理解两步:

1. 追加一条日志记录到 Log 文件。
2. 写入 Memtable, 其中 Memtable 被实现为跳表, 跳表的功能和平衡树类似, 但实现更为简单, 并且维持平衡的操作更轻量。



当 Memtable 容量达到上限时, 它会转变为一个 Immutable Memtable, 只允许读而不允许写, 并创建一个新的 Memtable 提供写入。Immutable Memtable 未来将转换成 SSTable 文件并 flush 到 L0, 为了更快速的下刷, L0 的 SSTable 是直接生成的, 因此文件之间是可能重叠的。

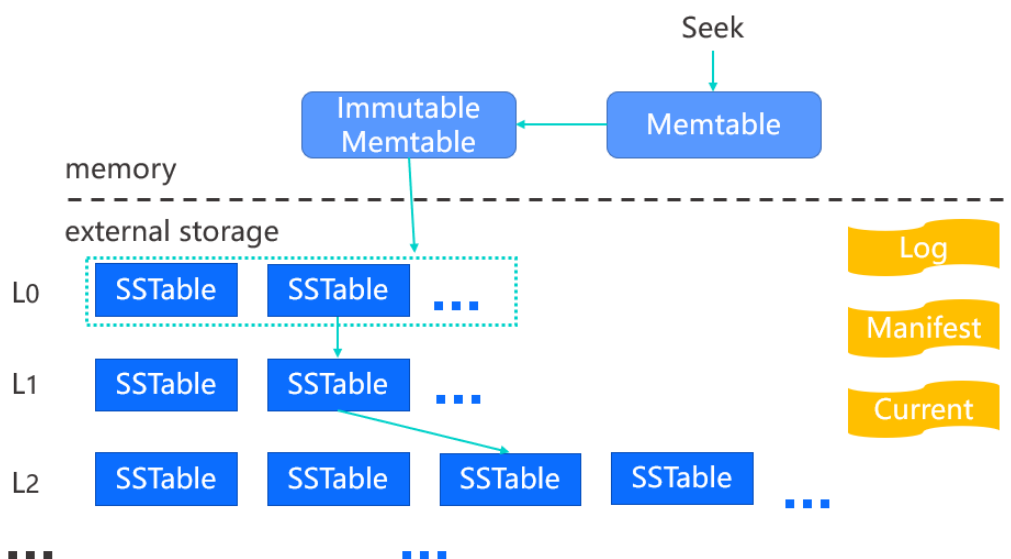
而随着 Memtable 不断的被转化成 SSTable 文件并写入 L0 后, L0 的文件数量会超过容量限制, 进而触发 Compaction 操作。



如上图所示，由于 L0 层的文件存在范围重叠，我们将从 L0 层读取所有文件，并在 L0 的下一层，即 L1 层找到 Key 范围与 L0 所读取文件有重叠的所有文件，然后将这些文件中的数据在内存进行合并排序后，重新生成新的 SSTable 文件写入 L1 层。由于可能有重复的新旧数据，合并过程中会删除旧数据，因此合并后一般能够减少文件的总量。其他层的合并也是类似的，只是每次只从当前层选取一个 SSTable 文件与下层进行合并。

Compaction 操作是 LSM-Tree 内部数据整合的机制，也是其中最复杂的过程之一，具体实现起来有很多细节与优化方法。

对于查询来说，首先是依次在 Memtable 和 Immutable Memtable 中查询，如果都没有找到，则在外存中继续查找。在外存中查找时，我们会从 manifest 文件中读取各层中 SSTable 文件的 Key 范围，然后找出可能包含查询 Key 的 SSTable 文件，这个信息一般会被缓存在内存中。



除了 L0 层外，其他各层最多只有一个 SSTable 文件可能包含查询键。随后我们将这些文件按所在层排序，位于 L0 层的在前，L0 层内的文件按时间先后逆序排列，然后按照顺序依次在 SSTable 文件内查找。

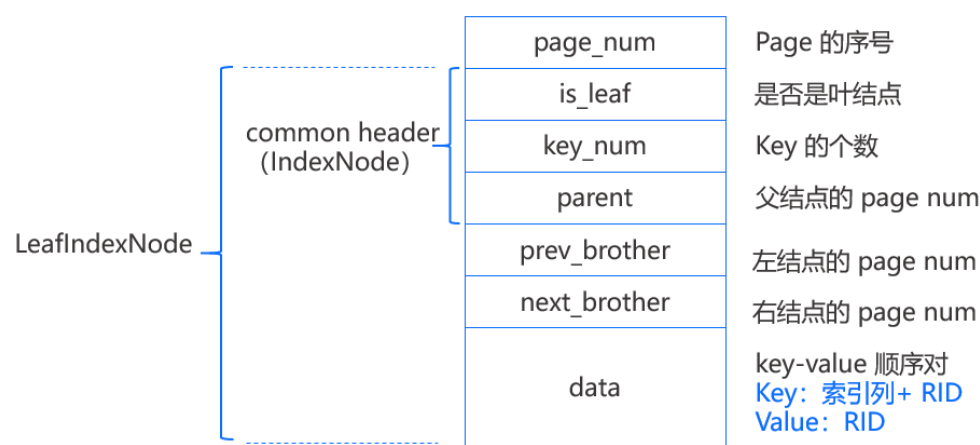
SSTable 文件内的数据是排序的，并被切割成多个数据块，由内部的索引信息记录着各个数据块中 Key 的范围。因此查询一般首先通过索引信息确定可能包含 Key 的数据块，然后在数据块内查找。

LevelDB 在每个 SSTable 文件中为每个数据块建立了一个布隆过滤器，在数据块内的查找前可以通过布隆过滤器来提前检验，如果确定不存在，则继续到下一个 SSTable 文件进行查找。

3.4 MiniOB B+Tree 实现

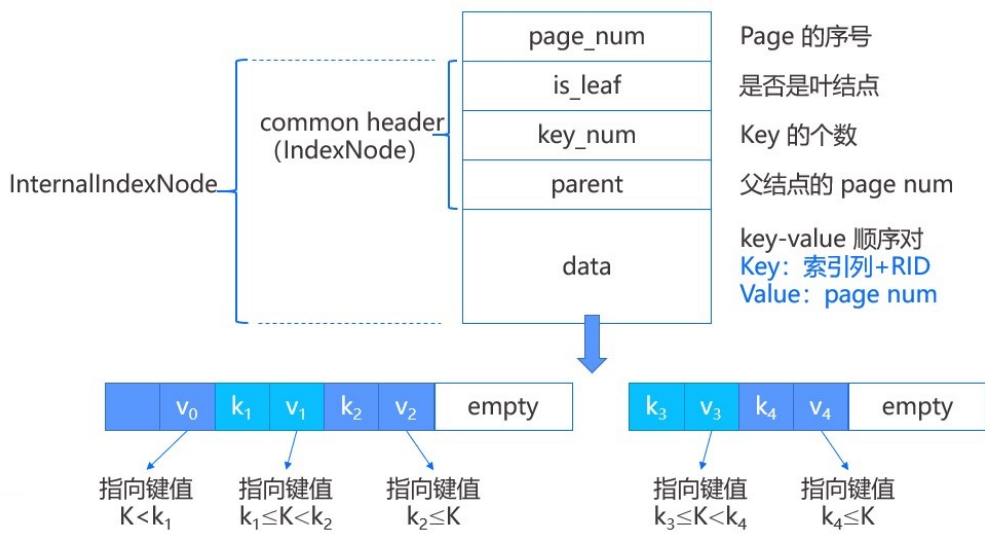
简介

在基本的逻辑上，MiniOB 的 B+Tree 和 B+Tree 是一致的，查询和插入都是从根逐层定位到叶结点，然后在叶结点内获取或者插入。如果插入过程发生叶结点满的情况，同样会进行分裂，并向上递归这一过程。

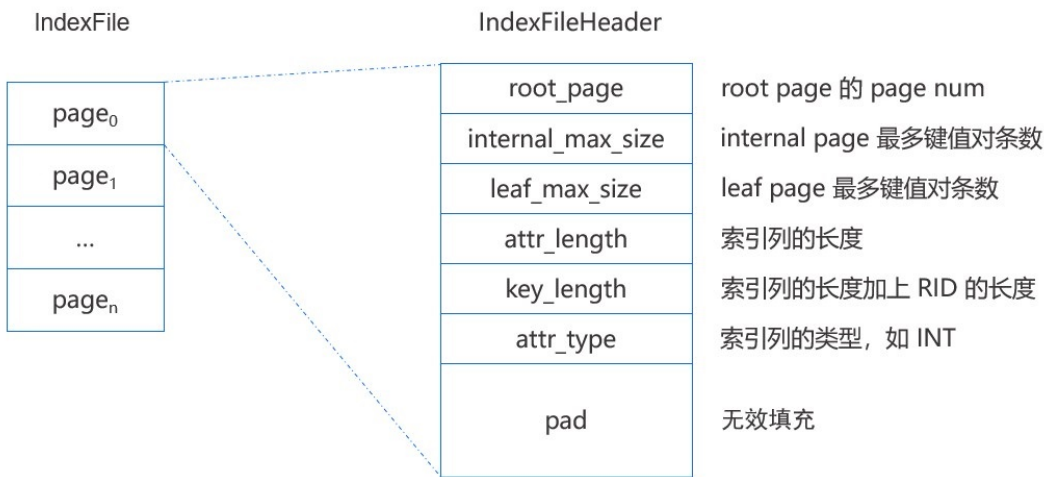


如上图，每个结点组织成一个固定大小的 page，之前介绍过每个 page 首先有一个 `page_num` 表示 page 在文件中的序号，每个结点 page 都有一个 `common header` 实现为 `IndexNode` 结构，其中包括 `is_leaf`（是否为叶结点）、`key_num`（结点中 key 的个数）、`parent`（结点父结点的 page num），当 `parent=-1` 时表示该结点没有父结点。

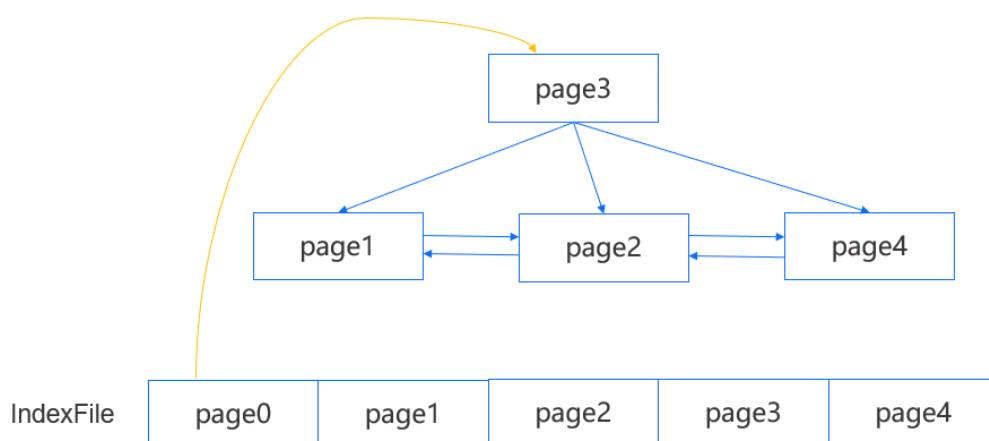
除此之外，Leaf page 还有 `prev_brother`（左结点的 page num）和 `next_brother`（右结点的 page num），这两项用于帮助遍历。最后 page 所剩下的空间就顺序存放键值对，叶结点所存放的 key 是索引列的值加上 RID（该行数据在磁盘上的位置），Value 则为 RID，也就是说键值数据都是存放在叶结点上的，和 B+Tree 中叶结点的值是指向记录的指针不同。



内部结点和叶结点有两点不同，一个是没有左右结点的 `page num`；另一个是所存放的值是 `page num`，也就是标识了子结点的 `page` 位置。如上图所示，键值对在内部结点是这么表示的，第一个键值对中的键是一个无效数据，真正用于比较的只有 `k1` 和 `k2`。



所有的结点（即 `page`）都存储在外存的索引文件 `IndexFile` 中，其中文件的第一个 `page` 是索引文件头，存储了一些元数据，如 `root page` 的 `page num`，内部结点和叶子结点能够存储键值对的最大个数等。

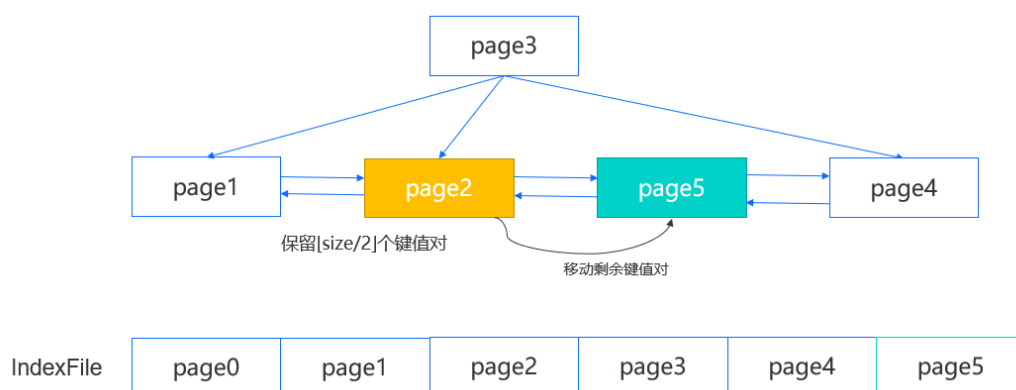


上图是一个简单的 MiniOB B+Tree 示例，其中叶结点能够访问到左右结点，并且每个结点能够访问到父结点。我们能够从 IndexFile 的第一个 page 得到 root page，而在知道一棵 B+Tree 的 root page 以后就足够访问到任意一个结点了。查询时我们会从 root page 开始逐层向下定位到目标叶结点，在每个 page 内遍历搜索查找键。

插入

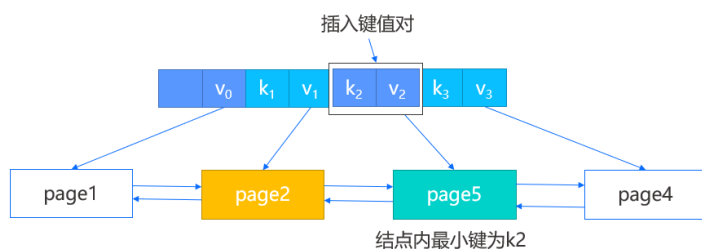
在插入时，我们首先定位到叶结点，如下图中的 page2，然后在结点内定位一个插入位置，如果结点未满，那么将键值对插入指定位置并向后移动部分数据即可；如果结点已满，那么需要对其进行分裂。

我们将先创建一个新的右兄弟结点，即 page5，然后在原结点内保留前一半的键值对，剩余的键值对则移动到新结点，并修改 page2 的后向 page num，page5 的前后向 page num 以及 page4 的前向 page num，再根据之前定位的插入位置判断是插入 page2 还是 page5，完成叶结点的插入。

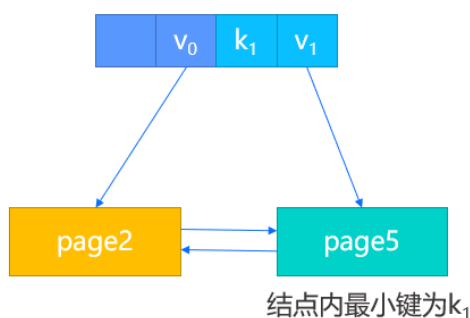


此外，由于我们新增了结点，我们需要在父结点也插入新的键值对，这一步将涉及到原结点，新结点以及新结点中的最小键，分为两种情况：

1. 有父结点，那么直接将新结点中的最小键以及新结点的 page num 作为键值对插入父结点即可。



2. 假设此时没有父结点，那么我们将创建一个新的根结点，除了把新结点键值对插入，还会将原结点的 page num 作为第一个键值对的值进行插入。

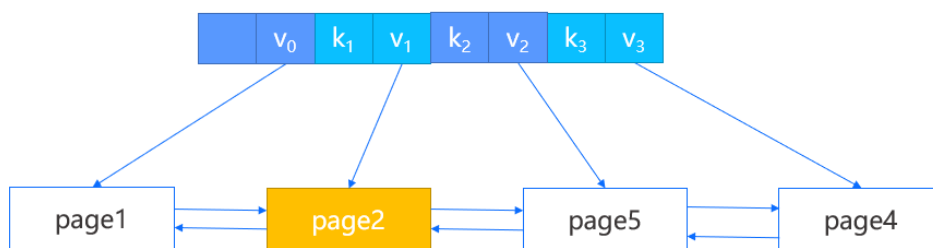


如果父结点的键值对插入同样触发了分裂，我们将按上述的步骤递归执行。

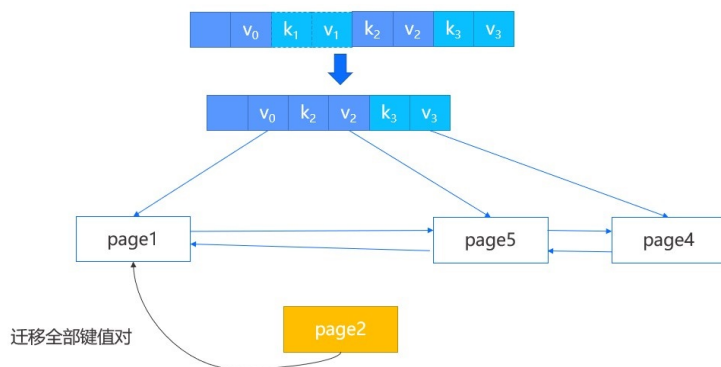
删除

正常的删除操作我们就不再介绍，这里介绍一些涉及结点合并的特殊情况。

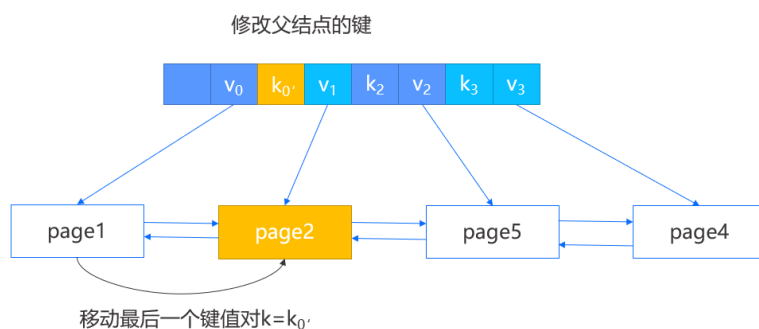
首先在结点内删除键值对，然后判断其中的键值对数目是否小于一半，如果是则需要进行特殊处理。比如 page2 中删除一个键值对，导致其键值对数目小于一半，此时通过它的父结点找到该结点的左兄弟，如果是最左边的结点，则找到其右兄弟。



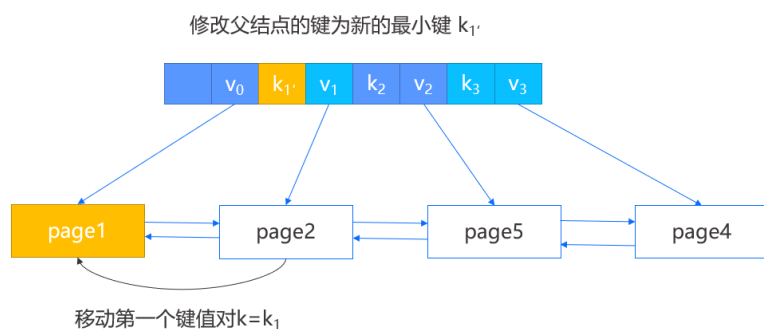
- 如果两个结点的所有键值对能容纳在一个结点内，那么进行合并操作，将右结点的数据迁移到左结点，并删除父结点中指向右结点的键值对。



- 如果两个结点的所有键值对不能容纳在一个结点内，那么进行重构操作。
- 当所删除键值对的结点不是第一个结点时，那么选择将左兄弟的最后一个键值对移动到当前结点，并修改父结点中指向当前结点的键。



- 当所删除键值对的结点是第一个结点时，那么选择将右兄弟的第一个键值对移动到当前结点，并修改父结点中指向右兄弟的键。



在上述两种操作中，合并操作会导致父结点删除键值对，因此会向上递归地去判断是否需要再次的合并与重构。相比第二节介绍的 B+ 树，MiniOB B+ 树的实现方式更简单，简化了分裂和合并时所需要考虑到的一些处理状况。

3.5 课后实践

MiniOB 题目：实现 select-meta 功能

【要求】：对 SQL 语句合法性的验证，比如 SQL 语句中查询不存在的表、字段等，需要报错。

注意

这个题目是其它题目的基础，在做其它题目时也可能会破坏这个题目的正确性，所以考虑的场景需要细节一些。

【训练营测试的原理】：后台执行时，有一系列的 SQL 语句，发送到 MiniOB，然后将执行输出的结果，跟预期的 result 文件做对比。

【练习方式】：完成后在 [训练营](#) 上提交测试，训练营使用文档请参见 [训练营使用说明](#)。

第 4 章：SQL 引擎

本章前 5 节主要介绍数据库 SQL 引擎基础，这部分内容大多数是数据库通用的知识，但也会结合 OceanBase 的实现进行讲解，来帮助大家理解。希望通过本章内容能让之前没有接触过 SQL 内核相关知识的同学对 SQL 引擎有一个整体的认识。

后几节的内容主要是在前几节的基础上，对 SQL 的优化与执行进一步拓展。

本章目录

4.1 SQL 层整体架构

4.2 Parser 和 Resolver 模块

4.3 Transformer 和 optimizer 模块

4.4 Executor 模块

4.5 Fast-parser 和 Plan cache 模块

4.6 基础代数符号与操作介绍

4.7 关系表达式的等价规则转换

4.8 执行计划的选择

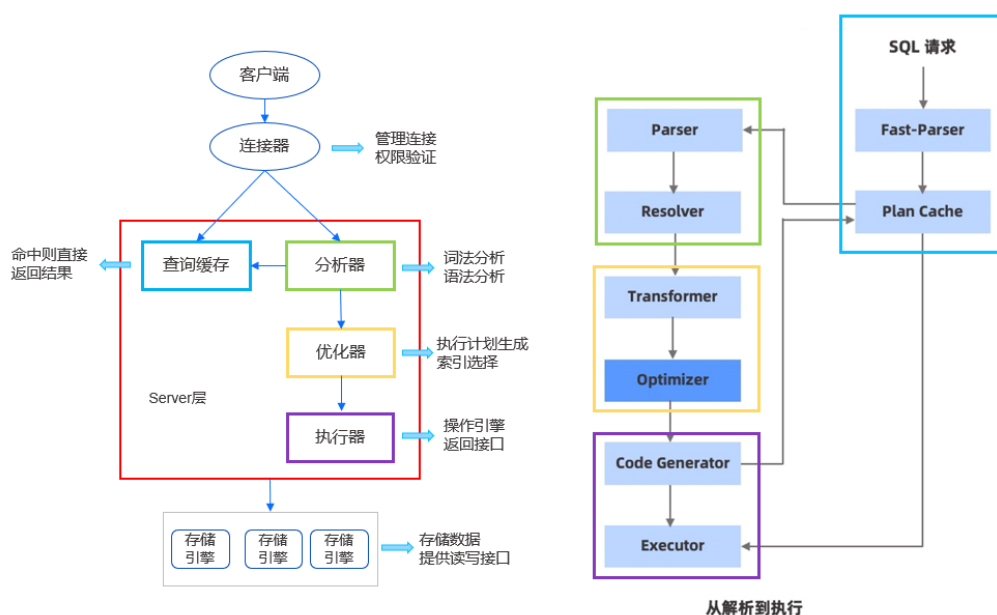
4.9 代价

4.10 课后实践

4.1 SQL 层整体架构

OceanBase 和 MySQL 架构介绍

数据库领域经过长期的发展已经形成了比较稳定的工程结构，本节主要通过介绍 MySQL 和 OceanBase 的 SQL 层架构来帮助大家理解整个 SQL 内核层是如何组织的。如图所示左边借鉴《MySQL 45 讲》中关于 MySQL 的基本架构图，其中红框标注出来的是 SQL 层部分，右边是 OceanBase 处理一条 SQL 请求的架构图。



连接器部分本节不展开讲解，可以直接通过 MySQL 命令连接，也可以使用各种数据库连接工具（如 JDBC），连接器 OceanBase 高度兼容 MySQL 生态。

从图中可以看出 MySQL 在 SQL 层的处理模块分别是查询缓存、分析器、优化器和执行器。在 OceanBase 数据库的架构解析图中列出了较为详细的处理模块，并在图中用不同的颜色框，与 MySQL 的各模块一一对应。在接下来几节中会按照右边图示，讲解一条 SQL 被发送给 Server 后从解析到执行的全过程。

SQL 语句结构

无论是经常使用数据库做查询，还是专门做数据库内核的同学，对一条 SQL 语句不同结构的执行顺序应该都有所了解。

```
SELECT <字段名> (9)
FROM <表名> (1)
JOIN <表名> (2)
ON <连接条件> (2)
WHERE <筛选条件> (3)
GROUP BY <字段名> (4)
HAVING <筛选条件> (5)
```

```
ORDER BY <字段名>(7)
LIMIT <限制行数>(8);

# 此处不包含 subquery
```

如上所示 SQL，其中标注出了执行序号，可以看到一条 SQL 语句的书写顺序与它的执行顺序通常都是不一样的，可能有些数据库的特定实现不一定按照这个顺序，但总体上都是比较类似的。感兴趣的同学可以查阅相关资料，了解为什么这样一条 SQL 的执行顺序是像示例中标注的这样，而不是其他顺序。也可以尝试打乱顺序，分析这样会对 SQL 执行造成什么样的影响。

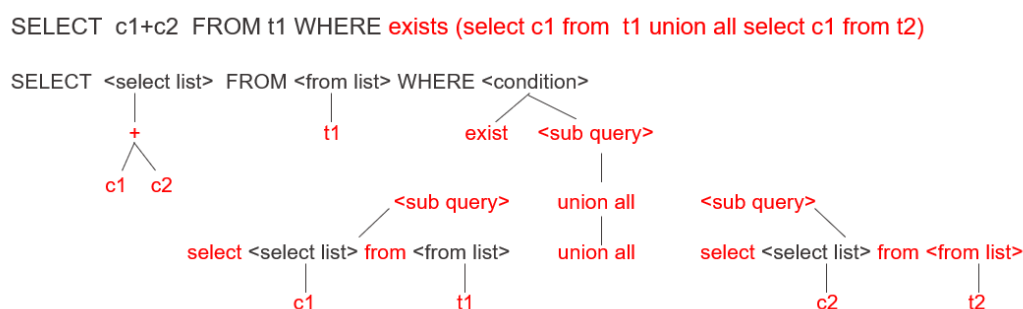
Server 端是怎么一步步的去解析这条 SQL 的呢？实际上，SQL 是为了方便让使用者去向数据库 Server 端表达信息的一种描述性的语言，具体 Server 端怎么样处理，是在数据库内核层面去实现的。

4.2 Parser 和 Resolver 模块

Parser 模块

当一条 SQL 请求到达 Server 端后，首先经过的是 Parser 模块。Parser 模块主要作用是根据定义的语法规则判断一条 SQL 语句的格式是否符合对应的语法结构，这个语法结构无论是在 MySQL 还是 OceanBase 中，都是使用 Yacc 和 Flex 工具自动生成的。需要重点理解 Parser 模块是如何将一条用户写的 SQL 请求转换成可以在数据库内核中所使用的数据结构。

如下图所示，SQL 语句 `SELECT c1+c2 FROM t1 WHERE exists (select c1 from t1 union all select c1 from t2)` 再接着一个子查询，然后子查询是由 union all 连接的另外两个子查询。Parser 模块最终解析出来的是一个逻辑结构图，就像示例图中这样一个树状结构。



接下来介绍一下具体是如何实现的，无论是 SELECT、FROM 还是 WHERE，都会有自己所附带的一些成分。

- select list 是一个表达式操作，即 `c1+c2`。再递归解析，它的根节点是 `+`，两个子节点分别是参与运算的两个常量 `c1` 和 `c2`。
- from list 的参数其实是一个 relation，也就是一个表名，即 `t1`。
- where 代表的是 condition 条件，这里的 condition 操作符是 `exists`，`exists` 后面是一个 sub query。这个 sub query 可以递归地向下去分解，它是由 union all 所连接起来的两个 sub query。同理这两个 sub query 也可以再递归地向下展开。

一条 SQL 语句，无论写得多么复杂，本质上就是一层一层嵌套的树状结构。Parser 模块这一个部分主要做的就是根据用户所写的语法规则正确的识别，并将一条 SQL 语句转化为一个语法树的结构。但一条符合语法树规则的 SQL 不一定是一条“合格”的 SQL，数据库中还存在着许多约束，一条 SQL 还必须满足语义的约束，许多语法的约束以及虚视图的展开都是在 Resolver 模块中完成的。

Resolver 模块

Resolver 模块主要作用是对 Parser 模块所生成的符合语法规则的树状结构进行进一步的约束检查，还可能会提取表达式的属性。为便于大家理解接下来介绍几个示例：

- 关系或属性的检查

关系或属性的检查主要是看表名、列名和别名是否有歧义、是否合法。比如当查询某张表中的一些列属性，FROM 子句出现的关系必须存在。也就是说当一张表都不存在，那查询肯定是没有意义的，或者当查询的属性没有出现在所查询的表中，当然也不满足数据库约束的要求。

- 类型的检查

- LIKE 操作符：要求匹配的属性必须是一个字符串，或可以转化成一个字符串结构。
- PARTITION BY 操作符：该操作符后不能跟正则操作。当创建了一张表，PARTITION BY HASH 带的是一个 RLIKE 的正则操作符，这张表创建成功后查询时却报错了。

```
OceanBase(admin@test)>CREATE TABLE v0 ( v2 INTEGER , v1 INTEGER (
  0 ) ) PARTITION BY HASH ( v2 RLIKE "0x7fffffff" "0xffffffff" ) P
ARTITIONS 86;
Query OK, 0 rows affected (0.16 sec)

OceanBase(admin@test)>SELECT * INTO DUMPFIL "66" FROM v0 WHERE v
2 = v2 ORDER BY - 69 RLIKE 8 , v2;
ERROR 4016 (HY000): Internal error
OceanBase(admin@test)>
```

4016 是 OceanBase 中处理错误的一种表示。这张表本就不应该创建成功，虽然这个语法符合 Parser 的语法要求，但实际不符合数据库中的语义要求，也就是说 PARTITION BY HASH 后不应该带 RLIKE 的正则操作符。

- 类型推演

类型推演主要是提取表达式的属性用于后续分析，比如共享一些表达式，或者优化一些表达式计算之类的，对之后查询计划的优化有很大的作用。

- 虚视图的展开

在 OceanBase 和 MySQL 中都支持语法 `SHOW FULL COLUMNS FROM t1`（查看表的属性）。实际上在 OceanBase 中是转化成查询内部租户下的一张虚拟表。table_id 对应的是 t1，在 OceanBase 内部表名会被转换成一个唯一的 table_id，这些都是在 Resolver 模块完成的。

```
OceanBase(root@oceanbase)>SHOW FULL COLUMNS FROM t1;
+-----+-----+-----+-----+-----+-----+-----+-----+
--+-----+
| Field | Type   | Collation | Null | Key | Default | Extra | Privilege
s | Comment |
+-----+-----+-----+-----+-----+-----+-----+-----+
--+-----+
| a     | int(11) | NULL      | YES  |     | NULL    |      | 
|      |         |           |      |     |         |      | 
+-----+-----+-----+-----+-----+-----+-----+-----+
--+-----+
1 row in set (0.00 sec)

OceanBase(root@oceanbase)>select * FROM oceanbase.__tenant_virtual_table
_column where table_id = 1100611139453777;
```

除此之外，Resolver 模块虽然是将 Parser 模块的语法树结构进行一个检查转换，但在这一层会转化成一个自己的数据结构。不同的数据库可能有不同的实现，在 OceanBase 中用的是 STMT（Statement）。

如下图所示，这里将 SQL 语句 `SELECT c1+c2 FROM t1 WHERE exists (select c1 from t1 union all select c1 from t2)` 转换成了 Statement Tree，大家可以观察一下，在 OceanBase 中 Resolver 这层是怎么去表示 Statement 结构，这个结构是多个 OceanBase Statement 和表达式对象的集合，多个 Statement 对象在 OceanBase 中可以表达一些查询中的子查询。



理论上可以通过 Statement 结构去还原原始的 SQL，每个数据库都有不同的做法，它既不是一个关系运算符的表示，也不是一个树状结构，而是一个实际的工程实现，在逻辑上与树状结构相似，大家也可以继续把它当成一种树状的层次结构来理解。

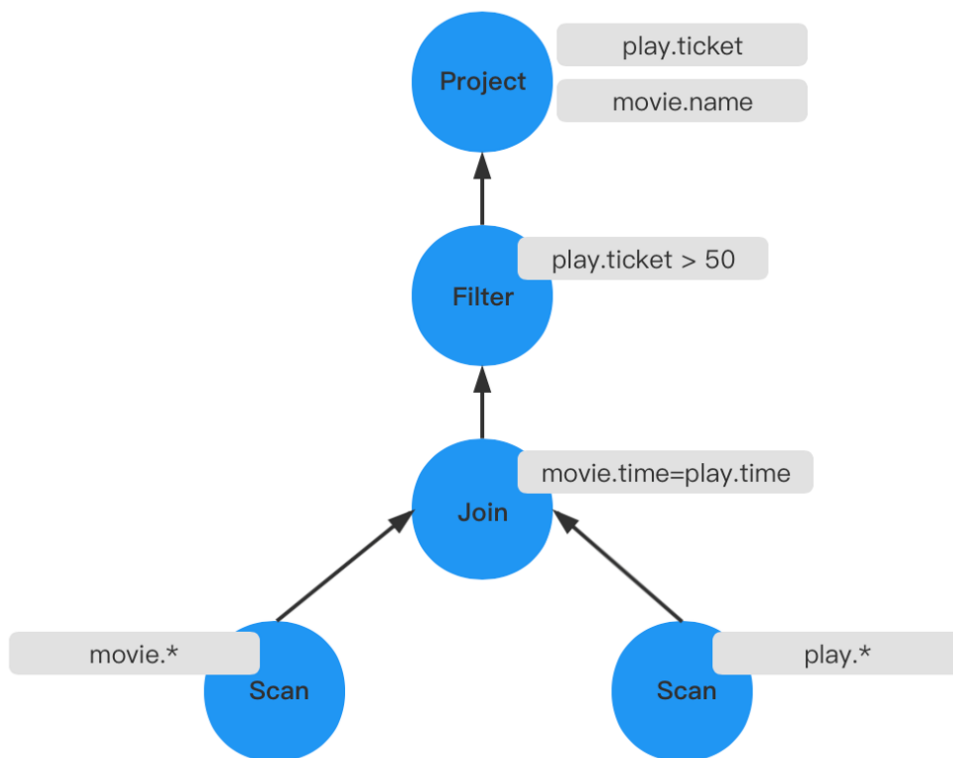
4.3 Transformer 和 optimizer 模块

通过前几节的介绍，了解了一条 SQL 请求在 Parser 模块主要关注的是语法树逻辑结构，在 Resolver 模块主要关注的是符合语义要求的树形的结构。在 OceanBase 中，数据结构在 Resolver 层实际上是一个 Statement，这也是 OceanBase 后续进行 Transformer 和 optimizer 改写的基础。

不过在 Transformer 和 optimizer 层，重点需要关注的是火山模型的结构，火山模型详细介绍请参见 [4.4 Executor 模块](#)。这里介绍一个示例来帮助大家理解，示例中 SQL 的语义是：挑选出某场电影的名字和卖出的票数，并且它的票数需要大于 50 张。这里假设在七点钟，只有一场电影在播放，并且卖出的票数超过了 100 张。即当执行这条 SQL 的时候，会查出这场电影和卖出的票数。

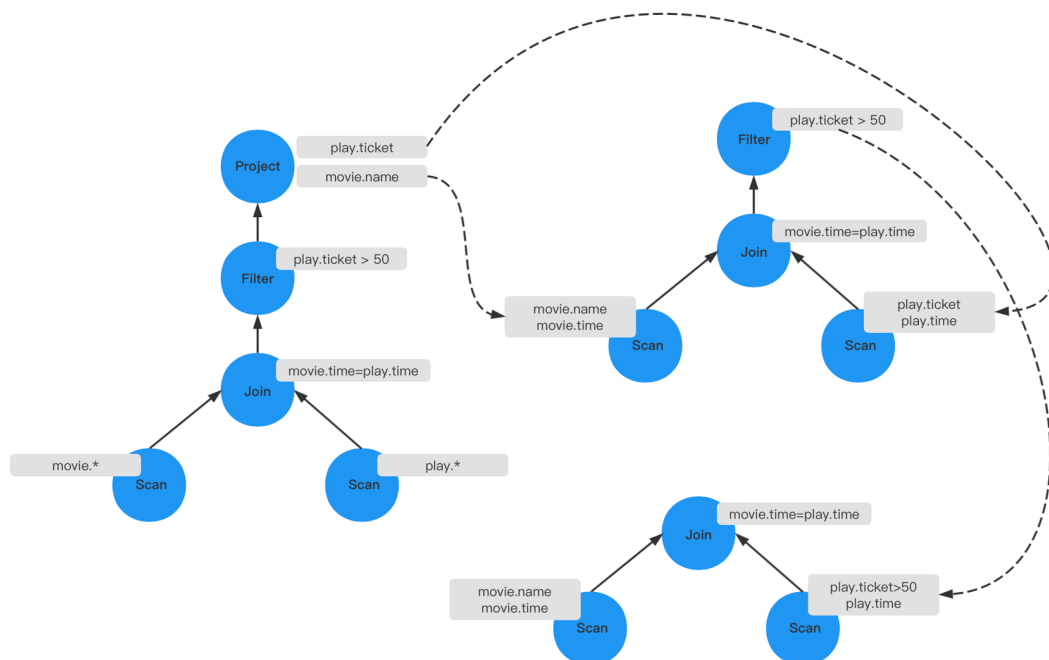
```
SELECT movie.name, play.ticket
FROM movie
JOIN play
ON movie.time = play.time
WHERE play.ticket > 50;
```

下图所示是这条 SQL 简单的执行计划。首先分别对两张表进行全部字段的遍历，遍历所有字段后做表连接操作，操作的条件为 `movie.time=play.time`，即时间一致。连接之后生成一张新表，再去根据条件 `play.ticker > 50` 去过滤，最后将需要查询的 `play.ticker` 和 `movie.name` 给投影出来。



接下来看一下它是如何优化的。

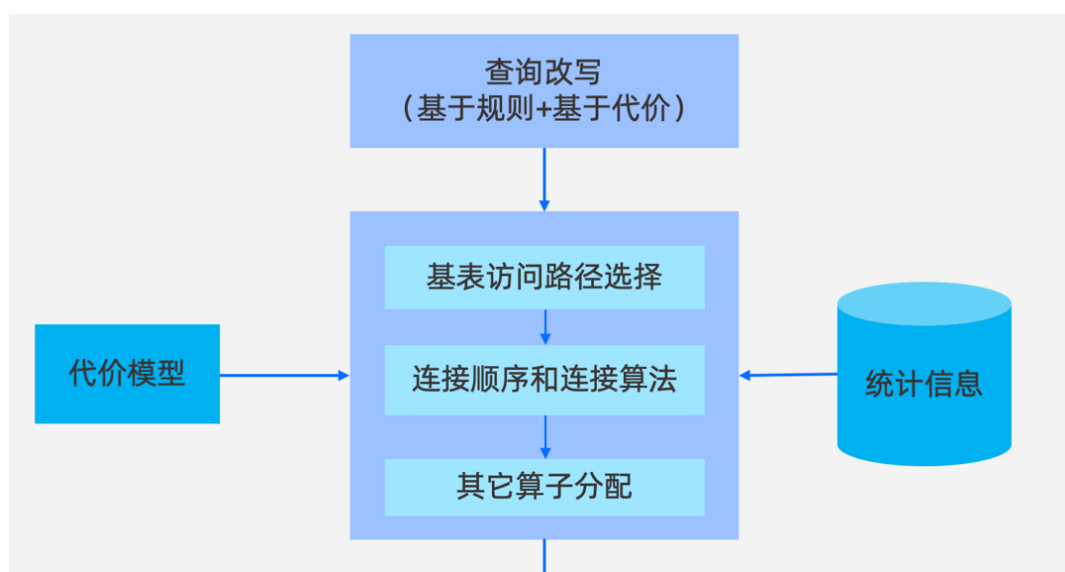
投影操作实际上只需要投影出特定的列，包括在做 Join 条件时也是需要需要一个时间的字段，所以第一个优化是减少遍历属性，可以将上层的投影给去掉，然后将 Scan 的操作只遍历特定的属性，两张表都是这样，做完这个优化后，就减少了一个投影操作。



第二个优化是因为最终符合条件的只需要卖出的票数大于 50 张的场次，所以在遍历 play 表的时候，不需要将所有的电影及对应票数都筛选出来，只需要筛选出票数大于 50 张的排片。

经过优化之后它由四层的结构变成了两层，只需遍历出需要的字段，然后做一个 Join 操作就可以了。不仅操作路径变短了，而且表的连接项也变得更少了，相比之前查询计划更优。

大部分成熟的数据库都会实现相关的计划优化模块，如下图所示，在 OceanBase 的数据库中，针对于查询计划也实现了一个比较完整的的优化模型，OceanBase 的优化模型主要包括查询改写、代价模型和统计信息三个模块。各模块的详细内容请参见[开发者进阶教程](#)。



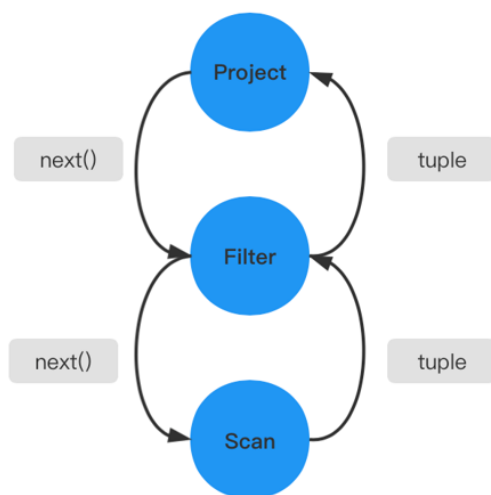
关于这两个模块最主要的内容就是前面讲的大家要懂得如何从一个火山模型的层次调用结构，去看是否有更多的优化空间。

4.4 Executor 模块

Executor 模块中最经典的模型是 Volcano Model（火山模型），它是一种基于行的流式迭代模型（Row-Based Streaming Iterator Model），目前主流的关系数据库中都采用了这种模型，例如 Oracle、SQL Server、MySQL 等。

在火山模型中，所有的代数运算符（operator）都被看成是一个迭代器，它们都提供一组简单的接口：`open()`—`next()`—`close()`，查询计划树由一个个这样的关系运算符组成，每一次的 `next()` 调用，运算符就返回一行

（Row），每一个运算符的 `next()` 都有自己的流控逻辑，数据通过运算符自上而下的 `next()` 嵌套调用而被动的进行拉取。



火山模型中每一个运算符都将下层的输入看成是一张表，`next()` 接口的一次调用就获取表中的一行数据，这样设计的优点是：每个运算符之间的代数计算是相互独立的，并且运算符可以伴随查询关系的变化出现在查询计划树的任意位置，这使得运算符的算法实现变得简单并且富有拓展性。

这种设计也会存在一些问题，如果没有一些阻塞性的操作，整个火山模型只需要很少的内存就能运作起来，每次只需要迭代一个 tuple 行。但有一些 Operator（如 `sort`），它是阻塞性的操作，要拿到所有的 tuple 之后，才能进行运算，这种操作符实际上是破坏了火山模型整体的流水性的运算。

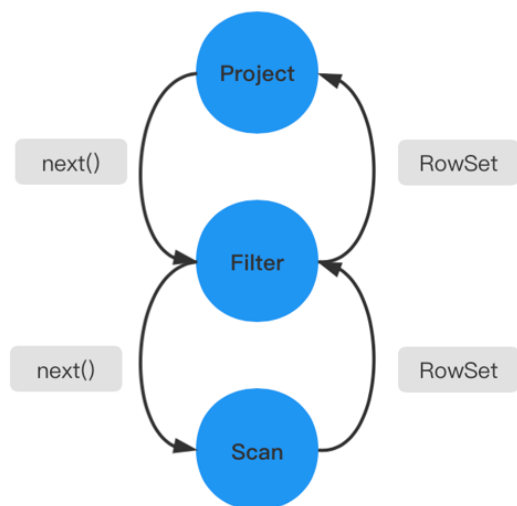
另外一方面当处理的数据量增大时，也会有明显缺陷。因为每次只迭代一行，所以要向下调用很多层的 `next()` 函数，它导致了大量的虚函数开销，非常消耗 CPU 资源。

这个问题也是因为一些历史原因，火山模型最早于 1990 年 Goetz Graefe 在 *Volcano, an Extensible and Parallel Query Evaluation System* 这篇论文中提出的，在 90 年代早期，计算机的内存资源十分昂贵，相对于 CPU 的执行效率，IO 的效率要差得多。因此当时火山模型考虑的是将更多的内存资源用于 IO 的缓存设计，而没有考虑去优化这种结构导致的 CPU 方面执行效率的低下。从当时的条件来看，在硬件上它是一个非常合理的权衡。

经过多年的研究和发展，无论学术还是工业界都在火山模型的基础上不断改进，下面介绍几个示例：

- RowSet 迭代

如下图所示，每次返回的不是一个 tuple，而是一个 RowSet。这种 RowSet 的迭代，它每一次的数据流传递不再是单行的模式，而是做单个 tuple 组成的集合。更多的运算就会停留在这个 next() 操作的内部，而不是在函数调用之间频繁的切换，可以减少 next() 的调用次数。同时这个局部的操作也有返回一个 tuple，变成了一个局部的循环操作。这种循环操作实际上是可以现代的一些编译技术或 CPU 动态指定预测技术来进行优化的，也能够提升我们的查询效率。



- 操作符融合

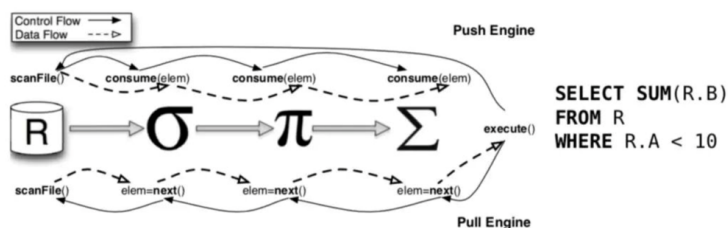
如下图所示，在做 TABLE SCAN 的时候，实际上做了一个过滤，过滤掉了 `ss_item_sk=1000` 的操作。可以看到无论是第几步都有 Filter 操作，而实际上是把 Filter 的操作放到了投影和 Scan 里面，这样可以减少一层的调用。比如说这里的 Filter 下推到 Scan 里面去过滤掉，这在前面介绍的查询计划的优化中也有类似的操作。

```
|=====|
|ID|OPERATOR      |NAME      |EST. ROWS|COST|
|-----|-----|
|0 |SCALAR GROUP BY|          |1        |601 |
|1 |TABLE SCAN     |store_sales|2        |600 |
|=====|

Outputs & filters:
-----
0 - output([T_FUN_COUNT(*)]), filter(nil),
   group(nil), agg_func([T_FUN_COUNT(*)])
1 - output([store_sales.ss_item_sk]), filter([store_sales.ss_item_sk = 1000]),
   access([store_sales.ss_item_sk]), partitions(p0)
|
```

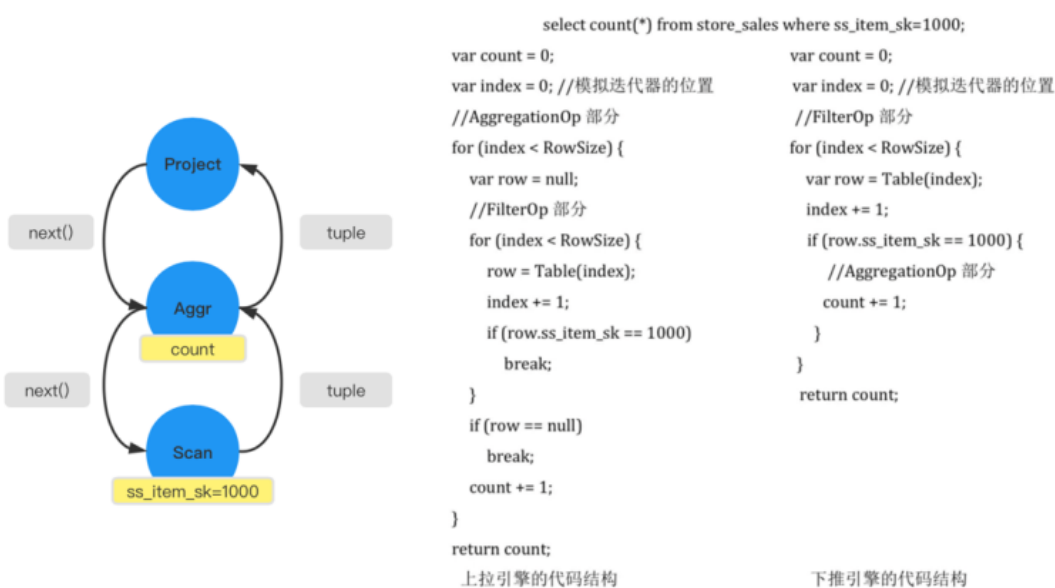
- 拉取模型和推送模型

火山模型是一种被动的数据读取方式，它由上层的算子来驱动，当下层算子接受到上层算子的调用请求后，会返回符合条件的 tuple。但更方便的方式，应该是由下层算子主动将数据推送给上层算子。



为帮助大家理解拉取模型和推送模型的差异，接下来介绍一个示例。

下图所示 SQL，语义是从 store_sales 中挑选符合条件的商品，并做聚合去计数。下图左边所示是对应的火山模型。挑选出符合条件的 tuple，然后返回给上层 Aggr 算子，再进行 count 操作去计数。



接下来分别来看一下上拉模型和下推模型的代码结构区别。

- 上拉引擎的代码结构

左边是经典的火山模型的一个伪代码，投影操作的代码这里没有列举出来。上拉模型是由上层的算子驱动，即从上层的 AggregationOp 去驱动，每次开始时，就会调用 FilterOp 的部分。FilterOp 遇到符合条件的时候，会结束自己的调用，然后将这条信息传回给上一层。当 row 不等于 null 时，就会将计数加一，这是外层 AggregationOp 的调用。

- 下推引擎的代码结构

推送模型的数据流动是从下层开始的一个 Scan 操作，也就是从它的 FilterOp 开始。即首先遍历他自己 Table，然后找到满足的行，推送给 AggregationOp 的部分，直接进行计数加一。可以看到下推引擎在代码的层次上相对上拉引擎简洁了许多，能够减少一些 CPU 资源的消耗。

4.5 Fast-parser 和 Plan cache 模块

这部分的主要作用是当接收到 SQL 请求时,如果能够直接查询到其对应的 Plan cash,那么就将这个查询计划返回给 Executor,避免经过长时间的 SQL 硬解析。

如果对一个 SQL 而言,执行时间远大于解析时间,那么做 Plan cash 模块的作用并不大。在 OceanBase 数据库面对的高并发场景里面,比如双十一期间,下单的流量非常的大,而操作时间很短,此时解析时间过长会造成很大的影响,因此需要避免 SQL 的硬解析。

OceanBase 提出了一种叫 Fast-parser 的方式,跟原来的 Parser 没有太大区别,主要是会对一个 SQL 语句进行常量的替换,方便去做一些 match。这里通过一个例子来帮助理解 Fast-parser 和 Plan cache 的作用。

首先来看什么是 Plan cash,如下图所示,当执行了两条 SQL 之后,查询 OceanBase 中的缓存视图,发现其中没有出现第二次执行的 SQL,只出现了第一次执行的 SQL,并且缓存命中数(hit_count)增加了 1 次。

出现上述情况是因为在进行第二条 SQL 查询的时候,进行了文本替换,将第二条 SQL 替换成了 `select * from t where c1=?` 这样的一条 Statement,而条 Statement 的查询计划已经缓存在数据库中了,所以会直接将查询计划交给 Executor 进行执行,记录中就不会生成第二条 SQL 对应的查询计划。

```
OceanBase(admin@test)>select * from t where c1=1;
Empty set (0.01 sec)

OceanBase(admin@test)>select * from t where c1=2;
Empty set (0.00 sec)

OceanBase(admin@test)>select query_sql,statement,hit_count from oceanbase.gv$plan_cache_plan_stat where statement like "%select * from t%";
+-----+-----+-----+
| query_sql | hit_count | statement |
+-----+-----+-----+
| select * from t where c1=1 | 1 | select * from t where c1=? |
| select query_sql,statement,hit_count from oceanbase.gv$plan_cache_plan_stat where statement like "%select * from t%" | 0 | select query_sql,statement,hit_count from oceanbase.gv$plan_cache_plan_stat where statement like "%select * from t%" |
+-----+-----+-----+
2 rows in set (0.04 sec)
```

为了避免使用原来的 Yacc/Bison 实际上是一个状态机的实现, OceanBase Plan 使用了 Fase-parser, 即匹配 Plan cache 时不需要将整个 SQL 去转化成一个语法树, 再进行一系列复杂的状态机转换解析, 而是只需要识别转换这个 SQL 语句中需要替换的部分, 这也是 OceanBase 的一个优化。

4.6 基础代数符号与操作介绍

基础关系代数符号与操作如下所示：

- 传统的集合运算
 - 并($\cup$)(union)
 - 差($-$)(except)
 - 交($\cap$)(intersect)
 - 笛卡尔积 ($\times$)
- 专门的关系运算
 - 选择(σ)(where)
 - 投影(π)(select)
 - 连接($\bowtie$)

除过传统的集合运算与专门的关系运算之外还有外连接、聚集、物化计算等，接下来会针对这些内容进行详细介绍。

选择操作符

使用选择操作符时的算法通常是 Scan，其中有线性扫描，也有索引扫描。通常对同一张表有多次选择的条件，应该将这多个条件优化到一次。

- 示例 1：合并多个选择，该方法适合于对同一张表上有多个相同的选择条件而言，此时可以合并多次的选择，以减少多次过滤带来的消耗。比如条件 `where T.a=T.b and T.b=T.c`，这个条件是一个和的操作，需要过滤两次。因此在内部可以将它优化成一个过滤条件，直接判断 `T.a=T.b=T.c`，而不是先判断 `T.a=T.b`，再判断 `T.b=T.c`。
- 示例 2：分解多个选择，比如 SQL 语句 `select * from T where T.a=3 OR T.b>8`，假设 T 表的 a 和 b 组上都有各自的索引。这时候可以将这个 SQL 分解为两个 SQL (`select * from T where T.a=3 UNION select * from T where T.b>8`)，各自利用索引加速进行查询，最后再合并结果，而不必因为 OR 导致索引失效。在实

际应用中 OR 改写为 UNION 是一种非常经典的改写，感兴趣的同学可以搜索相关资料进行研究。

投影操作符

使用投影操作符能够减少列，从而减少中间临时关系的内存消耗，这里通过一个例子来帮助理解。

$$\sigma_{(T1.a,T2.b)}(T1 \bowtie T2) \equiv \sigma_{(T1.a,T2.b)}(\pi_a(T1) \bowtie \pi_b(T2))$$

如上图所示，在这个示例中最终需要过滤出来的是表 T1 的 a 属性和表 T2 的 b 属性。假设表 T1 和表 T2 都是具有非常多属性的复杂表，在连接时做了全连接，导致产生的中间临时关系比较大。此时可以使用投影操作符提前进行列的投影，通过列投影操作生成的中间临时关系会比较小，并且连接操作的复杂度也会有所下降。

笛卡尔积和连接

笛卡尔积不要求两个属性之间有公共属性，导致最终生成的结果集也会比较复杂。因为它是对两个关系中的每一条元素都一一进行连接，如下图所示，r 属性有两条元组，s 属性有四条元组，最后生成了八条元组，也就是乘积的关系。

Relations *r*, *s*:

A	B
α	1
β	2

r

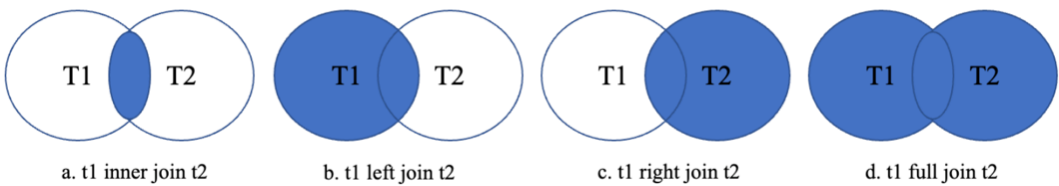
C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

r x *s*:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

在这个基础上，还有 inner join、left join、right join、full join。其中 inner join 称为内连接，其余三个称为外连接（out join）。



通常都会将 out join 改写为 inner join，因为内连接允许更多的连接顺序选择、基表谓词条件下压，有助于查询

优化器选择到更好的查询执行计划，提升查询效率。之前 OceanBase 优化器团队出过一篇改写系列的文章，感兴趣的可以研究一下，博客链接：<https://open.oceanbase.com/blog/10900347> 《SQL 改写系列九：外连接转内连接的常见场景与错误》。

这里列举了几种连接算法的伪码，可以帮助理解每一种连接算法的原理。

- Nest-loop join 算法的基本思想就是两个 for 循环。每次会迭代外表的一行，然后再以此为驱动去迭代内表的每一行 tuple，如果满足连接条件，就会加入结果集。这里有一个基本的优化思想，即可以将小表放在外层作为驱动表，这样会减少内表的循环次数，从而也会减少内表的遍历次数。

```
For t IN Rt:
  For s IN Rs:
    result.add(t,s)
```

- Index Nest-loop join 算法是利用索引加速的一个嵌套循环算法。假设在内表有索引，且是一个等值的，每次驱动外表的一行，内表可以利用等值进行一个索引搜索，就不需要去遍历内表的全部元组，从而减少 IO 的消耗。

```
For t IN Rt:
  For s IN Rs using index:
    result.add(t,s)
```

- Block Nest-loop join 算法的基本思想是每次的迭代不再是一行的 tuple，而是按块进行连接运算。先读取 Rt 表的一个 Block，再读遍读取 Rs 的一个 Block，然后将这两个表的元组进行连接。这个算法的基本思想就是不再以行为粒度，而是以块为粒度，从而减少 IO 的消耗。

```
For Bt IN Rt:
  For Bs IN Rs :
    For t IN Bt:
      For s IN Bs:
        result.add(t,s)
```

- merge join 算法利用了归并排序的思想，先将两张关联表各自做排序，然后从各自的排序表中去抽取数据到另一个排序表中做一些条件匹配，从而生成结果集，示例中没有把它的伪码写的很详细，但可以说明基本的原理就是归并的思想。

```
Ra=Sort(Rt)
Rb=Sort(Rs)
Merge_join(Ra,Rb)--result.add(t,s)
```

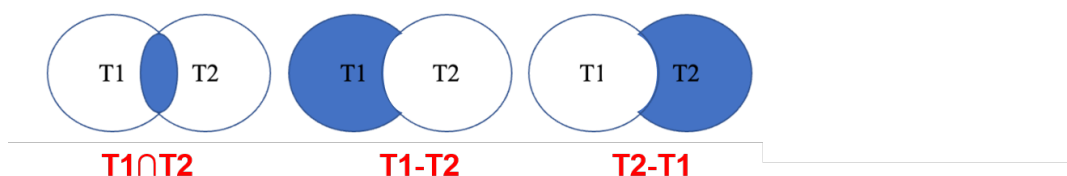
- hash join 算法是将其中一张表先在内存中建立一个哈希表，这张表通常是两张表中比较小的那一张表。需要注意的是，如果这个哈希表比较大，无法一次性构造在内存中，那就需要将这张表分多个 partition 写入磁盘，就会多一个写的拆架，也会降低效率。因此 hash join 算法通常适合于较小的表，可以完全存放在内存中。

```
HT=Build_hash(Rt)
For s IN Rs:
    if hask_key(s) found in HT:
        result.add(t,s)
```

通常 merge join 的效率会比 hash join 差, 但如果两张关联的表已经排序好了, 此时就不需要做排序操作, 这时 merge join 的效率就不一定比 hash join 差了。

交、并、差和聚合

交、并、差也是涉及到多表的运算, 通常是用 merge join 或 hash join 的思想, 只不过前面介绍的 merge join 和 hash join 是做连接, 而交、并、差是在 merge 或 hash 的过程中做去重。

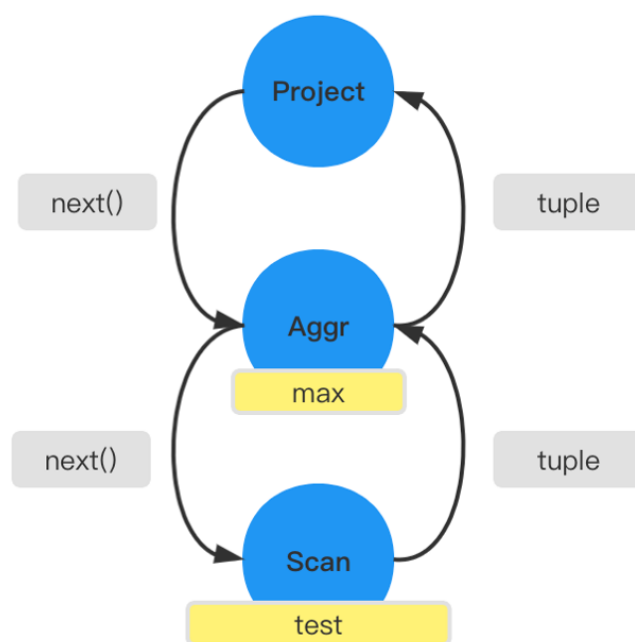


聚合操作也是用 merge 或 hash 的思想, 通常通过 merge 或 hash 将同一组中的元组放在一起, 然后在每个组内进行相应聚合函数的实现, 这里列举了几个优化示例帮助理解。

- 部分聚合, 对于 count、min、max 和 sum, 可以保留组中目前找到的元组的聚合值。
- 当将部分聚合合并为计数时, 将部分聚合相加。
- 对于 avg, 保留 sum 和 count, 并在最后用 sum 除以 count。

物化和流水线计算

通过一个示例来介绍物化和流水线计算, SQL 语句 `select max(c1) from test;` 的语义是为了查询 test 表中 c1 的最大值。



如上图所示这是火山引擎的一个执行流水线图。假设没有任何的潜在知识设想，那么一种比较自然的想法，就是对每一个操作都预先计算生成中间的临时关系。比如先扫描出 `test` 中的元组，放在内存缓冲块中，当投影操作向下驱动的时候，聚合算子可以直接读取已经扫描出来在缓冲块中的 `test` 元组的数据，然后直接去计算 `c1` 的最大值就可以了。但这样有个明显的缺点，如果申请的内存比较大（假设 `test` 比较大），那么这种代价的消耗是不划算的。

流水线计算的思想是把从上到下的多个算子，当成一体的算子。每次当投影算子向下驱动的时候，聚合算子会从扫描算子中取到一行元组，扫描算子每次只会向上驱动一行，然后聚合算子会马上根据这一行做运算，去判断是否是更大的值，然后进行更新。在整个过程中只需要消耗一行 `tuple` 大小的空间，而不需要去申请一个较大的内存缓存区块去存取物化的整个表的代价。

物化计算不一定比流水线计算效果差，这需要看具体的业务需求，只是在这个示例中，用流水线的计算方式更优。

希望通过以上知识的学习，在以后看到一条业务 SQL 的计划时，能够去注意这些计划中关系对应操作的基本原理，这也有助于判断查询计划是否更加高效。

4.7 关系表达式的等价规则转换

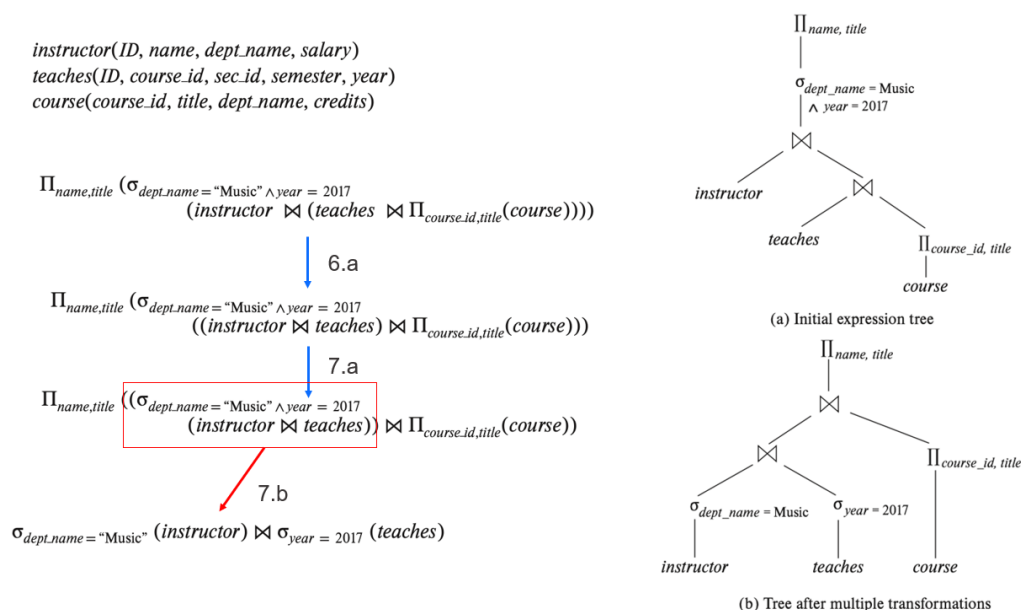
本节主要介绍关系表达式的等价规则转换。如下所示列举了一些常见的规则，在这些规则的基础上，还能推导出更多规则。

1. $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
2. $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$
3. $\Pi L_1(\Pi L_2(\dots(\Pi L_n(E))\dots)) \equiv \Pi L_1(E)$ 其中 $L_1 \subseteq L_2 \dots \subseteq L_n$
4. (a) $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$
(b) $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$
5. $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$
6. (a) $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$
(b) $(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$ θ_2 条件只涉及 E_2 和 E_3 的属性.
7. (a) θ_0 只涉及 (E_1) 的属性 $\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$
(b) θ_1 只涉及 (E_1) 的属性, θ_2 只涉及 (E_2) 的属性 $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$
8. (a) if θ 只涉及 $L_1 \cup L_2$ 的属性, $L_1 \cup L_2$ 分别来自 E_1, E_2 : $\Pi L_1 \cup L_2(E_1 \bowtie_{\theta} E_2) \equiv \Pi L_1(E_1) \bowtie_{\theta} \Pi L_2(E_2)$
(b) L_3, L_4 分别是连接条件 θ 涉及 E_1, E_2 的属性, 另外 L_1 与 L_3 无次, L_2 与 L_4 无次集:
 $\Pi L_1 \cup L_2(E_1 \bowtie_{\theta} E_2) \equiv \Pi L_1 \cup L_2(\Pi L_1 \cup L_3(E_1) \bowtie_{\theta} \Pi L_2 \cup L_4(E_2))$
外连接 $\bowtie, \bowtie, \text{ and } \bowtie$ 类似
9. (a) $E_1 \cup E_2 \equiv E_2 \cup E_1$
(b) $E_1 \cap E_2 \equiv E_2 \cap E_1$
10. $(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$
 $(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$
11. 反操作与 $\cup, \cap$ and $-$
a. $\sigma_{\theta}(E_1 \cup E_2) \equiv \sigma_{\theta}(E_1) \cup \sigma_{\theta}(E_2)$
b. $\sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap \sigma_{\theta}(E_2)$
c. $\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$
d. $\sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap E_2$
e. $\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - E_2$
12. 投影与并操作
 $\Pi L(E_1 \cup E_2) \equiv (\Pi L(E_1)) \cup (\Pi L(E_2))$
13. 聚合与选择 $\sigma_{\theta}(G \gamma A(E)) \equiv G \gamma A(\sigma_{\theta}(E))$
连接条件 θ 只涉及 G 上的属性
14. a. 全连接可交换
 $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$
b. 外连接不可交换, 但可以有如下变换:
 $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$
15. 选择操作与外连接, θ_1 只涉及 E_1 上的属性
a. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$
b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv E_2 \bowtie_{\theta} (\sigma_{\theta_1}(E_1))$
16. 外连接在一定条件下转内连接
a. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2)$
b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_1) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2)$
 θ_1 涉及 E_2 上空值判断, 比如 where $E_2.a$ is not null

接下来简单介绍几个规则，其他规则的详细内容可以查阅相关资料进行研究。

- 第一个规则是在 E 表上选择满足 θ_1 和 θ_2 两个条件的属性。它可以分解为先对 E 表做 θ_2 条件的选择，再做满足 θ_1 条件的选择。因为这个规则的意思是选择两个过滤条件的交集部分。
- 第二个规则说明多次的选择，是满足交换率的。可以先做 θ_2 的选择，也可以先做 θ_1 的选择。
- 第三个规则的意思是多次的投影可以简化为单次的投影。
- 第四个规则主要关于笛卡尔积转连接，a 规则是说可以在连接的过程中直接进行过滤，而不用先做完全部的笛卡尔积操作再进行过滤。b 规则表示在连接的基础上再进行一次过滤，本质上跟 a 规则是一样的。

为帮助大家理解，接下来讲解一个示例。



这个示例中有三个关系，分别是表 `instructor`、`teaches`、`course`。通常来说迭代算法一般是从左边的算子开始迭代，也就是最左端的表是最先进行运算的。从右上角这个查询计划 (a) 中可以看出 `instructor` 表连接的关系其实是 `teaches` 表与 `course` 表投影的中间关系上的连接。在这个过程中，消耗是比较大的，因为在进行第一个连接的时候，需要先保存 `teaches` 表和 `course` 表连接的中间结果，这样代价是比较大。

接下来看一下，经过前面介绍的规则改写之后，这个查询计划会发生什么样的变化。

首先利用规则 6.a，先不进行 `teaches` 表和 `course` 表的投影的中间关系的连接，而是先将 `instructor` 表和 `teaches` 表进行连接，再利用规则 7.a 将选择下推。也就是说，这里的选择 `dept_name` 和 `year` 其实是 `instructor` 表和 `teaches` 表上的属性，可以看到图中红框部分，把选择条件下推到连接上了。

然后单独看红框的部分，由于这两个选择的属性涉及到了两张表，因此可以利用规则 7.b 将这个选择条件分解，下推到两张表上，再进行一个连接。可以看到最终优化生成的连接计划，也就是图中右下角的查询计划 (b)。

经过优化之后，查询计划形状变得更加左偏了。在这种情况下，`instructor` 表和 `teaches` 表进行连接的时候，首先第一个保存的临时关系变成了 `teaches` 表选择之后的一个中间临时关系，然后得到中间连接结果之后，生成临时关系就可以不用保存了。而优化之前的计划，是在连接之前需要保存一个较大的中间临时关系。因此在关系代数的基础上进行等价规则的转换可以帮助优化查询计划。

4.8 执行计划的选择

这里介绍几个简单的例子帮助大家理解执行计划选择时的一些影响因素。

连接顺序和连接算法的选择

如下图所示, r_1 到 r_n 的连接, 根据等价规则转换, 可以先进行任意表的连接, 具体有多少种连接顺序可以根据公式 $(2(n-1))/(n-1)!$ 计算得出。当 n 等于 3 的时候, 有 12 种; 当 n 等于 7 的时候, 就有六十多万种, 大家可以去计算一下这个连接数量是非常可怕的, 如果要去遍历搜索每一种连接顺序, 这个代价非常的高。

$$r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

$$\begin{array}{cccc} r_1 \bowtie (r_2 \bowtie r_3) & r_1 \bowtie (r_3 \bowtie r_2) & (r_2 \bowtie r_3) \bowtie r_1 & (r_3 \bowtie r_2) \bowtie r_1 \\ r_2 \bowtie (r_1 \bowtie r_3) & r_2 \bowtie (r_3 \bowtie r_1) & (r_1 \bowtie r_3) \bowtie r_2 & (r_3 \bowtie r_1) \bowtie r_2 \\ r_3 \bowtie (r_1 \bowtie r_2) & r_3 \bowtie (r_2 \bowtie r_1) & (r_1 \bowtie r_2) \bowtie r_3 & (r_2 \bowtie r_1) \bowtie r_3 \end{array}$$

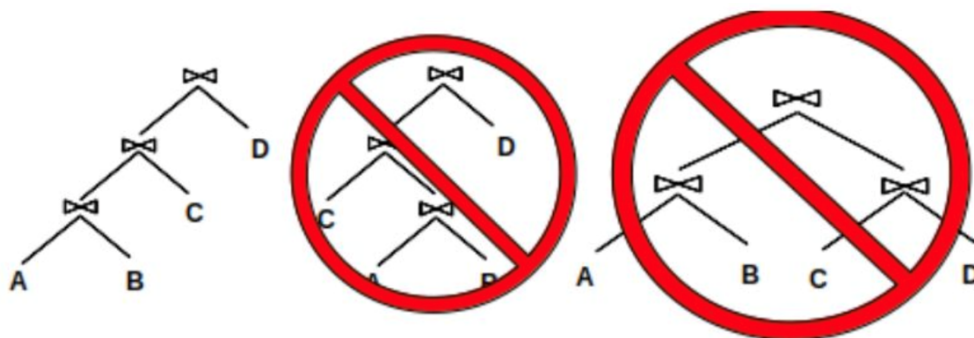
$$(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$$

根据上节介绍的等价规则, 连接操作是满足交换率的。假设当 $n=5$ 时, 可以在前三个的 12 种连接顺序中选择最好的一种与另外的两个进行连接, 这样就又有 12 种连接顺序。对比原来的 5 个表进行任意连接导致较大的搜索空间而言, 只有 12+12 种, 这其实就是一种动态规划的思想。

另外这里面还涉及到 Interesting sort order 的概念, 假设选择先进行前三个表的连接, 采用的是 merge join (一般是有序的), 即三个表连接之后会产生一个有序的顺序。这时需要判断这个有序的顺序能否在跟第四表和第五张表的连接中起到作用, 如果能起到作用, 那么这个连接就能产生一种叫做 Interesting sort order 的优势, 通常也是会优先考虑这个优势的。

计划树形状

如下图中非左深树打了红叉, 即这两个计划树不被推荐。在上一节介绍等价转换的例子中, 也提到过左深跟右深的一些区别, 等价转换之后更有优势的是偏左深的计划树的形状。



根据前文介绍可以总结出，左深树有以下两个明显优势：

- 减少了中间的临时关系，因为许多的算法都是从左边开始迭代的。
- 减少了连接顺序，前面介绍的公式 $(2(n-1))/(n-1)!$ 是在任意连接的基础上总结归纳得到的，也就是说上图中这三种情况其实都包含在这条公式里面。如果只考虑左深这种情况，那么归纳出来的公式应该是 n 的阶乘，相比于这个公式的计算结果，连接的选择空间就会小很多。

启发式规则

在前文介绍的等价规则转换的基础上，加入代价的判断，有些代价不好估计，但有些规则经过长期的验证，已经确认是有效的。如下列举了一些规则，在优化器里面会考虑这些启发式规则带来的优势，尽量去应用这些启发式的规则。

- 选择尽量下推
- 投影尽早下推
- 连接消除避免完全笛卡尔积操作
- 连接的一个关系在连接属性上有索引，将这关系放在内层循环中采用索引连接
- 连接的一个参数是排序的，用 merge join 可能比 hash join 好
- 多个关系的并或交时，先对最小关系进行组合
- 子查询的消除
- 嵌套连接的消除
- 使用等价谓词重写对条件化简

4.9 代价

在前面的介绍中，很多的地方都提到了代价，但一直没有说代价是什么，只是说了在不同的算法中哪些更有优势，例如左深为什么有优势、为什么要利用索引的优势、为什么要利用不同的嵌套循环算法等。这些其实都与底层效率有关，主要是 IO 代价。为帮助大家理解代价，接下来介绍两个示例。

示例 1: 选择运算的操作代价

- 线性扫描

$$\text{Cost} = \text{br block transfers} + 1 \text{ seek}$$

$$\text{Cost} = (\text{br}/2) \text{ block transfers} + 1 \text{ seek}$$

说明

br 指一个表的元组所占据的磁盘块的块数。比如有一张 test 表，它元组只需要磁盘或闪存上的三个 block 就存得下，每个 block 是 2M。

如果采用线性扫描，假设表的元组的存放顺序是按照组件存放的，也就是如果要读取一张表的内容，并且采用顺序存放，那代价就是一次寻址的代（需要先寻到这张表的第一个块，再去读取它的全部块），再加上读取全块的代价，即 $\text{br block transfers} + 1 \text{ seek}$ 。

假设是在组件上进行索引扫描，由于组件是有序的，就可以利用二分搜索（一次寻址的代价加上读取二分搜索的代价），即 $(\text{br}/2) \text{ block transfers} + 1 \text{ seek}$ 。

- B+ 树索引扫描

$$\text{Cost} = \text{hi} * (\text{tT} + \text{tS}) + \text{tS} + \text{tT} * \text{b}$$

$$\text{Cost} = (\text{hi} + 1) * (\text{tT} + \text{tS})$$

如果这张表上有 B+ 树索引，这个时候代价计算方式就不一样了。假设是全表扫描，首先需要进行索引表的扫描，按照层高每层读取一个块，每一个块都要进行寻址，也就是读取每一层的块，加上寻道时间。到了叶子节点之后（B+ 树索引上，叶子结点的是连续的），又有一次的寻道时间，这时候再读取 b 个块。

如果是主键扫描，就不需要连续的去读取叶子节点上的每一个块，只需要多一层的读取，也就是 $(\text{hi} + 1)$ ，再乘以每一层的这个块的寻道，还有读取的时间。

示例 2: 连接顺序的代价

在上节中讲到左深树的形状更有利于连接，因为可以减少中间临时空间。这里的示例也是这个原理，即先将两个较小关系的表进行组合，这样产生的中间临时关系比较小。

$$(r_1 \bowtie r_2) \bowtie r_3 \equiv r_1 \bowtie (r_2 \bowtie r_3)$$

统计信息

统计信息是在进行代价估计过程中不可缺少的一部分，统计信息通常有以下几项：

- 表的行数（row count），对于关系 R ，行数可以表示为 $\text{row_count}=|R|$ 。
- 某一列中 ndv 的数量（number of distinct value），即去重之后元素的数量。
- 选择率（selectivity），对于某一个过滤条件，被过滤的行的比例。对于关系 R 上的过滤条件（谓词） p ，其选择率可以表示为 $\text{sel}(p)=|\sigma_p R|/|R|$ 。
- 基数（cardinality），狭义上的基数是指数据库中某个表的某个列中不重复行的总个数。

这里通过一个简单的例子来帮助大家理解统计信息在代价估计中的意义。在 Scan 操作的时候，应该使用全表扫描还是索引扫描呢？前几节介绍了一些索引扫描在等值比较查询中能够有效的减少 IO 的说法，但这个说法只能针对于单次的操作而言，从整体上对 SQL 的查询效率不一定是最优的。

比如 SQL 语句 `SELECT * FROM tab WHERE id=2022`，假设在 `tab` 表上 `year` 字段基本都是 2022，那么针对 `id=2022` 这个过滤条件被过滤的行是很少的，因为这整张表上的每一个 tuple，在年份这个字段上基本都是 2022，就没有过滤的意义。这时候如果还进行索引扫描，只会单纯多了一个回表的代价，还不如全表扫描，而且它的投影（SELECT）也是针对于全部的元组的，所以这时候就不需要走索引扫描。

4.10 课后实践

- MiniOB 题目 1: 扩展支持 date 字段类型

【题目说明】：当前 MiniOB 已经支持整数 (ints)、浮点数 (floats) 和字符串 (chars) 类型，请参考现有的代码，增加 date 类型字段，在训练营上提交测试。

注意

字段相关的操作会贯穿整个 SQL 处理过程，从词法解析到执行，还要考虑索引，就是 b+ 树相关的操作，此外还需要考虑 date 的对比、合法性判断、与字符串的转换等。

- MiniOB 题目 2: 实现多表查询功能

【题目说明】：当前 Miniob 支持单表查询，需要扩展 MiniOB 支持多表查询，在训练营上提交测试，完成 select-tables 题目。

【训练营测试的原理】：后台执行时，有一系列的 SQL 语句，发送到 MiniOB，然后将执行输出的结果，跟预期的 result 文件做对比。

【练习方式】：完成后在 [训练营](#) 上提交测试，训练营使用文档请参见 [训练营使用说明](#)。

第 5 章：事务引擎

本章主要介绍事务相关内容。事务是数据库很重要的概念之一，通过事务保证了很多操作量大，数据量高的业务的稳定性。

本章目录

5.1 事务简介

5.2 故障类型和事务模型

5.3 undo 日志

5.4 redo 日志

5.5 undo/redo 日志

5.6 隔离级别

5.7 锁

5.8 时间戳

5.9 多版本并发控制

5.10 课后实践

5.1 事务简介

可靠性是衡量数据库好坏的重要指标，它要求数据库始终保证数据的一致性，当故障发生时已经提交的数据不会丢失，并在系统重启之后数据能够快速恢复。

数据在写入时往往先被写到内存缓冲区，之后在某一时间才会真正持久化到磁盘中。对于一个简单的增、删、改、查操作，可以等待数据写到磁盘后再返回操作成功，这样就不会出现一致性问题。

而实际应用中经常出现一组操作需要作为整体执行的情况，比如两个账户之间进行转账，一个账户的资金转出和另一个账户的资金转入要作为一个完整的操作，要么都成功要么都失败，不允许出现数据不一致的情况。基于这样的需求，数据库提供了事务机制，用于将一系列数据库操作组合成一个完整的逻辑过程。

事务就是数据库中一系列数据操作的集合，集合可能有大有小，但无论集合中有多少操作，要么一起操作成功，要么失败一起回滚。无论集合中有多少操作，对于用户来说，就是一个操作，相当于一个原子操作。

事务的 ACID 属性：

- 原子性（Atomicity）：事务是最小的执行单位，不允许分割。事务的原子性确保动作要么全部完成，要么完全不起作用，比如上文转账的示例。
- 一致性（Consistency）：确保从一个正确的状态转换到另外一个正确的状态，这就是一致性。要么事务成功，进入一个新的状态，要么事务回滚，回到过去稳定的状态。
- 隔离性（Isolation）：并发访问数据库时，一个用户的事务不被其他事务所干扰，各并发事务之间是独立的。
- 持久性（Durability）：一个事务被提交之后，对数据库中数据的改变是持久的，即使数据库发生故障，比如断电宕机，也不应该对其有任何影响。

由于一个事务包含多个操作，可能出现事务进行到一半发生故障的情况，此时数据库会处于不一致的状态。数据库要恢复到一致性状态，要么撤销已经执行的操作，恢复到事务执行前的状态，要么重做未完成的操作，恢复到事务执行后的状态。要知道哪些操作需要撤销、哪些操作需要重做，一般会用到一种技术：日志。

在数据库设计中，将记录撤销操作的日志称为“undo log”，将记录重做操作的日志称为“redo log”，接下来的介绍中会讨论它们如何处理数据不一致的问题。

5.2 故障类型和事务模型

上节中提到数据库可能会发生“故障”，实际上数据库的故障类型有很多种，本节主要讨论这些故障的产生和处理方法，并建立数据库的存储管理模型，以便后面讨论事务的并发访问。

故障类型

- 数据输入错误

数据内容的错误是无法避免的，如果用户在输入姓名或身份证号时输错了一位，那么错误是很难被直接发现的。处理数据输入错误的常见技术就是编写约束和触发器，可以及时找出不满足格式的错误数据。

- 介质故障

磁盘的部分区域损坏，一般只会影响少数几位数据，可以通过奇偶校验等方式检测到。如果整个磁盘损坏，要么为数据维护一个备份，要么采用 RAID 阵列保存数据。此外还可以将数据的副本保存在多个远程节点上，一个节点的故障不会导致数据丢失，这就是分布式的思想。

- 系统故障

系统故障主要包括掉电和软件错误，由于内存是“易失性的”，如果数据提交了但没有写入到磁盘，当系统掉电时这部分数据就会丢失。软件错误可能会直接覆盖内存中的数据，就相当于数据已经丢失。解决这类问题的办法就是维护日志，将所有对数据库的修改操作都记录下来，以便重启系统后进行恢复。

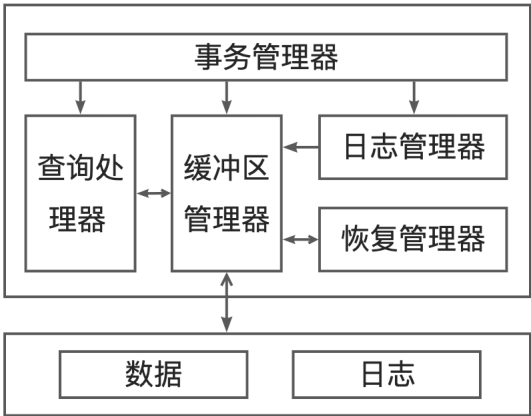
- 机房故障

机房故障有可能是整个机房发生了火灾、爆炸等意外，或是自然灾害导致一片地区受到破坏，这种情况下 RAID 和数据校验都无法发挥作用，只有备份机制（异地）才可以防止数据丢失。

事务模型

为了更好地理解事务机制，这里建立一个如下图所示的模型。

- 事务管理器统筹管理事务的执行
- 查询处理器负责解析 SQL 命令
- 缓冲区管理器负责维护内存缓冲区和刷写数据
- 日志管理器负责维护日志
- 恢复管理器负责在系统重启后恢复数据



事务原语

在数据库运行过程中，刚插入的数据往往不会直接写入磁盘，而是先缓存在内存中。对于一个运行中的数据库，可以将其地址空间简单分成三个部分：

- 1. 持久化保存数据的磁盘空间
- 2. 缓冲区对应的内存或虚拟内存空间
- 3. 事务的局部地址空间（也在内存中）

事务要读取数据，首先要将数据取到缓冲区中，然后缓冲区的数据可以被事务读取到局部空间。事务的写入过程与此相反，先在局部空间中创建新值，然后再将新数据拷贝到缓冲区中。缓冲区中的数据通常是由缓冲区管理器决定何时写入磁盘，而不是立刻持久化到磁盘。

为了便于研究日志和事务管理的细节，我们使用一系列原语来描述数据库操作：

- 1. INPUT(X)：将数据库元素 X 从磁盘拷贝到缓冲区。
- 2. READ(X, t)：将数据库元素 X 从缓冲区拷贝到事务的局部变量 t。
- 3. WRITE(X, t)：将局部变量 t 的值拷贝到缓冲区的数据库元素 X，如果 X 不在缓冲区，先执行 INPUT(X)。
- 4. OUTPUT(X)：将数据库元素 X 从缓冲区拷贝到磁盘。

在这里假设：数据库元素 X 的大小 = 磁盘块大小 = 缓冲区块大小。

【例 5.1】银行转账事务

一个数据库中有 A、B 两个账户（元素），A 和 B 之间进行转账操作，在任何一致的状态中它们的值的总和是固定的。

一个转账事务 T 主要有两个步骤：

A: A 账户减 10

B: B 账户加 10

假设 A 和 B 的初值都为 15，事务 T 从一个一致的状态（A+B=15+15=30）开始，事务正常执行且期间没有发生系

统故障，那么最终的状态必然也是一致的，A 和 B 的值发生了变化，但他们的和没有发生变化（ $A+B=5+25=30$ ）。

T 的执行包括从磁盘读取 A 和 B，执行运算，将 A 和 B 的新值写入缓冲区。之后缓冲区管理器会执行 OUTPUT 原语，将数据写回磁盘。表 5-1 中展示了事务 T 的执行过程，以及每个步骤执行之后 A 和 B 在缓冲区和磁盘中的值。

表 5-1

操作	t	A（内存）	B（内存）	A（磁盘）	B（磁盘）
READ(A, t)	15	15		15	15
t: = t-10	5	15		15	15
WRITE(A, t)	5	5		15	15
READ(B, t)	15	5	15	15	15
t: = t+10	25	5	15	15	15
WRITE(B, t)	25	5	25	15	15
OUTPUT(A)	25	5	25	5	15
OUTPUT(B)	25	5	25	5	25

1. 第 1 步 READ(A, t) 命令将 A 的值拷贝到局部变量 t 中，如果 A 不在缓冲区中，那么就会先执行 INPUT(A) 命令。
2. 第 2 步将 t 减 10，这一步不会改变 A 在缓冲区和磁盘上的值。
3. 第 3 步将 t 写到缓冲区的 A 中，这一步也不会影响磁盘上 A 的值。
4. 直到第 7 步，OUTPUT(A) 将 A 的新值写入磁盘，完成持久化。

如果表 5-1 中的步骤顺利执行，那么事务前后数据库都处于一致性状态。

- 如果在 OUTPUT(A) 之前系统发生故障，因为磁盘上的数据没有任何变化，一致性得以保持。
- 如果系统故障在 OUTPUT(B) 之后发生，磁盘上的 A 和 B 都已经修改，仍然满足一致性要求。
- 如果系统故障发生在 OUTPUT(A) 和 OUTPUT(B) 之间，那么数据库就会处于不一致的状态，这种情况下就需要进行修复，要么将 A 和 B 都重置为原值，要么将它们都更新为新值。

5.3 undo 日志

日志就是记录事务相关操作的文件，每个改变数据库的操作都会生成日志记录，在系统故障发生之后，可以通过日志将数据库恢复到一致性状态。本节主要介绍回滚日志（undo log），它通过撤销未提交的事务来恢复数据库的一致性状态。

日志记录

日志由多个日志记录组成，每条日志记录对应一个重要操作，比如更新数据、删除数据、提交事务等，这些操作往往会改变数据库的一致性状态。日志空间最初在内存中由缓冲区管理器分配，日志记录则由日志管理器负责创建，缓存中的日志记录会尽快被写到持久性的存储介质中。

记录事务操作的几种日志记录类型如下：

1. [START T]：表示事务 T 开始。
2. [COMMIT T]：表示事务 T 已经完成。[COMMIT] 记录不能保证数据已经更新到磁盘上，如果需要应由日志管理器控制事务的逻辑。
3. [ABORT T]：表示事务 T 无法完成，需要终止。如果事务终止，那么它所做的任何更改都不能写到磁盘上，如果已经写到磁盘就必须消除这些影响。
4. [T, X, v]：undo 日志的一条更新记录，表示事务 T 改变了元素 X，它的原值是 v。更新记录在数据库对内存执行 WRITE 操作时产生，不需要记录数据库元素的新值。

事实上日志记录的类型还有很多，涉及的操作也不限于插入、更新、删除等基础操作，为了便于说明，在本节中只考虑对已有数据的更新操作。

undo 日志规则

为了保证数据能从系统故障中顺利恢复，undo 日志必须遵循以下两条规则：

R1：只有当形如 [T, X, v] 的日志记录写到磁盘后，事务 T 才能够改变数据库元素 X 的值。

R2：如果事务提交了，只有当事务 T 改变的所有数据库元素的值写到磁盘之后，[COMMIT T] 日志记录才能被写入（而且应该尽快）。

规则 R1 比较好理解，如果数据库元素已经发生改变但是没有记录下来，那么系统恢复时就无法知道数据库是否处于不一致的状态，也不知道数据库元素原来的值是什么，恢复流程也无法进行下去。R2 是由 undo 日志的性质决定的，如果 [COMMIT T] 记录已经写入日志，那么磁盘上的数据一定全部更新了，不用撤销事务相关操作。但如果如果没有 [COMMIT T] 记录，哪怕事务实际已经完成，也要撤销事务的所有操作，以保证数据库的一致性。

为了尽快将日志记录写到磁盘上，需要一条“刷写日志”命令来告诉缓冲区管理器将所有未持久化的日志写入磁盘。在后面的示例中，将这一命令记作“FLUSH LOG”。

【例 5.2】考虑上节例 5.1 中的事务，并将 undo 日志应用到其中，最终事务 T 的操作以及日志刷写操作流程如表 5-2 所示。

表 5-2

步骤	操作	t	A（内存）	B（内存）	A（磁盘）	B（磁盘）	日志
1							[START T]
2	READ(A, t)	15	15		15	15	
3	t := t-10	5	15		15	15	
4	WRITE(A, t)	5	5		15	15	[T, A, 15]
5	READ(B, t)	15	5	15	15	15	
6	t := t+10	25	5	15	15	15	
7	WRITE(B, t)	25	5	25	15	15	[T, B, 15]
8	FLUSH LOG						
9	OUTPUT(A)	25	5	25	5	15	
10	OUTPUT(B)	25	5	25	5	25	
11							[COMMIT T]
12	FLUSH LOG						

1. 事务的第 1 步，将 [START T] 记录写到日志中，标志着事务的开始。
2. 第 4 步，将 A 的新值写入缓冲区，并生成一条日志项 [T, A, 15] 记录 A 的旧值。
3. 第 8 步，将日志刷写到磁盘中，只有反映 A 和 B 发生变化的日志记录完成持久化，对 A 和 B 的修改才能落盘。
4. 第 11 步，创建 [COMMIT T] 日志记录，这一步必须在 A 和 B 的新值写到磁盘之后才能进行。
5. 第 12 步，刷写日志，整个事务彻底完成，这一步需要尽快，但不一定会立刻进行，中间还可能还有其他事务的操作。

如果 [COMMIT T] 记录没有及时写到磁盘，在较长一段时间内查看日志都会认为事务 T 还未提交，这时候如果系统发生故障则可能需要撤销早已完成的事务。由于事务 T 可能和其他事务并行执行，如果其中一个事务刷写日志，那么事务 T 的日志记录可能比预期更早地出现在磁盘上。这对于反映数据原值的日志记录而言并没有影响，但对于 COMMIT 操作，必须等到 OUTPUT 全部完成之后才创建 [COMMIT T] 记录。

使用 undo 日志恢复数据

当有了前面的 undo 日志之后，就足以应对一般的系统故障了。现在假设系统故障发生了，事务 T 的部分更改已

经应用到磁盘上，部分缓冲区中的更改却丢失了，此时数据库就可能处于不一致的状态。

首先考虑一种简单的方案：不管日志文件有多大，都要遍历整个日志。

恢复管理器首先将事务分成已提交事务和未提交事务，如果存在日志记录 [COMMIT T]，根据规则 R1 可知，事务 T 所做的更改已经全部写到磁盘，事务 T 不会影响数据库的一致性状态。

如果在日志记录中发现了 [START T] 记录，但是并未发现 [COMMIT T] 记录，那么事务 T 所做的更改可能只持久化了一部分，数据库就会处于不一致状态，这时必须撤销事务 T 所做的更改。当然也有可能，事务 T 更新的数据都写到了磁盘上，只是日志记录没有及时刷写下去，但是除了读到 [COMMIT T] 记录之外，无法判断事务是否已经提交，所以仍然只能撤销事务 T。

根据规则 R1，如果事务 T 修改磁盘上的元素 X，那么日志中必然存在 [T, X, v] 记录，系统恢复时依据该记录对数据库元素 X 写入旧值 v，无需考虑 X 是否真的被事务 T 修改了。

现在开始恢复数据，恢复管理器必须从尾部开始扫描日志，记住所有存在 [COMMIT T] 或 [ABORT T] 记录的事务 T，对于每一条 [T, X, v] 记录，有以下两种情况：

1. 如果事务 T 存在 COMMIT 记录，则无须做其他操作。
2. 如果事务 T 未完成（没有 COMMIT 记录）或者已终止（存在 ABORT 记录），那么需要将数据库中 X 的值修改为 v，以防系统故障前 X 的值就已经被修改。

扫描完所有日志并做了上述工作后，恢复管理器需要为未完成且未终止的每个事务 T 写入一条日志记录 [ABORT T]，然后刷写日志。在下次故障发生后，这些具有 ABORT 记录的事务仍然需要回滚，因为无法判断它们是因为系统故障终止还是因为事务本身的问题终止。

【例 5.3】考虑表 5-2 中的事务流程，系统可能随时发生故障，这里说明几种具有代表性的情况：

1. 故障发生在第 12 步之后：[COMMIT T] 记录已经写到磁盘，进行恢复时判断事务 T 已经提交，与 T 相关的日志记录都将被忽略。
2. 故障发生在第 11 步和第 12 步之间：如果 COMMIT 记录已经刷写到磁盘（可能是其他事务执行的 FLUSH 命令），那么和情况 1 一样不需要做任何更改。如果 COMMIT 记录没有写到磁盘，那么恢复管理器认为 T 未完成，对于遇到的 [T, B, 15]、[T, A, 15] 记录，它会将 A 和 B 的值都重置为 15，最后还会写一条 [ABORT T] 记录到日志中。
3. 故障发生在第 10 步和第 11 步之间：COMMIT 一定没有写入，因此事务 T 未完成，会像情况 2 一样撤销。
4. 故障发生在第 8 步和第 10 步之间：由于 COMMIT 记录没有写入，事务 T 也会被撤销，虽然 A 和 B 的更新可能并未到达磁盘，仍然需要确保它们恢复到旧值。
5. 故障发生在第 8 步之前：现在日志是否到达磁盘并不确定，但是规则 R1 保证了日志先于数据到达磁盘，所以只要把遇到的修改记录全部撤销，就能恢复到事务 T 开始前的状态。

极端情况下，可能会在恢复数据的过程中再次发生系统故障，由于 undo 日志保存的是元素的旧值，所以无论执行多少次系统都能恢复到开始的状态，也就是说恢复步骤是幂等的。

检查点机制

使用 undo 日志恢复数据中说明了系统在恢复时需要检查整个 undo 日志。而一旦事务的 COMMIT 记录被写到磁盘上，该事务相关的记录就不再需要了。那么可以在 COMMIT 日志写入时把之前的日志全都删掉吗？答案是否定的，因为通常有多个事务同时执行，如果一个事务删掉 [COMMIT T] 以及之前的记录，另一些活跃事务在这之前的记录就可能会丢失。

解决上述问题一个简单的办法就是周期性地对日志做检查点，在一个检查点中可以：

- 1. 停止接受新的事务。
- 2. 等到所有活跃事务写入 COMMIT 或 ABORT 记录。
- 3. 将日志刷写到磁盘。
- 4. 写入日志记录 [CKPT] 并再次刷写日志。
- 5. 重新开始接受新的事务。

现在可以确保检查点之前执行的事务已经全部完成并提交，系统恢复时这些事务都不用撤销，所以检查点之前的日志都可以删除。此时从后往前扫描日志，未完成事务都是在检查点记录写入之后才能开始，所以它们的记录只可能出现在 [CKPT] 记录之后，当看到 [CKPT] 记录时，就知道已经扫描了所有未完成事务，之前的记录无需扫描。

【例 5.4】 考虑一个事务并发执行的例子，假设某一时间日志内容为：

[START T1]

[T1, A, 10]

[START T2]

[T2, B, 20]

这时决定做一个检查点，由于存在活跃的事务 T1 和 T2，需要等到它们结束，并且从现在开始不再接受新的事务。检查点完成后日志可能的情况如表 5-3 所示。

表 5-3

[START T1]
[T1, A, 10]
[START T2]
[T2, B, 20]
[T2, C, 30]
[T1, E, 40]
[COMMIT T2]

[COMMIT T1]
[CKPT]
[START T3]
[T3, M, 50]

假设这时发生系统故障，从尾部开始扫描日志，可以判断 T3 是一个未完成事务，需要撤销。当看到[CKPT]记录时，就知道之前的事务已经提交，没必要再检查之前的日志记录，数据库已经恢复到一致性状态。

动态检查点

检查点机制 中描述的检查点技术已经足以解决系统故障后的数据恢复问题，但它存在一个问题：在进行检查点时数据库无法执行新的事务，如果当前活跃事务耗时较长，在用户看来系统似乎停止工作了。“动态检查点”的技术可以避免这一问题，它在系统进行检查点时允许新事务进入。

动态检查点的主要步骤如下：

1. 写入日志记录 [START CKPT(T1, ..., Tk)] 并刷写日志，其中 T1 到 Tk 是当前所有活跃事务。
2. 等待所有活跃事务提交或终止，期间允许其他新的事务执行。
3. 当 T1 到 Tk 全部完成时，写入记录 [END CKPT] 并刷写日志。

采用这种方式，在系统故障恢复时考虑两种情况（从后往前扫描日志记录）：

1. 如果先遇到 [END CKPT] 记录，那么所有未完成的事务都是在 START CKPT 记录之后开始的。因此只需要向前扫描直到 START CKPT 记录，之前的事务都已经完成，日志记录可以全部删掉。
2. 如果先遇到 [START CKPT(T1, ..., Tk)] 记录，那么故障发生在检查点过程中，除了 START CKPT 记录之后开始的事务，其他未完成事务都在 T1, ...,Tk 之中，所以只需要扫描到这些事务中最早的事务的 START 记录即可，之前的事务在检查点开始时就已经完成，不用再检查。

一旦 [END CKPT] 记录写到了磁盘，上一个 START CKPT 记录之前的日志内容可以全部删除。

【例 5.5】考虑例 5.4 中的情况，日志一开始状态如下：

[START T1]

[T1, A, 10]

[START T2]

[T2, B, 20]

现在开始做一个动态检查点，此时 T1 和 T2 是活跃的事务，将其写入日志记录：

[START CKPT(T1, T2)]

然后假设另一个事务 T3 开始了，后续日志的整体内容可能如表 5-4 所示。

表 5-4

[START T1]
[T1, A, 10]
[START T2]
[T2, B, 20]
[START CKPT(T1, T2)]
[T2, C, 30]
[START T3]
[T1, D, 40]
[COMMIT T1]
[T3, E, 50]
[COMMIT T2]
[END CKPT]
[T3, F, 60]

假设这时发生了系统故障，从尾部开始扫描日志，T3 由于是未完成的事务所以必须被撤销，由 [T3, F, 60] 记录可知需要将 F 的值恢复为原值 60。当发现 [END CKPT] 记录时，就知道所有未完成的事务都在前一个 START CKPT 后开始。继续扫描，发现 T1 和 T2 都已经提交，由 [T3, E, 50] 记录可知需要将 E 的值恢复为 50，最后扫描到 [START CKPT(T1, T2)]，至此就完成了数据库恢复。

现在假设故障发生在检查点过程中，故障后的日志内容如表 5-5 所示。

表 5-5

[START T1]
[T1, A, 10]
[START T2]
[T2, B, 20]
[START CKPT(T1, T2)]
[T2, C, 30]
[START T3]
[T1, D, 40]
[COMMIT T1]

[T3, E, 50]

从后往前扫描，可以确定 T3 和 T2 是未完成事务，当发现 [START CKPT(T1, T2)] 记录时，就知道其它可能未完成的事务为 T1，而已经扫描到了 [COMMIT T1]，并且也看到了 [START T3]，因而只需要继续扫描直到看到 [START T2] 记录，就完成了数据恢复。

5.4 redo 日志

上节使用 undo 日志完成了系统故障后的数据恢复，但 undo 日志有一个限制，即在将事务对数据的修改刷写到磁盘前不能提交该事务。这样做可能会导致事务因为等待磁盘 IO 而被阻塞，也会占用较多的磁盘带宽。此外，由于每次故障恢复都要把相关数据恢复到事务之前的状态，即便这些数据已经部分或全部更新到磁盘了，这样反复修改依然浪费了很多资源。

事实上只要有日志用于恢复数据，那么把修改的数据暂存在主存中是安全的，并非一定要马上同步到磁盘。redo（重做）日志就是这样一种日志，它不需要等待数据写到磁盘就可以提交事务。

redo 日志与 undo 日志的区别如下：

- 1. undo 日志在恢复时撤销未完成事务的操作并忽略已提交事务，而 redo 日志忽略未完成事务并重做已提交的事务。
- 2. undo 日志要求先将修改后的数据写到磁盘再写入 COMMIT 日志记录，而 redo 日志要求先写入 COMMIT 日志记录再将修改后的数据写到磁盘。
- 3. 在使用 undo 日志时，记录的是数据库元素的旧值，而使用 redo 日志时，记录的是数据库元素的新值。

redo 日志规则

在 redo 日志中，日志记录 [T, X, v] 的含义是“事务 T 将数据库元素 X 的值修改为 v”。每当一个事务 T 修改一个数据库元素 X 时，必须往日志中写入一条形如 [T, X, v] 的记录。redo 日志需要遵循如下规则：

R1：在修改磁盘上任何数据库元素 X 以前，要保证更新记录 [T, X, v] 以及事务提交记录 [COMMIT T] 都已经写到磁盘上。

【例 5.6】考虑与例 5.2 中相同的事务 T，表 5-6 给出了该事务一个可能的事务序列。

表 5-6

步骤	操作	t	A（内存）	B（内存）	A（磁盘）	B（磁盘）	日志
1							[START T]
2	READ(A, t)	15	15		15	15	
3	t := t-10	5	15		15	15	
4	WRITE(A, t)	5	5		15	15	[T, A, 5]
5	READ(B, t)	15	5	15	15	15	
6	t := t+10	25	5	15	15	15	
7	WRITE(B, t)	25	5	25	15	15	[T, B, 25]

8							[COMMIT T]
9	FLUSH LOG						
10	OUTPUT(A)	25	5	25	5	15	
11	OUTPUT(B)	25	5	25	5	25	

表 5-6 与表 5-2 的主要区别有以下几点:

1. 在第 4 行和第 7 行, 表 5-6 中记录的是 A、B 的新值而不是旧值。
2. [COMMIT T] 记录出现得更早, 只有先将日志刷写到磁盘, 才能将 A 和 B 的新值更新到磁盘上。
3. 在第 10 和第 11 行才将数据写入磁盘, 实际上它们发生的时间可能更晚。

使用 redo 日志恢复数据

因为“先写日志规则”的存在, 只要日志中没有 [COMMIT T] 记录, 就可以知道事务 T 所做的修改没有写到磁盘上, 在恢复时也不需要做任何处理。对于已提交的事务, 记录了数据修改后的新值, 所以可以恢复到事务完成后的状态。在系统故障后使用 redo 日志恢复时, 需要进行以下工作:

1. 确认已提交的事务。
2. 从头开始扫描日志, 对于每一个 [T, X, v] 记录:
 - a. 如果 T 是未提交的事务, 则不需要处理。
 - b. 如果 T 是已提交的事务, 则将数据库元素 X 的值改为 v (不管 X 的值是否已经是 v)。
3. 对每个未完成的事务 T, 在日志中写入一个 [ABORT T] 记录并刷写日志。

【例 5.7】考虑表 5-6 中的日志。

1. 如果故障发生在第 9 步之后, 那么 [COMMIT T] 记录已经被刷写到了磁盘。恢复管理器判定 T 是一个已提交的事务, 继续扫描日志, 日志记录 [T, A, 5] 和 [T, B, 25] 告诉恢复管理器将 A 的值修改为 5, 将 B 的值修改为 25。如果故障发生在第 10 步或者第 11 步之后, 那么写 A 或者写 B 是多余的, 但也是无害的。
2. 如果故障发生在第 8 步和第 9 步之间, 那么 [COMMIT T] 记录已经写入了日志。如果该记录已到达磁盘, 则像情况 1 那样进行恢复即可, 而如果该记录没能到达磁盘, 那么应该和下面的情况 3 一样。
3. 如果故障发生在第 8 步以前, 那么 [COMMIT T] 记录肯定没有到达磁盘, 因此 T 被认为是一个未完成的事务。磁盘上的 A 和 B 不为 T 做任何改变, 并且需要写一条 [ABORT T] 记录到日志中。

redo 日志的检查点

对于 redo 日志, 事务对数据的修改到达磁盘的时间可能比事务提交的时间晚很多, 所以不能仅考虑创建检查点时的活跃事务, 还需要明确知道哪些缓冲区数据还没刷写到磁盘, 以及哪些事务修改了哪些缓冲区。经过 [5.3 undo](#)

日志的分析，已经知道了动态检查点效率更高，所以这里直接介绍 redo 日志的动态检查点机制。

为 redo 日志创建动态检查点的步骤如下：

1. 写入日志记录 [START CKPT(T1, ..., Tk)] 并刷写日志，其中 T1, ..., Tk 是所有活跃（即未提交的）事务。
2. 将当前所有已提交事务已经写到缓冲区但还没有写到磁盘的数据库元素写到磁盘。
3. 写入日志记录 [END CKPT] 并刷写日志。

【例 5.8】表 5-7 给出了一个可能的 redo 日志，其中发生了一个检查点。当开始创建检查点时，只有事务 T2 是活跃的。T1 对 A 写入的新值可能已经到达磁盘，如果没有，那么必须在检查点结束前将 A 拷贝到磁盘。

检查点的结束在几个其他的事件后发生：T2 为数据库元素 C 写入一个值，一个新事务 T3 开始并为 D 写入一个值。在检查点结束后发生的唯一的事情是 T2 和 T3 的提交。

表 5-7

[START T1]
[T1, A, 10]
[START T2]
[COMMIT T1]
[T2, B, 20]
[START CKPT(T2)]
[T2, C, 30]
[START T3]
[T3, D, 40]
[END CKPT]
[COMMIT T2]
[COMMIT T3]

正如 undo 日志那样，检查点可以帮助缩小需要恢复的日志范围。根据最后一条检查点记录是 START 还是 END，有两种不同的情况。

1. 假设故障发生前最后一条检查点记录是 [END CKPT]，对应的 [START CKPT(T1, ..., Tk)] 前提交的事务已经将其所做的修改写到了磁盘。因此只需考虑 T1, ..., Tk 等活跃事务以及检查点之后开始的新事务，必须要将这些事务全部像 使用 redo 日志恢复数据 中那样进行恢复。

在扫描日志时，当找到最早的已提交事务 Ti 的 [START Ti] 记录后就不必继续扫描，因为之前的日志都已经确认提交并刷写到磁盘。由于这些 START 记录可能出现在任意多个检查点前，就可以将所有日志记录链接在一起，以便更快地找到所需记录。

2. 假设日志中最后一条检查点记录是 [START CKPT(T1, ..., Tk)], 此时不能确定在此检查点开始前提交的事务是否已经将其修改写到磁盘。因此必须要索引到前一条 [START CKPT(S1, ..., Sn)] 记录, 并重做这些已经提交的事务, 包括某些 S_i 以及检查点后新开始并提交的事务。

【例 5.9】重新考虑表 5-7 中的日志。

1. 如果故障发生在日志末尾, 从后往前搜索, 找到 [END CKPT] 记录。这时只需要知道写在 START CKPT 记录里的所有事务和检查点后开始的新事务, 因此我们考虑 {T2, T3}, 当找到 [COMMIT T2] 和 [COMMIT T3] 记录时, 就知道它们都需要重做。继续扫描日志直到找到 [START T2] 记录, 根据读到的 [T2, B, 20]、[T2, C, 30]、[T3, D, 40] 记录, 将 B、C、D 的值修改为 20、30、40。
2. 假设故障发生在 [COMMIT T2] 和 [COMMIT T3] 之间, 整体恢复过程与上述过程类似。不同的是事务 T3 会被判定为未提交事务, 不需要重做, 而且恢复完成后需要写入一条 [ABORT T3] 日志记录。
3. 假设故障发生在 [END CKPT] 记录之前。原则上需要扫描到倒数第二个 START CKPT 记录, 但在当前情况下前面没有检查点, 就需要一直扫描到日志头部。最终可以确定已提交的事务为 T1, 并在恢复后将记录 [ABORT T2] 和 [ABORT T3] 写入日志中。

由于事务可能在几个活跃点期间都是活跃的, 因此 START CKPT 记录不仅包含事务的名字, 还可以包含事务的起始地址, 方便恢复时查找日志。而且当写入 [END CKPT] 记录时, 就会知道最早开始的活跃事务 T_i 之前的日志记录都可以删除, 因为它们对数据的修改在生成 [END CKPT] 记录之前就已经刷写到磁盘了。

5.5 undo/redo 日志

前文介绍了两种恢复日志，它们的主要区别在于写日志时保存数据库元素的新值还是旧值，两种方式各有优劣：

- 1. undo 日志要求在事务提交前把数据写到磁盘，可能增加磁盘 IO 次数。
- 2. redo 日志要求在事务提交和日志刷写前保留所有的数据块缓存，这样会增加事务需要的缓冲区空间。
- 3. 如果一个缓存块包含多个数据库元素，那么 undo 日志和 redo 日志在处理缓存区时都存在冲突。例如同一个缓存块包含一个已提交事务的元素 A 和一个未提交事务的元素 B，那么需要将 A 刷写到磁盘，但是 B 又要求不能这样做。

为了解决上述问题，引入了一种称为 undo/redo 日志的日志类型，它通过维护更多的信息来提供日志的灵活性。

undo/redo 日志规则

undo/redo 日志与前两种日志类型基本相同，但在修改数据库元素时写入的日志记录有四个部分。一条 [T, X, v, w] 记录表示事务 T 将数据库元素 X 的值从 v 修改为 w。undo/redo 日志需要遵循以下原则：

UR1：在事务 T 对数据库元素 X 的修改到达磁盘前，日志记录 [T, X, v, w] 必须刷写到磁盘上。

【例 5.10】表 5-8 是例 5.6 中事务 T 的一个变体，其中日志内容和事务操作顺序发生了变化。一条更新日志记录同时包含数据库元素的旧值和新值，[COMMIT T] 可以出现在第 7 步之后的任意位置。

表 5-8

步骤	操作	t	A（内存）	B（内存）	A（磁盘）	B（磁盘）	日志
1							[START T]
2	READ(A, t)	15	15		15	15	
3	t := t-10	5	15		15	15	
4	WRITE(A, t)	5	5		15	15	[T, A, 15, 5]
5	READ(B, t)	15	5	15	15	15	
6	t := t+10	25	5	15	15	15	
7	WRITE(B, t)	25	5	25	15	15	[T, B, 15, 25]
8	FLUSH LOG						
9	OUTPUT(A)	25	5	25	5	15	
10							[COMMIT T]
11	OUTPUT(B)	25	5	25	5	25	

使用 undo/redo 日志恢复数据

undo/redo 日志的恢复策略如下：

- 1. 按照从前往后的顺序，重做所有已提交的事务。
- 2. 按照从后往前的顺序，撤销所有未提交的事务。

【例 5.11】考虑表 5-8 中的动作序列。

- 1. 假设故障发生在 [COMMIT T] 记录刷写到磁盘之后，这时 T 被认为是已提交的事务，需要为 A 和 B 写入日志记录中的新值。
- 2. 假设故障发生在 [COMMIT T] 记录到达磁盘之前，则 T 被认为是未完成的事务，需要为 A 和 B 写入日志记录中的旧值。

和 undo 日志一样，使用 undo/redo 日志的系统中可能出现这样的行为：事务在用户看来已经提交（如在网上预订了一个航班座位然后断开连接），但由于 [COMMIT T] 记录尚未刷写到磁盘，后来的一次故障使该事务被撤销而不是重做。如果这样的可能性是一个问题，那么建议为 undo/redo 日志使用一条附加的规则：

UR2: [COMMIT T] 记录一旦出现在日志中就必须被刷写到磁盘上。

例如，在表 5-8 中我们将在第 10 步后加入“FLUSH LOG”。

undo/redo 日志的检查点

undo/redo 日志的动态检查点相对来说更为简单，创建检查点主要有以下几步：

- 1. 写入日志记录 [START CKPT(T1, ..., Tk)] 并刷写日志，其中 T1, ..., Tk 是所有活跃的事务。
- 2. 将缓冲区的所有脏数据写到磁盘。可以容忍将未完成事务修改的数据写到磁盘，所以能够容忍小于完整块的数据库元素，并因此可以在事务之间共享缓冲区。
- 3. 写入日志记录 [END CKPT]。

【例 5.12】表 5-9 给出了类似于表 5-7 中 redo 日志的一个 un-do/redo 日志，区别在于更新记录的不同。

和例 5.8 中一样，T2 被确定为检查点开始时唯一的已提交事务。由于这一日志是 undo/redo 日志，有可能 T2 写入的 B 的新值 20 已写到磁盘，而这在 redo 日志中是不可能的。在检查点过程中，如果 B 的新值还不在于磁盘上，因为会刷写所有的脏缓冲区，所以肯定会将 B 刷写到磁盘上。同样，如果由已提交的事务 T1 写入的 A 的新值还不在于磁盘上，那么将刷写 A 到磁盘上。

表 5-9

[START T1]
[T1, A, 5, 10]
[START T2]

[COMMIT T1]
[T2, B, 10, 20]
[START CKPT(T2)]
[T2, C, 20, 30]
[START T3]
[T3, D, 30, 40]
[END CKPT]
[COMMIT T2]
[COMMIT T3]

如果系统故障在这一系列步骤的末尾发生，则 T2 和 T3 被判定为已提交的事务。事务 T1 在检查点前提交，由于在日志中发现 [END CKPT] 记录，可知 T1 已经完成并已将其改变写到磁盘上。因此只需要重做 T2 和 T3，就像在例 5.8 中那样。当重做像 T2 这样的事务时，并不需要查看比 [START CKPT(T2)] 还早的记录，因为 T2 在检查点开始前的改变在检查点过程中已经被刷写到磁盘。

假设故障正好在 [COMMIT T3] 记录写到磁盘前发生，那么可以判定 T2 是已提交的而 T3 是未提交的。可以通过将磁盘上的 C 置为 30 来重做 T2；没有必要将 B 置为 20，因为我们知道这一改变在 [END CKPT] 前已到达磁盘。和 redo 日志不同的是，这里还要撤销 T3，也就是将磁盘上的 D 置为 30。如果 T3 在检查点开始时是活跃的，将不得不查看比 START CKPT 记录更早的记录，以确定 T3 是否有更多的动作已到达磁盘因而需要撤销。

可以注意到，这里并没有指明在使用 undo/redo 日志恢复时先撤销还是先重做。事实上，不管先撤销还是先重做，都会面临如下情况：事务 T 提交了且被重做了。然而，T 读取了一个值 X，该值是由某个未提交并被撤销的事务 U 写入的。

问题不在于先重做还是先撤销，实际上数据库管理系统必须保证上述的情况绝不会发生。我们将在后续章节中讨论隔离像 T 和 U 这样的事务的方法，使它们能够并行地访问数据库元素 X 但不会相互影响。

undo/redo 日志的应用

这里介绍两个新的概念：steal 和 force。

- steal/no-steal
 - 如果采用 steal 策略，那么数据库在事务提交之前就可以把修改后的数据写到磁盘，如果系统发生故障就可能导致数据库的不一致状态，因此数据库需要记录 undo log，在系统恢复时回滚未提交的事务。
 - 如果采用 no-steal 策略，在事务提交之前对数据的任何修改都不会到达磁盘，因此不需要记录 undo log，但这种方式限制了缓存的使用方式，导致不得不缓存大量的脏数据直到事务提交。

- force/no-force
 - 如果采用 force 策略，在事务提交之前必须将修改的数据全部写到磁盘，这样做可以很好地保证数据库一致性，但会导致很多不必要的磁盘写入。
 - 如果采用 no-force 策略，可以在事务提交之后再把数据写入磁盘，但由于系统故障可能导致缓存中的数据丢失，所以需要记录 redo log，在系统恢复时重做已提交的事务。

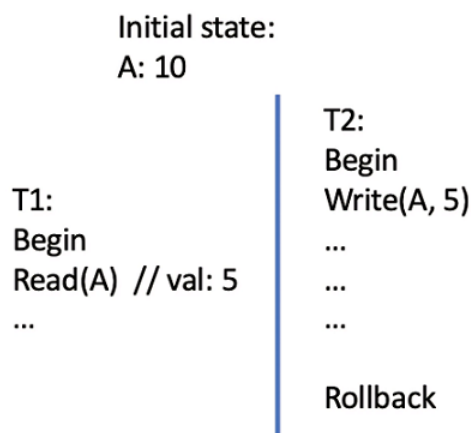
目前数据库常用的就是 steal/no-force 策略，既允许在事务提交之前将脏数据写入磁盘，也允许在脏数据全部写入磁盘之前提交事务，当然这种策略需要 undo/redo 日志来进行故障恢复。不过，undo/redo 日志虽然能够带来较高的性能，但是也会增加数据库从故障中恢复的时间，所以并非在所有情况下都适用。

5.6 隔离级别

数据库会有多种隔离级别，每种隔离级别都会带来不同的并发影响。

读未提交

读未提交，即在事务还没有提交的时候，其他事务就可以来读，条件非常宽松，但可能会出现以下这种情况。

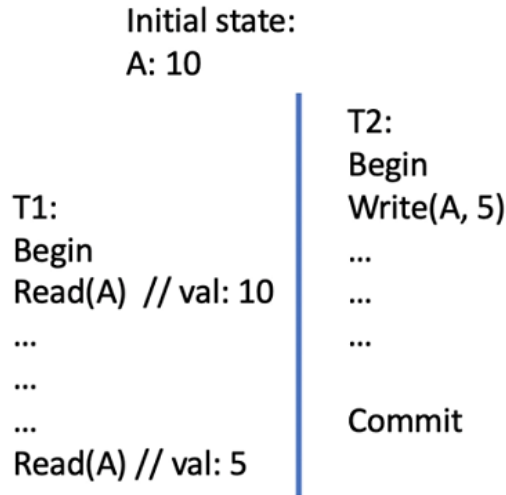


事务 T1 读的时候，事务 T2 还没有提交，T1 读到了数值 5，但 T2 最后回滚了，A 的值又变回了初始数值，这种情况就是脏读。

脏读（dirty read）指一个事务读取了另一个事务还未提交的修改。虽然大多数情况下，都会认为脏读产生了不正确的结果。但是脏读也不一定是错误的情况，有时在数据要求并发量很高的情况下，甚至可以允许产生脏读。

读提交

为了应对上文这种情况，我们引入读提交，只有事务提交后，才能读到正确的数值，但可能会出现以下这种情况。

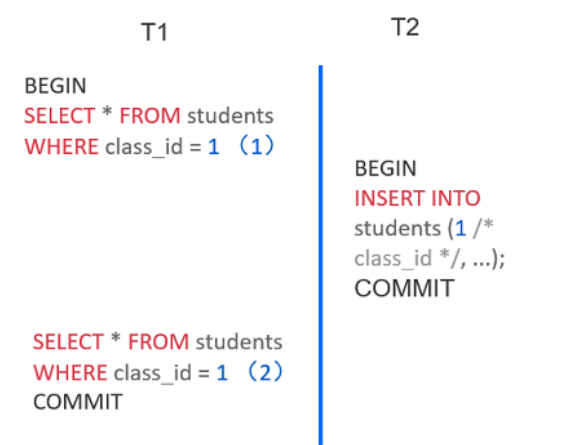


事务 T1 分别在事务 T2 提交前和提交后都读取了一次，两次读取的结果是不同，这其实违反了事务的隔离性，一个事务应该不需要知道其他事务的状态，这种情况叫做不可重复读。

不可重复度读（nonrepeatable read）指在一个事务过程中，可能出现多次读取同一数据但得到的值不同的现象。

可重复读

可重复读，根据上文这种情况，我们再加一些条件，在事务执行过程中，不需要另外的数据对这部分数据进行改写，简单的想法就是加一个读锁，这样就避免了不可重复读的情况，但可能会产生以下这种情况。



事务 T1 在执行过程中，事务 T2 新增了一个数据，因为这部分数据不在读锁限制的范围内，所以插入数据是成功，因此 T1 再查询的时候，发现满足条件的结果产生了改变，这就是幻读。

幻读（phantom read）指在一个事务中，当查询了一组数据后，再次发起相同查询，却发现满足条件的数据被另一个提交的事务改变了。

可有序化

根据上述的各种意外情况，最后我们让事务串行化，所有的事务必须按照一定顺序执行，直接避免了不同事务并发带来的各种问题。但这种方式代价也是巨大的，会导致数据库的并发能力特别差。

隔离级别	脏读	不可重复读	幻读
读未提交	可能出现	可能出现	可能出现
读提交	不能	可能出现	可能出现
可重复读	不能	不能	可能出现
可有序化	不能	不能	不能

5.7 锁

锁是一种解决数据库并发问题的一种设计。

如果一昧的加一种锁，比如全局锁，只要有事务来访问数据，就会申请一个全局锁，虽然从理论上来说，这样是可行的，但毫无疑问，这样的并发管理能力太差，所以需要一些更细粒度化的锁来进行并发控制。

细粒度的锁

将锁更细粒度化可以分为共享锁（share-mode lock; S-lock）和独占锁（exclusive-mode lock; X-lock），也就是读锁和写锁，事务进行读取的时候，获取数据的读锁，其他事务要修改必须要等待解锁。如果事务进行更新操作的时候，需要获取数据的写锁，只有该事务能对该数据进行读写操作。

只有读锁之间可以相互兼容，其他情况都是不兼容。

	S-lock	X-lock
S-lock	兼容	不兼容
X-lock	不兼容	不兼容

这里通过一个简单的示例来帮助大家理解。

- T1: B 转账给 A 50 元
- T2: 读一下 A 和 B 的余额之和

T1:

X-lock(B);
Read(B);
B = B - 50;
Write(B);
Unlock(B);
X-lock(A);
Read(A);
A = A + 50;
Write(A);
Unlock(A).

T2:

S-lock(A);
Read(A);
Unlock(A);
S-lock(B);
Read(B);
Unlock(B);
Display(A+B).

第一眼看上去感觉结果是合理，假设 A 和 B 原来都只有 100 元，事务 T2 无论在 T1 执行前还是执行后进行操作，读取的结果都是 200 元。

T1: X-lock(B); Read(B); B = B - 50; Write(B); Unlock(B); X-lock(A); Read(A); A = A + 50; Write(A); Unlock(A).	T2: S-lock(A); Read(A); Unlock(A); S-lock(B); Read(B); Unlock(B); Display(A+B).
--	---

这样 T2 在 T1 中间执行，锁之间的使用是合理的，这种情况下 T2 的结果就会是 150 元。

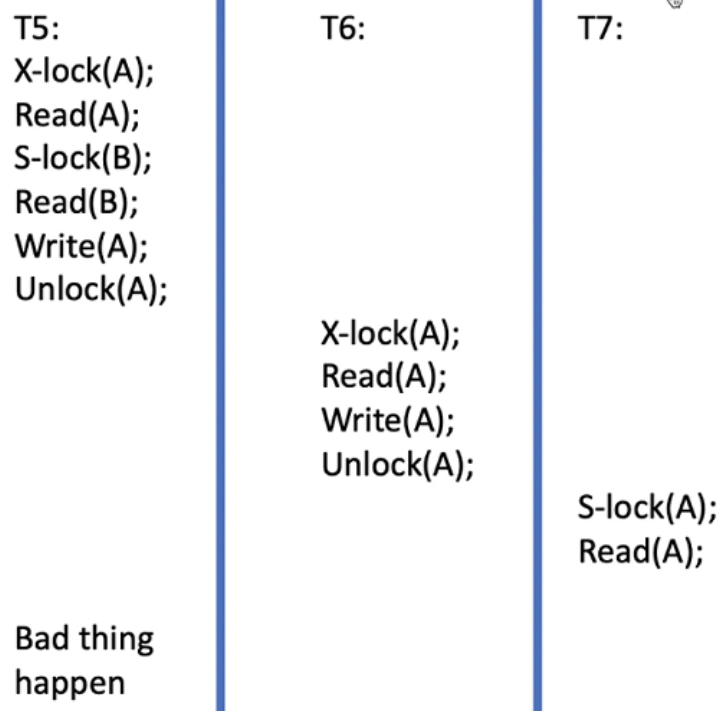
那怎么处理才能解决上面这种情况呢，很容易想到的就是事务在最后的时刻释放读写锁，这样看上去也很合理，但会出现下面这种情况：

T3: X-lock(B); Read(B); B = B - 50; Write(B); X-lock(A); Read(A); A = A + 50; Write(A); Unlock(B); Unlock(A).	T4: S-lock(A); Read(A); S-lock(B); Read(B); Display(A+B); Unlock(A); Unlock(B).	T3: X-lock(B); Read(B); B = B - 50; Write(B); X-lock(A);	T4: S-lock(A); Read(A); S-lock(B);
--	---	--	--

这种情况下，T3 占着 B 的写锁，T4 占着 A 的读锁，这个时候 T3 去申请 A 的写锁，T4 申请 B 的读锁，很明显，这种情况下死锁了。

因此为了避免这种情况，我们引入了两阶段加锁，在获取锁的阶段（growing phase），事务只能不断地获取新锁，在释放锁（shrinking phase）阶段，事务只能不断的释放锁。

两阶段加锁

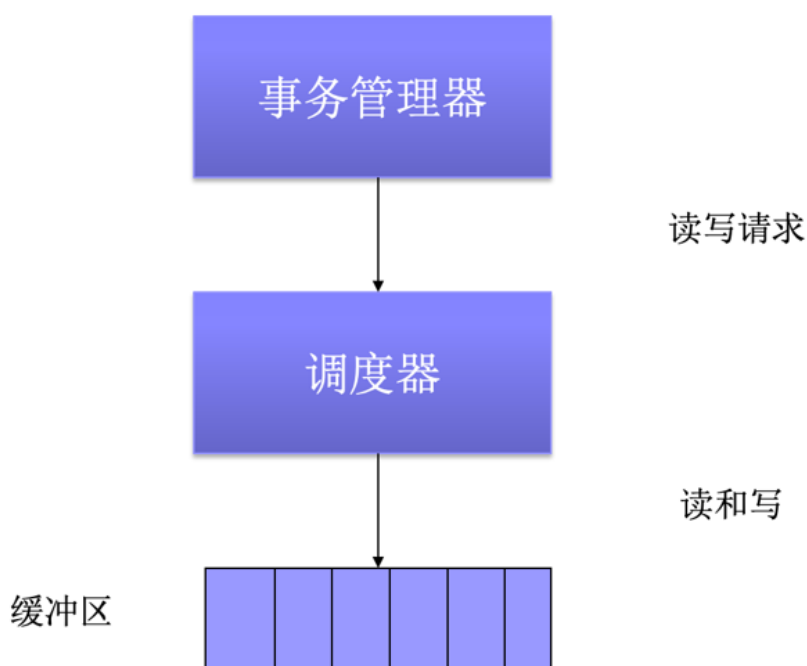


但同样也会遇到上述的情况，在 T5 遇到错误的时候，会导致 T6 和 T7 的连锁回滚。为了避免连锁回滚，我们引入更严格的两阶段加锁，写锁必须等事务结束后释放，这样就避免在事务未提交的时候，其他事务不会来进行数据改写，因此避免了连锁回滚。

5.8 时间戳

锁是一种比较悲观的并发控制逻辑，在某些情况下，锁并不合适。因此我们也引入一些比较乐观的并发控制逻辑，比如时间戳。

如果每个事务开启的时候，系统可以记录这个事务开启的时间 $TS(T_i)$ ，记为事务 T_i 的时间，通过比较不同事务的时间戳，给事务排序。



时间戳不一定是一种严格意义上的时间戳，主流数据库都是通过事务管理器（保证事务的时间戳不会一致）进行时间戳的管理，获取时间戳一般是通过一个调度器进行申请，调度器往往是单线程的，保证两个事务的时间戳不会出现相同的情况。

同样，时间戳也有类别，规定了两种对数据的时间戳，分别是数据 A 读时间戳和写时间戳，具体解释如下所示。

1) W-timestamp(A): 记录对于数据 A，最近一次被某个事务修改的时间戳。

2) R-timestamp(A): 记录对于数据 A，最近一次被某个事务读取的时间戳。

- 对于事务 T_i 要读取数据 A read(A):

1. 如果 $TS(T_i) < W\text{-timestamp}(A)$ ，A 被一个新的事务改写了， T_i 会被回滚。

2. 如果 $TS(T_i) > W\text{-timestamp}(A)$ ，说明 A 最近一次被修改小于 $TS(T_i)$ ，因此读取成功，并且 $R\text{-timestamp}(A)$ 被改写为 $TS(T_i)$ 。

- 对于事务 T_i 要修改数据 A write(A):
 1. 如果 $TS(T_i) < R\text{-timestamp}(A)$, 说明 A 已经被一个更大 TS 的事务读取了, T_i 对 A 的修改就没有意义了, 因此 T_i 的修改请求会被拒绝, T_i 会被回滚。
 2. 如果 $TS(T_i) < W\text{-timestamp}(A)$, 说明 A 已经被一个更大 TS 的事务修改了, T_i 对 A 的修改也没有意义了, 因此 T_i 的修改请求会被拒绝, T_i 会被回滚。
 3. 其他情况下, T_i 的修改会被接受, 同时 $W\text{-timestamp}(A)$ 会被改写为 $TS(T_i)$ 。

接下来看一个合理的时间戳调度机制示例, 如下图所示。

T1:
Read(B);
Read(A);
Display(A+B).

T2:
Read(B);
B = B - 50;
Write(B);
Read(A);
A = A + 50;
Write(A);
Display(A+B).

T1:
Read(B);

Read(A);

Display(A+B).

T2:

Read(B);
B = B - 50;
Write(B);

Read(A);

A = A + 50;
Write(A);
Display(A+B).

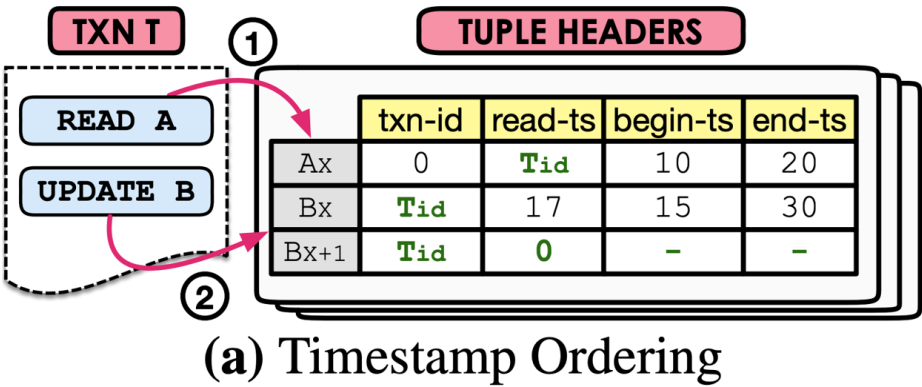
5.9 多版本并发控制

多版本并发控制（MVCC）是现在数据库领域内最主流的并发控制逻辑。

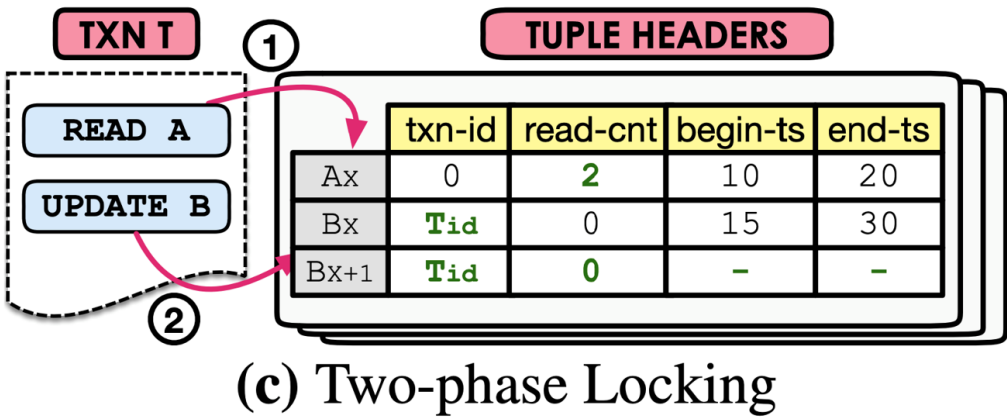
多版本并发控制有很多方面来讲，比如版本并发控制协议、版本垃圾回收、版本存储、版本索引等等。本节我们简单讲一下版本控制协议。

版本控制协议

基于时间戳的多版本控制协议：



基于两阶段加锁的多版本控制协议：

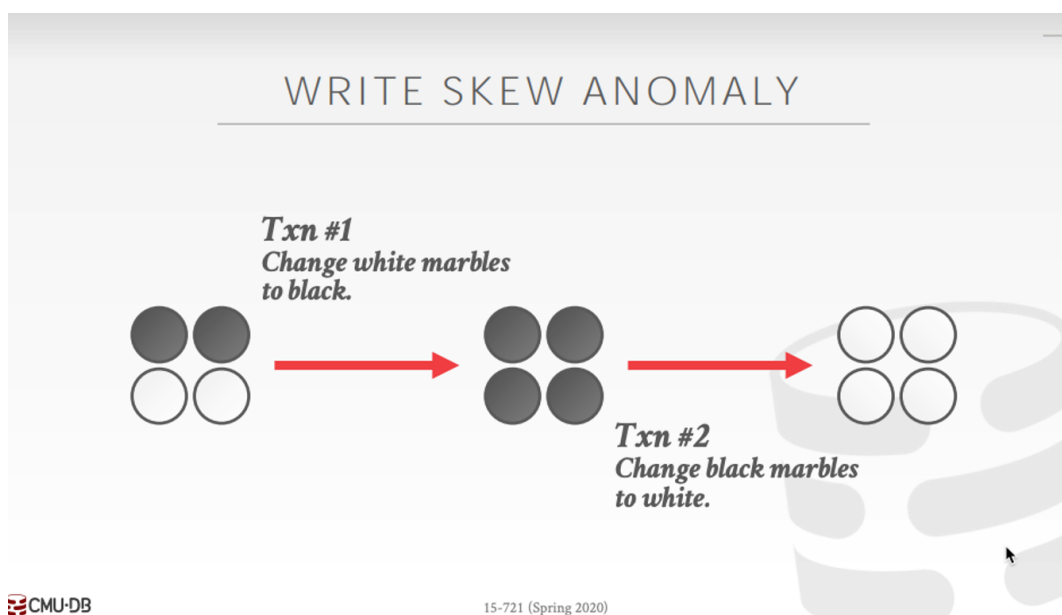
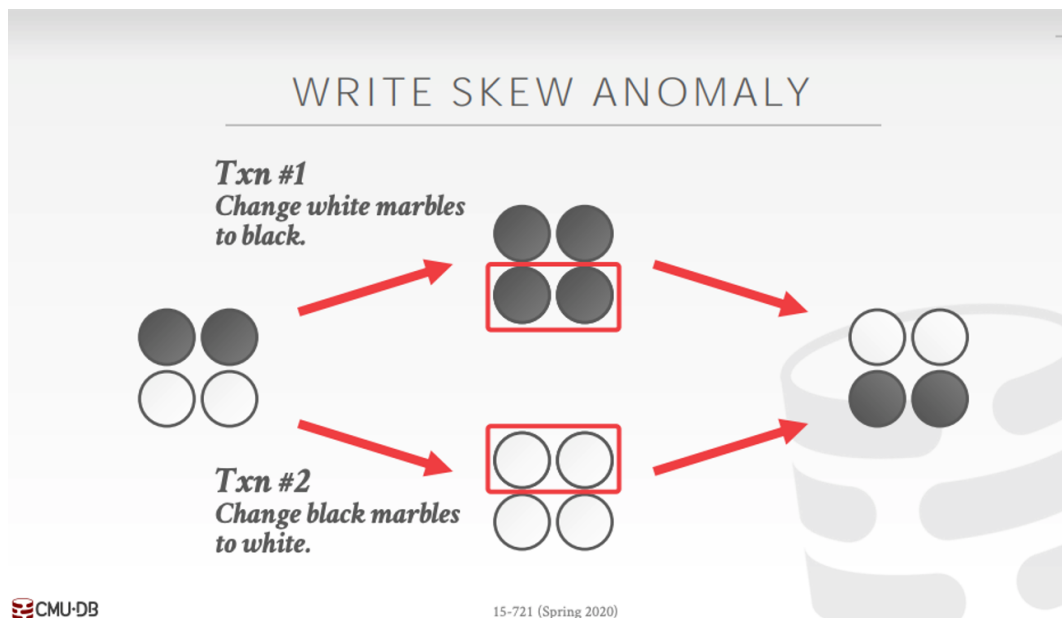


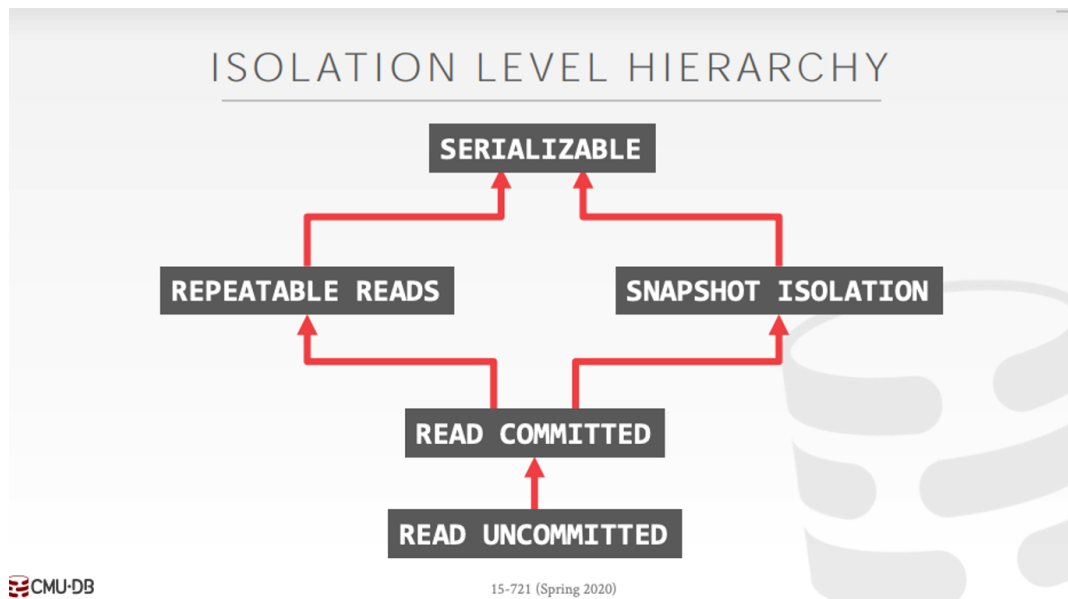
快照隔离

多版本并发控制（MVCC）是一种什么类别的隔离机制？很容易想到的就是有序化，将并发事务通过锁或者时间戳

进行编排，但其实 MVCC 是一种快照隔离。

这里引用 CMU 的 15-721 中的一个示例来进行说明，棋盘上有黑子和白子，如果事务 T1 是将白子变成黑子，事务 T2 是将黑子变成白子。如果按照 MVCC 的设计，棋盘上的颜色会反转，原来的白子变成黑子，原来的黑子变成白子。但如果是按照有序化设计，棋盘上最后会全是黑子或者全是白子，这取决于事务 T1 和 T2 的执行顺序。





5.10 课后实践

MiniOB 题目：支持一条语句插入多条数据，要同时成功或失败。

【训练营测试的原理】：后台执行时，有一系列的 SQL 语句，发送到 MiniOB，然后将执行输出的结果，跟预期的 result 文件做对比。

【练习方式】：完成后在 [训练营](#) 上提交测试，训练营使用文档请参见 [训练营使用说明](#)。